

SEJA UM PROGRAMADOR FODA!

MANUAL PARA UMA CARREIRA
DE SUCESSO

DEIBSON CARVALHO

Deibson Moraes de Carvalho

Seja um Programador Foda!

Manual para uma carreira de sucesso

1ª Edição

Rio de Janeiro/RJ
Edição do Autor

Copyright © 2020 Deibson Moraes de Carvalho

Seja um Programador Foda!

Manual para uma carreira de sucesso

Deibson Carvalho

Carvalho, Deibson

Seja um Programador Foda: Manual para uma carreira de sucesso -
1ª Edição - Deibson Carvalho - Rio de Janeiro/RJ:2020

ISBN 9798593594532

1. Programação de Computadores.
2. Computação, internet e Mídia Digital.

Todos os direitos reservados

Sumário

[SOBRE O AUTOR](#)

[INTRODUÇÃO](#)

[CAPÍTULO 1 - POR QUE VOCÊ QUER SER PROGRAMADOR?](#)

[1.1 - QUANTO GANHA UM PROGRAMADOR](#)

[1.2 - COMO É O MERCADO DE TRABALHO DE UM PROGRAMADOR](#)

[1.3 - O MERCADO DE TRABALHO OU O SALÁRIO DEVEM PESAR NA ESCOLHA PROFISSIONAL?](#)

[1.4 - PORQUE ME TORNEI PROGRAMADOR](#)

[1.5 - ENTÃO, ESTOU DECIDIDO, QUERO REALMENTE SER PROGRAMADOR, POR ONDE DEVO COMEÇAR](#)

[CAPÍTULO 2 - ANTES DE COMEÇAR A JORNADA](#)

[2.1 - ONDE ESTOU AGORA](#)

[2.2 - DEVO CONHECER PROFUNDAMENTE UM SISTEMA OPERACIONAL OU LINHA DE COMANDO PARA SER UM PROGRAMADOR?](#)

[2.3 - PRECISO SER BOM EM MATEMÁTICA PARA PROGRAMAR?](#)

[2.4 - TER INGLÊS FLUENTE](#)

[2.5 - PARA ADOLESCENTES OU JOVENS QUE ESTÃO NA FASE DE ESCOLHER A PROFISSÃO](#)

[2.6 - PARA PESSOAS QUE QUEREM MUDAR DE PROFISSÃO](#)

[CAPÍTULO 3 - POR ONDE COMEÇAR?](#)

[3.1 - SEU PERFIL PESSOAL](#)

[3.2 - QUAL CURSO ESCOLHER?](#)

[3.3 - AINDA ESTOU EM DÚVIDA SE FAÇO UMA GRADUAÇÃO](#)

[3.4 - EXISTEM BONS CURSOS LIVRES DE PROGRAMAÇÃO](#)

[3.5 - ESTOU MATRICULADO EM UM CURSO UNIVERSITÁRIO, PRECISO FAZER CURSOS LIVRES?](#)

[3.6 - O QUE PRECISO ESTUDAR, POR ONDE COMEÇAR?](#)

[3.7 - QUERO SER AUTODIDATA](#)

[3.8 - QUERO EMPREENDER. EXISTEM FORMAS MAIS RÁPIDAS DE APRENDER?](#)

[CAPÍTULO 4 - MUITO ALÉM DAS HABILIDADES TÉCNICAS](#)

[4.1 - TRABALHO EM EQUIPE, LIDERANÇA E COMUNICAÇÃO](#)

[4.2 - EMPATIA - EXPERIÊNCIA DOS USUÁRIOS](#)

[4.3 - CRIATIVIDADE](#)

[4.4 - PENSAMENTO CRÍTICO E RESILIÊNCIA](#)

[4.5 - FOCO NA SOLUÇÃO](#)

[4.6 - CONSTRUA SUA MARCA PESSOAL](#)

[4.7 - APRENDA A DIZER NÃO!](#)

[CAPÍTULO 5 - JÁ COMECEI, E AGORA?](#)

[5.1 - QUAL RAMO ESCOLHER, DEVO ME PREOCUPAR COM ISTO?](#)

[5.2 - QUAIS OS OPORTUNIDADES DE TRABALHO PARA UM PROGRAMADOR ATUALMENTE](#)

[5.3 - QUANDO TEREI QUE ESCOLHER A LINGUAGEM DE PROGRAMAÇÃO?](#)

[5.4 - ESCOLHA LINGUAGENS E FRAMEWORKS ATIVOS](#)

[5.5 - A LINGUAGEM DE PROGRAMAÇÃO IMPORTA PARA O SUCESSO DO PROGRAMADOR?](#)

[5.6 - APRENDENDO A USAR AS FERRAMENTAS](#)

[5.7 - O QUE É GIT](#)

[5.8 - O GITHUB](#)

[5.9 - ENTRE EM COMUNIDADES DE DESENVOLVEDORES](#)

[5.10 - PRATIQUE SUA HABILIDADE DE RESOLVER PROBLEMAS](#)

[5.11 - GERENCIAMENTO DE PROJETOS](#)

[CAPÍTULO 6 - ESCOLHENDO A SUA LINGUAGEM DE PROGRAMAÇÃO](#)

[6.1 - PARADIGMA DE PROGRAMAÇÃO](#)

[6.2 - PARA CADA FUNÇÃO TEM UMA LINGUAGEM ESPECIALIZADA](#)

[6.3 - PORQUE O PYTHON É UMA ESCOLHA CERTA PARA PROGRAMADORES INICIANTE](#)

[6.4 - PRECISO ESCOLHER UM FRAMEWORK?](#)

[6.5 - PRECISO ESTUDAR BANCO DE DADOS OU SQL?](#)

[6.6 - PRECISO UTILIZAR UM EDITOR DE TEXTO OU CÓDIGO, OU ATÉ UMA IDE?](#)

[CAPÍTULO 7 - JÁ ESCOLHI MINHAS FERRAMENTAS, AGORA COMO FAÇO PARA COMEÇAR A TRABALHAR](#)

[7.1 - COMPUTADOR PARA PROGRAMADOR](#)

[7.2 - CRIE SEU PORTFÓLIO](#)

[7.3 - PLATAFORMAS DE FREELANCERS](#)

[7.4 - REDES SOCIAIS](#)

[7.5 - CRIE SEU BLOG E PUBLIQUE ARTIGOS](#)

[7.6 - PARTICIPE DE DEBATES, DISCUSSÕES E RESPONDA PERGUNTAS NO GITHUB E STACKOVERFLOW](#)

[7.7 - ENCONTRE ESTÁGIO](#)

[CAPÍTULO 8 - PRIMEIRO PROJETO. POR ONDE COMEÇAR?](#)

[8.1 - SEJA PROATIVO](#)

[8.2 - MEU PRIMEIRO PROJETO SOLO](#)

[8.3 - PROJETANDO O SISTEMA](#)

[8.4 - STORYBOARD](#)

[8.5 - MODELO DE DADOS](#)

[8.6 - CRIE PROTÓTIPOS](#)

[8.7 - INICIANDO E DOCUMENTANDO O PROJETO](#)

[8.8 - POR ONDE COMEÇO MEU PROJETO?](#)

[8.9 - TEMPLATES DO PROJETO](#)

[CAPÍTULO 9 - SOU PROGRAMADOR, E AGORA, COMO ALAVANCAR MINHA CARREIRA](#)

[9.1 - ÉTICA PROFISSIONAL](#)

[9.2 - NÃO À PIRATARIA](#)

[9.3 - HACKERS](#)

[9.4 - MELHORE A VIDA DAS PESSOAS](#)

[9.5 - VALORIZE SEU TRABALHO](#)

[9.6 - FAÇA O SIMPLES](#)

[9.7 - SIGA AS BOAS PRÁTICAS DE PROGRAMAÇÃO](#)

[9.8 - ESCREVA TESTES PARA SEUS CÓDIGOS](#)

[9.9 - FAÇA PROGRAMAS SEGUROS](#)

[9.10 - TRATE TODAS AS POSSÍVEIS FALHAS](#)

[9.11 - ENCANTE SEUS CLIENTES](#)

[9.12 - DOE SEU TEMPO E TALENTO](#)

[9.13 - COMPARTILHE SEU CONHECIMENTO](#)

[9.14 - TENHA PACIÊNCIA!](#)

[9.15 - APRENDA A LIDAR COM DINHEIRO](#)

[9.16 - APERFEIÇOAMENTO CONSTANTE](#)

[9.17 - CUIDE DA SAÚDE](#)

[9.18 - PRODUTIVIDADE](#)

[CAPÍTULO 10 - É SÓ O COMEÇO DE UMA LONGA JORNADA](#)

SOBRE O AUTOR

Deibson Moraes de Carvalho é Bacharel em Ciências da Computação pela Universidade Federal Fluminense. Possui especialização em desenvolvimento web. É Bacharel em Direito pela Universidade Cândido Mendes. Possui Mestrado LLM em Mercado Financeiro, com ênfase em mercado de capitais.

Atua na área de desenvolvimento de *software* desde 2003. Seu primeiro grande projeto foi o ERP para a empresa Desk Móveis, uma indústria de móveis escolares, tendo concluído o projeto com cerca de 150 mil linhas de código em pouco mais de 1 ano, sendo o único programador da equipe. Iniciou o desenvolvimento web em 2006, utilizando PHP e *MySQL* para sites de todos os tamanhos, desde pequenos projetos locais a grandes sites.

Trabalha como freelancer desde 2008, quando abandonou o mundo corporativo, tendo trabalhado em diversos projetos em mais de 7 países, através de diversas plataformas de *freelancers*, utilizando linguagens como *Python*, HTML, CSS, *javascript*, *Node.js*, *React*, C++ e PHP, bem como os *frameworks Django*, *FastAPI*, *Laravel*, com os bandos de dados *PostgreSQL*, *MySQL* e *MongoDB*.

Desde 2013, participa de projetos para *Expert Advisors* para operações de *Day Trader* e *Swing Trader*, no mercado financeiro brasileiro e *FOREX*, tendo desenvolvido diversos robôs e indicadores, utilizando as linguagens MQL4 e MQL5, para a plataforma *MetaTrader*, e *Python* e R para análises estatísticas.

Atualmente, se dedica a estudar e se aperfeiçoar em aprendizado de máquina para análises preditivas quantitativas para ativos financeiros e aprimoramento de algoritmos para HFT desenvolvidos em C++.

Quer se dedicar a ensinar o que aprendeu em quase 20 anos de carreira, e pretende manter publicações regulares nas suas redes sociais e no seu blog pessoal deibsoncarvalho.com.br.

INTRODUÇÃO

Escrevi um livro que eu quero ler. Segui uma preciosa dica de *Austin Kleon*, dada em seu livro *Roube como um artista*. Quando eu tinha 15 anos, não sabia por onde começar. Na época, por volta dos anos 2000, a internet estava engatinhando aqui no Brasil e as informações eram muito escassas. Não tinha ninguém para conversar. Morava em uma cidade pequena, de origem humilde. Meus pais eram analfabetos. Então, como descobrir o que preciso fazer para seguir a minha paixão precoce por programação?

Então, eu queria ter lido este livro quando tinha 15 anos para me esclarecer como me tornar um bom programador. Como a maioria das pessoas, aprendi na marra, batendo a cabeça, apanhando da vida, ou seja, aprendendo com meus erros.

O livro que o leitor tem em mãos é quase tudo que aprendi em cerca de 20 anos de carreira, com altos e baixos, erros e acertos. Compartilhei com leitor o que eu queria ter ouvido quando era jovem e perdido.

Hoje os jovens estão imersos em informação. Há dezenas de cursos universitários e profissões para escolher, o que pode deixá-los confusos, ou deixarem uma escolha tão importante ao acaso, ao que dá mais dinheiro ou à profissão da moda.

Quantos jovens não querem se tornar *influencers*? Espero que o leitor queira algo a mais da sua vida. Queira viver com propósito.

Nada melhor do que levantar todos os dias para fazer algo que amamos e ajudar as pessoas a terem uma vida melhor.

O mundo precisa de programadores bons, dedicados e criativos. Espero com este pequeno e humilde livro estar colaborando para formar programadores mais qualificados e preparados para os novos desafios que batem à nossa porta.

Neste livro, abordo questões espinhosas como fazer ou não uma graduação. Faço algumas críticas às formações e grades curriculares atuais.

Eu estimulo o leitor a refletir porque quer se tornar programador, além de apresentar o caminho inicial para iniciar a sua jornada com o pé direito.

Nos capítulos finais, dou algumas dicas para o leitor iniciar, do tipo de por onde começar a projetar seus sistemas, estruturação de projetos bem como a documentação.

CAPÍTULO 1 - POR QUE VOCÊ QUER SER PROGRAMADOR?

“Não é tanto o que fazemos, mas o motivo pelo qual fazemos que determina a bondade ou a malícia.”

Santo Agostinho

Para mim é uma questão importante você saber por que escolheu o que quer fazer. Muitos, para não dizer a maioria, tomam esta importante decisão considerando fatores como dinheiro, fama, ou alguma métrica de vaidade, *status* etc.

Eles se esquecem que para você atingir a fama e ganhar dinheiro, independente da profissão que você escolha, você deve ser muito bom ou até excepcional no que faz. Há médicos bem remunerados, porém, há poucos médicos famosos e ricos. Assim, há professores, dentistas, engenheiros, consultores, pintores ricos e famosos.

Para mim o que determina se você vai ter sucesso na sua carreira, o menos importante é a profissão em si, mas sim como você a desempenha.

Para se tornar um bom engenheiro, com reconhecimento e renome, é preciso muito trabalho, dedicação e esforço, assim como qualquer outra profissão.

Assim, não seria diferente com a programação. Há programadores famosos e ricos, que com muito trabalho duro e esforço, e, também um pouco de sorte, atingiram o alto patamar, assim como há os medíocres, que não se dedicaram o suficiente, ou até não desejaram nada além do que conforto e estabilidade.

Então, se quer ser um programador foda, caia na realidade e reflita: **Por que quer se tornar programador?** Está ciente de que vai ter que estudar a vida toda? Está ciente de que as coisas não vão ser fáceis e que há um longo caminho a seguir?

Assim, se não tiver um motivo forte, provavelmente você desistirá, ou pior, se tornará mais um programador medíocre.

Sugiro a leitura do livro [Encontre seu porquê](#) e [Comece pelo porquê](#), do *Simon Sinek*, para te ajudar na sua tomada de decisão.

1.1 - QUANTO GANHA UM PROGRAMADOR

Afinal, programador ganha muito dinheiro? Qual o salário médio de um programador? Bom, como em qualquer profissão, os valores são variados.

Comecei minha carreira como programador júnior, em uma empresa industrial, com salário que na época era o equivalente a 4 salários mínimos, não me lembro exatamente do valor, mas com 18 anos, era um salário excelente.

Há sites e serviços de empregos na internet que divulgam média salarial ou salário inicial. Segundo o *guiadecarreiras.com.br*, a média salarial para programador júnior foi de R\$ 2.461,00 em 2020. O antigo *Love Mondays*, e atual *GlassDoor*, cita uma média de R\$ 2.388,00.

Encontrei vagas abertas para salários iniciais de R\$ 1.750,00 à R\$ 2.450,00, para trabalhar em empresas de diversos ramos. Já em casas de *softwares* e *startups* de tecnologia o salário inicial pode ser bem maior. porém, é um pouco mais difícil encontrar essas oportunidades.

Além disso, casas de *softwares* e *startups*, *geralmente* exigem conhecimento aprofundado em diversas tecnologias, linguagens de programação da moda, além de *frameworks*, entre outras. Sinceramente, na minha opinião, elas não valem o que pagam.

Em contrapartida, a experiência é enriquecedora e traz muita bagagem para o seu currículo.

Já como *freelancer*, a média salarial é de R\$ 40,00 a hora, segundo os sites de serviço de *freelancers Workana* e *99Freela*.

1.2 - COMO É O MERCADO DE TRABALHO DE UM PROGRAMADOR

A demanda por serviços de programadores tem aumentado muito. Há muitas oportunidades de trabalho em todas as áreas e a tendência é aumentar cada vez mais.

Afinal, a Programação é uma carreira em ascensão.

Em outros países, há programas de incentivo para atrair talentos e grandes empresas como o *Google*, fazem eventos para descobrir e contratar programadores habilitados.

Quando comecei não existiam tantas oportunidades. As empresas tinham departamentos que elas chamavam de CPD (centro de processamento de dados). Eu trabalhei em alguns onde programadores não eram registrados como tal, mas sim como técnico de informática.

Eles colocavam os técnicos de computadores, de impressoras, programadores, desenvolvedores todos no mesmo saco, e pagando o mesmo salário. Perdi as contas de quantas vezes tive que largar um programa no meio para consertar uma impressora, ou dar suporte para um usuário do *Windows*.

Porém, as coisas mudaram muito nos últimos 20 anos. As empresas reconheceram a importância de sistemas de informação, banco de dados, *sites de internet*, e até redes sociais, exigindo cada vez mais profissionais especializados em cada uma dessas áreas.

Hoje, vemos indústrias contratando programadores para web, exigindo conhecimento em linguagens específicas como PHP, ou até em *frameworks* como *CakePHP*, ou até a sensação do momento para programadores PHP, que é o *Laravel*.

Que evolução!

Na minha época, era vaga para programador júnior, e quando você ia para a entrevista, o entrevistador não sabia nada de programação, fazendo perguntas de *windows* ou de rede de computadores. Já me perguntaram se eu sabia fazer cabos de rede cross-over! Eu questionei o entrevistador se a vaga era para programador ou para técnico de rede.

Há ainda o crescente mercado *freelancer*, cada vez mais explorado por todos, mesmo empresas que possuem setores de desenvolvimento interno com equipe formada, contratam programadores especializados em uma determinada tecnologia que os membros de sua equipe não dominam.

Por exemplo, já fui contratado para escrever um *script* em *Python* para produzir e consumir filas de processamento de mensagens de *RabbitMQ* para um *endpoint* de uma API pública, porque os programadores internos não tinham experiência com este tipo de função.

Dependendo da tecnologia e da área que você escolher atuar, pode ter muita demanda.

Cada época tem seus protagonistas e coadjuvantes. Quando comecei, o queridinho da *web* era o *Macromedia Flash*, com seu *actionscript*. Eu cheguei a iniciar o estudo desta linguagem, mas não prossegui porque, como era uma tecnologia proprietária, cara e complicada estava fadada a ser superada.

Putz, que programador das antigas não se lembra do *ColdFusion*, outra tecnologia da mesma empresa que também naufragou? Atualmente, para *web* há os *frameworks* como *Vue*, *React*, *Laravel* entre outros que são sensação. Vejo muitos programadores aprenderem estes *frameworks* sem se aterem à linguagem ou a lógica que está por trás deles. Vamos falar mais sobre isto à frente. O que não muda nunca? As habilidades técnicas e complementares que te fazem um bom programador.

Em suma, há oportunidades incríveis e muito desafiadoras em quase todas as áreas, para programadores especializados. Desse modo, se você for bom e dominar as boas práticas e técnicas não precisará se preocupar com o trabalho.

1.3 - O MERCADO DE TRABALHO OU O SALÁRIO DEVEM PESAR NA ESCOLHA PROFISSIONAL?

Para ser sincero, eu penso que fatores como mercado de trabalho e salário não deveriam pesar na decisão de seguir ou não uma profissão. No entanto, sendo realista, para muitos são os fatores determinantes.

Eu citei anteriormente algumas considerações sobre vagas de emprego e salários iniciais, porém, sendo absolutamente sincero, quando escolhi ser programador eu não pensei nisto. Na verdade, nem sequer passou pela minha cabeça o quanto ganha um programador e onde ele trabalhava.

Contudo, cada um tem um jeito de pensar e de ver as coisas, ou seja, um paradigma. Então, o que não é importante para mim, pode ser para você, ou vice-versa.

Nasci em 1983. Sou da geração Y. Fui criado longe de celulares e computadores. Apaixonei-me pelos números e pela lógica de programação quando tinha 15 anos, por volta do final de 1998, quando estava terminando o antigo ensino fundamental, muito antes de ter um computador em casa.

Nesta época, estava na moda cursos de informática de *Windows*, *Word*, *Excel*, etc. Então, convenci minha mãe a me matricular nesses cursos. Como eu morava em cidade pequena, não havia muitas opções, e não existia curso de programação. Eu usava os computadores e a internet (quando tinha) dos cursos e, até o único computador da biblioteca municipal, para pesquisar e ler sobre o assunto.

Nunca pesquisei sobre a profissão ou quanto ganhavam. Apenas queria saber o que eles faziam, o que criavam, e, à medida que lia sobre o assunto, aquilo me encantava.

Então, eu escolheria esta profissão mesmo que não existisse mercado de trabalho, o que na minha cidade e região era realidade, pois, não existiam vagas de trabalho para esta área, e mesmo que o salário inicial fosse baixíssimo.

Se para você, leitor, um emprego ou o salário inicial é importante, deveria refletir se está é realmente a profissão que mexe com seu coração, se você é apaixonado por programar, ou criar coisas importantes.

Helena P. Bravatsky, uma brilhante filósofa russa do século XIX, em especial no seu pequeno e brilhante livro, [A Voz do Silêncio](#), ensina que uma das coisas mais importantes no ser humano é a vontade, ou seja, a motivação pela qual alguém faz algo.

Não digo que é errado querer saber onde você pode trabalhar ou quanto você vai ganhar, mas se estes são os elementos principais para sua decisão, será que você vai aguentar o tranco?

O que fará você aguentar as madrugadas, noites, finais de semana para se dedicar a aprender, escrever código e se atualizar?

Perdi as contas de quantas vezes passei horas sentado, escrevendo código. O meu recorde pessoal foi 30 horas, isso mesmo, passei quase 30 horas sentado, com a cara na tela do computador escrevendo um programa, só me levantava para ir ao banheiro, pois tinha ao meu lado biscoito e água. Não recomendo a ninguém fazer isto, pois, não é sustentável.

Odeio romantizar as coisas, ou criar ilusão!! Como aprendi a pensar como programador desde cedo, tenho um pensamento mais pragmático, direto, objetivo e lógico.

Então vamos direto ao ponto: se você não amar ser programador, será medíocre, ou desistirá pelo caminho!

Como já disse anteriormente, se você for excepcional, não precisará se preocupar com trabalho ou salário.

Na minha humilde opinião, se você realmente quer, e, se é apaixonado por criar, desenvolver, por resolver problemas, adora um desafio, encara a dificuldade com coragem, com toda a certeza, você será um excelente programador. Assim sendo, a única variável, na minha opinião, que deve ser considerada é a vontade.

1.4 - PORQUE ME TORNEI PROGRAMADOR

Quando eu era criança, adorava desmontar e montar coisas, saber como elas funcionavam, queria saber como as coisas se moviam, e adorava números. Eu desenhava prédios, casas, calculava distâncias, pesos, tamanho das coisas.

Em resumo, eu adorava descobrir o poder dos números. A minha melhor disciplina na escola era matemática. Eu fazia tabuadas o tempo todo. Era péssimo em disciplinas como português, literatura, história etc. Era péssimo decorador de datas, personagens e nomes de guerras.

Uma professora do antigo primário me indicou estudos complementares de matemática. Na terceira série do primeiro grau eu estudava por livros de ensino médio e de matemática complexos que conseguia na biblioteca da escola.

Foi quando tive o primeiro contato com a lógica de programação. Li um livro de Lógica e técnica de programação em *Pascal* e *Cobol*. Devorei livros de algoritmos antigos e de lógica de programação.

Nesta época eu não tinha computador em casa. Eu escrevi meus primeiros algoritmos em papel. Eu fiz cursos de MS-DOS (*Microsoft Disk Operating System*), e li tudo que encontrava deste sistema operacional, e me deparei com a linguagem *Shell Script*, juntamente com todos os comandos do DOS.

Eu adorava ter contato com computador. Adorava aquela tela preta do DOS. Aos que se lembram, ou com mais de 30 anos, vão se recordar do *Windows 3.11*, no qual se acessava, digitando “win” na linha de comando do DOS.

Só tive contato com o *Linux* mais tarde quando comprei um livro de conhecimento avançado de sistema operacional e técnicas *hacker*.

Eu mantive a paixão até conseguir comprar meu primeiro computador, quando completei 16 anos e arrumei meu primeiro emprego. Como eu era de família pobre, meus pais não podiam comprar um computador.

Nesta época eu pensava que seria engenheiro civil, pelo meu amor aos números, vontade e desejo de criar e construir algo.

Na minha cidade havia uma escola politécnica, que era muito boa por sinal, e que hoje em dia não existe mais (os políticos conseguem acabar com tudo que é bom). Nela havia o curso de Processamento de Dados, e para ter acesso ao curso, havia uma prova de seleção.

Então, fiz a prova e fiquei entre os 10 primeiros de todos os cursos, o que me possibilitou concluir ensino médio técnico em processamento de dados. Confesso que não aprendi muito sobre programação em si, mas me deu uma boa base para estrutura de dados e algoritmos.

Quando comprei meu primeiro computador, passei a programar feito louco. Escrevi programas em *Visual Basic*, C, e tive meu primeiro contato com *Python*. Eu gostava tanto de fazer aquilo que deixava de sair com amigos para ficar em casa o final de semana programando.

Já passei várias madrugadas coladas na tela de tubo de 14” com a vista doendo escrevendo códigos. Eu pagaria para fazer aquilo de tanto que eu amava fazer.

Não importava se eu era pobre, ou se não tinha ninguém para me ajudar, eu iria seguir aquela carreira.

Então, resumindo, me tornei programador por amor.

1.5 - ENTÃO, ESTOU DECIDIDO, QUERO REALMENTE SER PROGRAMADOR, POR ONDE DEVO COMEÇAR

No judaísmo, a pessoa se torna judeu se nasce do ventre de uma mulher judia. Caso você não tenha ascendência judia, como não se trata de uma religião missionária, e, você queira muito se tornar um judeu reconhecido pela comunidade, o Rabino irá te testar de todas as formas, e tentará de dissuadir a se tornar um.

Eu acho muito interessante isto, ou seja, para ser um judeu, o gentio deve batalhar muito e merecer ser reconhecido como um integrante do povo escolhido.

Entendo que assim são boas oportunidades. Elas devem ser merecidas e conquistadas. Pois, quase nada que venha fácil, ou lhe seja oferecido é de fato bom ou pode te trazer bons frutos.

Não escrevi este livro para te dissuadir a ser um programador, mas para te mostrar um pouco como é a vida de um programador, e fazer você refletir se realmente é isto que você quer. Não sou muito religioso, mas, como cristão, acredito que a nossa vida é uma dádiva divina, um presente dado por Deus, e que cada dia de vida deve ser agradecido.

Desse modo, penso que não devemos desperdiçar a nossa bela vida com coisas, profissão, emprego, ou pessoas que não valham à pena.

O prazer de acordar para fazer algo que você ama é indescritível. Mesmo quando o “bicho pega”, ou quando devemos “apagar um fogo”, estamos lá felizes, fazendo o que acreditamos que nascemos para fazer.

Se tem dúvida quanto à programação, não deixe de ler o livro na íntegra, pois, mais à frente, vamos ver os diversos ramos de atuação e funções que você pode exercer, e o quão maravilhoso é o universo do desenvolvimento.

Neste capítulo, tentei te mostrar de forma sucinta que o caminho para se tornar um excelente programador é tortuoso, cheio de altos e baixos. Você enfrentará muitos desafios, e precisará de uma razão muito maior do

que dinheiro, fama ou *status* para conquistar altos patamares e o desejado sucesso profissional.

Mas, cá entre nós, qual profissão tem caminho fácil? Nem atores, cantores ou jogadores de futebol famosos tiveram vida fácil. Leia biografias de pessoas bem sucedidas, e verá como o caminho delas foi espinhoso.

Nos demais capítulos iremos abordar outras nuances sobre esta magnífica profissão, além dos caminhos que você pode seguir.

No próximo capítulo vamos avaliar onde você está e aonde quer chegar.

CAPÍTULO 2 - ANTES DE COMEÇAR A JORNADA

“Os ramos da árvore são sacudidos pelo vento; o tronco permanece imóvel.”

Helena Blavatsky

Como exposto no capítulo anterior, o caminho não será fácil, mas garanto que será recompensador. A jornada de um programador é longa e por vezes cansativa, e deve ser trilhada por inteira, sem pegar atalhos. Mas, afinal, que jornada profissional é fácil!?

A programação requer constante estudo e atualização. Temos novas tecnologias, atualizações ou lançamentos de novas linguagens de programação e *frameworks* todos os dias.

Assim, se você não se atualizar constantemente, fica para trás.

Não vamos abordar nenhuma linguagem de programação em específico para não comprometer a escolha do leitor. Vamos apenas indicar fatores que podem pesar na escolha dentre centenas de linguagens para se especializar.

Falaremos mais sobre isto mais a frente.

Então, vamos definir qual seu estado atual e alguns conhecimentos gerais desejados para um bom programador.

2.1 - ONDE ESTOU AGORA

Presumo que se você se interessou em ler este livro é porque já teve contato com a programação em algum nível ou com algum programador, e desenvolveu o interesse pela profissão, certo? Além disso, não tem certeza se quer seguir o seu interesse, ou por onde começar.

Você já programou? Já tentou aprender algo sobre alguma linguagem? Já fez algum curso pago, ou gratuito no *YouTube*? Se já, talvez tenha que desaprender algo, ou voltar um pouco para aprender o essencial para ser um programador.

Você tem conhecimento de algum sistema operacional ou plataforma? Sabe ou conhece linhas de comando como *Shell Script* ou comandos de algum sistema operacional? Você usa o *Windows*? Vamos fazer um teste rápido:

Abra o *prompt* de comando do *windows*. Você conhece os principais comandos de gerenciamento de arquivos, ou de rede? Se você esquecer algum comando, sabe como obter ajuda? Com o *Prompt* de Comando aberto, digite *help* e aperte a tecla *enter*. Com o comando executado, abrirá uma lista de comandos com a descrição do que ele faz. Para saber o que um comando faz e quais os parâmetros, como por exemplo o comando *CD*, digite *help CD* e tecle *enter*. Aparecerá um texto de ajuda com a descrição e execução, recomendação, e lista de parâmetros do comando.

Se sentiu confortável em executar comandos tão simples sem o mouse, somente com o teclado? Espero que sim, porque ao longo da jornada de desenvolvedor, você utilizará muito a linha de comando, ou interfaces do tipo CLI.

O intuito aqui não é ensinar a usar a linha de comando ou os comandos do sistema operacional *Windows*, mas demonstrar que programadores devem ter familiaridade com linha de comando. Na verdade, programadores experientes somente usam linhas de comando. Os mais *hardcores* usam o *Linux* como o sistema da estação de trabalho, e alguns que conheço.

Ao longo do livro, vamos abordar mais sobre linhas de comando, e em que você precisará usá-la, e como aprender da forma correta.

Você é bom em matemática ou lógica? Tem facilidade em lidar com números? Bom se você não for bom em matemática, deverá aprimorar seus conhecimentos se quiser se tornar um bom programador.

2.2 - DEVO CONHECER PROFUNDAMENTE UM SISTEMA OPERACIONAL OU LINHA DE COMANDO PARA SER UM PROGRAMADOR?

Não precisa, mas um conhecimento aprofundado em um determinado sistema operacional ou plataforma que irá atuar, é recomendado. Conhecimento mínimo é requisito.

Todas as plataformas possuem o SDK, ou o kit de desenvolvimento para a plataforma. Se você for escrever programas para o *windows*, provavelmente precisará estudar e aprender o seu SDK.

Mas, calma, pois, estes conhecimentos já são avançados, não sendo exigidos de programadores iniciantes, mas se você conhecer o sistema operacional, já sairá na frente.

Ao longo de sua jornada de aprendizado você irá se deparar com diversos comandos a serem executados no *prompt*, como automatizadores de *scripts* de distribuição ou compilação de programas. Então, se você não conhecer o mínimo de *Shell Script*, provavelmente não entenderá diversos comandos a serem executados, ou será um simples decorador de comandos, ou copiador de comandos pré-definidos na *internet*. Para você ser um programador de verdade, terá que criar seus próprios *scripts* adequados à realidade do seu programa.

Além disso, conhecimento de uma determinada plataforma será de grande valia, podendo até ser um diferencial. Se você conhecer as nuances e a arquitetura do sistema operacional, quando for escrever programas, terá mais facilidade, além de escrever códigos com mais qualidade e mais eficazes, e, melhor ainda, com menos erros.

Se você for um programador web, por exemplo, precisará conhecer profundamente os protocolos mais utilizados na web, como HTTP, HTTPS, além do SSH, entre outros. Há muito material sobre estes protocolos. Vamos abordar cada um no seu determinado tempo para não embolar o meio de campo.

Se for escrever aplicativos para celular, pela plataforma *Android* ou *IOS*, precisará conhecer as suas respectivas arquiteturas básicas. Pode-se

utilizar *frameworks* multiplataformas, ou até algum tipo de abstração, mas se for necessário *features* mais avançadas, até efeitos diferenciais, ou a criação de algo novo, você terá que conhecer a fundo a arquitetura e desenvolver de forma nativa.

2.3 - PRECISO SER BOM EM MATEMÁTICA PARA PROGRAMAR?

Programação é lidar com algoritmos e cálculos. A matemática elementar ensinada na escola já é o suficiente para programar. Conhecer as regras básicas da matemática, bem como as operações já atende aos requisitos mínimos.

Dependendo do ramo que você atue, pode ser necessário estudo de pré-cálculo e cálculo. Na graduação de Ciências de Computação, na grande maioria das universidades brasileiras que oferecem o curso, possuem as disciplinas de cálculo I, II e III na grade curricular. Se você for lidar com *Machine Learning* ou *Deep Learning*, precisará de conhecimentos matemáticos mais avançados para cálculos mais complexos.

Eu sempre amei jogos de videogame. Cheguei a pensar em me tornar programador de jogos, mas não fui adiante, pois, preferia a web e sistemas comerciais e financeiros. Para se tornar um desenvolvedor de jogos exige-se conhecimento muito aprofundado em matemática e algoritmos complexos, física de movimento, além de ter que aprender aos *frameworks* dos pacotes de desenvolvimento de games, como *directx*, entre outros, dependendo da plataforma do game, ou até ao SDK do *PlayStation* ou *XBOX*.

Além disso, é quase impossível produzir um game sem um estúdio que envolve outros profissionais, como produtores, roteiristas, designers, técnicos de som, e no Brasil não conhecia nenhum deste tipo. Até tentei reunir uma equipe no antigo *Orkut* (quem se lembra desta falecida rede social) para desenvolver uma ideia de game, mas não foi adiante. Neste caso, para escolha do ramo levei em consideração o mercado de trabalho, e, a minha preferência eram outros tipos de sistemas.

É indispensável o conhecimento de lógica, e o desenvolvimento do raciocínio lógico matemático.

Não acho necessário você dominar álgebra linear ou cálculo para ser um bom programador, mas se tiver será um diferencial e você estará treinado e preparado para problemas complexos que exigirão muito cálculo para serem resolvidos.

Então, a resposta objetiva é: depende do ramo no qual pretende atuar. Se não sabe, é bom descobrir o quanto antes para treinar, e aprimorar o seu raciocínio lógico-matemático.

Mas, caso queira um bom livro para estudar e para consulta, vou indicar um livro didático utilizado em muitos currículos, [Fundamentos matemáticos para a ciência da computação: Matemática Discreta e Suas Aplicações](#), de *Judith L. Gersting*, o preço é salgado, contudo, vale cada centavo.

Caso você queria pular esta etapa, tudo bem, por sua conta e risco, mas certamente, mais a frente, na sua jornada por aprender uma linguagem de programação, ou a interpretar um algoritmo e na implementação, terá que aprender o que deixou para trás, ou será um decorador, ou copiador de códigos prontos *web*.

Como disse, o intuito deste livro é fugir do trivial, ou da formação rasa ou medíocre, e ajudar ao leitor tomar a decisão de se tornar um profissional foda e fora de série.

Caso você sinta dificuldade em aprender matemática, ou em ter um raciocínio mais lógico, não creio que isto seja empecilho para você se tornar um programador, mas limita as áreas de atuação. Para este caso, há um livro que indico para alunos ou pessoas com dificuldade em aprender matemática que é o [Aprendendo a aprender](#), da professora Barbara Oakley.

2.4 - TER INGLÊS FLUENTE

Infelizmente, sem saber falar, ler e escrever inglês dificilmente você irá muito longe como programador. Os melhores conteúdos sobre linguagens de programação e algoritmos são inscritos em inglês. Os melhores vídeos tutoriais, seminários, palestras sobre programação, tecnologias, plataformas e padrões de desenvolvimento são em inglês. As traduções costumam ser muito ruins.

As documentações de APIs públicas, plataformas e linguagens de programação são em inglês também. Há diversos cursos online gratuitos distribuídos pelas melhores universidades do mundo em inglês, então, ser fluente na língua do Tio Sam é quase uma exigência.

Quando eu ainda estava aprendendo inglês, perdi alguns contratos e oportunidades para participar de projetos estrangeiros por não ser fluente. Deixei muitos trabalhos à época por não conseguir realizar uma reunião pelo *Skype* ou responder um *email*.

Então, se você não tem fluência na língua anglicana, é bom priorizar este conhecimento o quanto antes. Quando você se apresenta como programador já está implícito que você é fluente em inglês.

Quando for comprar livros de programação, prefira os originais. Evite comprar livros traduzidos. As traduções em geral são feitas por programadores, mesmo assim às vezes são péssimas. Então, beba direto da fonte.

Eu aprendi inglês sozinho. Sou autodidata. Cheguei até a me matricular em cursinhos de inglês, mas não me adaptei a nenhum método de ensino, e concluí que levaria anos para atingir o nível básico de fluência. Mas, cada um tem a sua forma de aprender e de estudar.

Se para você for necessário curso presencial, ou um curso online, faça o quanto antes, e priorize cursos rápidos. As palestras de programação são fáceis de entender, não necessitando nível elevado de fluência para a completa compreensão.

2.5 - PARA ADOLESCENTES OU JOVENS QUE ESTÃO NA FASE DE ESCOLHER A PROFISSÃO

Caso o leitor seja adolescente, e esteja no momento primordial de escolha da sua profissão, espero que este livro seja um bom guia para ajudá-lo a fazer uma boa opção. No próximo capítulo vamos abordar qual curso de graduação você pode fazer e se é necessário concluir um para se programador.

Você é bom em matemática, não é? Eu também era. Lembra de logaritmo, potenciação, espero que sim, pois vai usar muito.

Como conheceu a programação? Seu pai é programador? Meu filho me perguntou certa vez sobre a programação e manifestou o seu interesse pela profissão, por me ver em casa, no meu escritório trabalhando, escrevendo meus códigos, e não saber o que queria fazer.

Mas, eu percebi que ele somente queria seguir a profissão por minha causa, e não por amor. Então dei-lhe um livro denso de Algoritmo, e ele odiou (ele não falou, mas a cara denunciou). Depois disso ele desistiu. Cada um tem que seguir o seu caminho e a sua vocação.

Você se interessou pela programação por assistir a algum *influencer* no *YouTube*? Bom, eles ganham mais dinheiro com seus vídeos do que com a programação em si. Além disso, quase todos são muito jovens (só conheço uns 2), não tendo tanta experiência.

Recomendo o canal do Cursos em Vídeos, com o Gustavo Guanabara, pois ele tem uma mente mais generalista e sabe falar a língua dos jovens para ensinar.

Desculpe-me pelas presunções acima, mas são o que mais ouço os jovens falar. Ou que conheceram um programador rico, ou melhor: querem ser os próximos *Bill Gates* e *Mark Zuckerberg*, ou querem ficar ricos com a programação. Eu participei da campanha “*Google compra meu site*”. Bons tempos, nesta época, da qual não me recordo o ano, o *Google* estava comprando vários sites menores e incorporando os desenvolvedores/fundadores.

Muitos dos atuais bilionários da *Forbes* eram desenvolvedores. Os fundadores do *Google* *Larry Page* e *Sergey Brin*, *Bill Gates* da *Microsoft*,

entre outros.

Hoje vejo disciplinas de empreendedorismo nos currículos dos cursos de desenvolvimento e programação, e é muito bom, pois, para mim, programadores amam criar, empreender, desenvolver coisas para melhorar a vida das pessoas.

Não me leve a mal jovem leitor, mas se o que te motiva para ser programador é puramente material, ou seja, ficar rico, talvez você não consiga aguentar a jornada, os percalços e dificuldades do caminho.

Estes grandes empreendedores que também eram programadores tinham habilidades que iam além da matemática e da lógica, além de muita vontade de criar coisas grandes que iam além da riqueza pessoal. Leia a biografia deles, entenda como eles se tornaram, o que são e as dificuldades que tiveram que enfrentar.

2.6 - PARA PESSOAS QUE QUEREM MUDAR DE PROFISSÃO

Caro leitor adulto, independente da sua idade e profissão, todos podem aprender a programar e serem excelentes desenvolvedores. Conheço alguns engenheiros, arquitetos que mudaram de profissão, se tornando desenvolvedores e empreendedores.

Como já falei antes, este livro é destinado à pessoa que queira exercer a profissão de desenvolvedor de *software*.

Vejo atualmente muitas pessoas se interessando em programação por necessidade específica, como lidar com *bots* de atendimento, ou robôs do *Telegram* ou *Instagram*.

Recentemente, lançaram curso de formação de *Brokers* (executores de ordens de compra e venda de ativos financeiros, ou *Traders* institucionais) onde havia no currículo noções de programação em *Python* para automatização de ordens.

Hoje podemos automatizar quase tudo. Desde macros escritos em VBA no *Microsoft Excel* a *scripts Python* para tarefas repetitivas. Vivemos na era da automatização. Já escrevi um *plugin* para o *WordPress*, em PHP, para automatizar funções específicas de administração de *blogs*. Já fiz *script* para automatizar consultas de cotação de preços em diversos sites em uma planilha de *Excel*, ou diversos outros tipos de automatizações.

Se o intuito do leitor for aprender o básico de programação para desenvolver um *script*, ou um código simples para automatizar alguma função mecânica, ou até criar um *bot* para *Telegram*, este livro não irá te ajudar muito.

Este livro se destina ao profissional que queira de fato se tornar programador, ou desenvolvedor de *software* capaz de criar grandes projetos de forma profissional e uma carreira excepcional.

Definir onde você está é muito importante para saber por onde começar, evitar pular etapas, e seguir o caminho que te levará mais rápido e com qualidade ao desejado sucesso.

Caso você ache que não tem o conhecimento de matemática ou lógica suficientes para programar, não deixe que isto o impeça, estude em paralelo com a disciplina de algoritmos e estrutura de dados, de acordo com a necessidade, e as situações cotidianas de programação, ou seja, os pepinos vão te ensinar muito. Quantas vezes não perdi 2 ou 3 dias para achar um *bug* em um código que era um simples sinal de + ou - no lugar errado, ou uma conta que dava o resultado errado.

Do mesmo modo, se não tiver fluência no inglês, force-se a aprender, a ler as documentações em inglês. Eu sempre escrevo a documentação do meu código em inglês, e leio livros direto do original, sem tradução.

Os conhecimentos básicos elencados neste capítulo são desejados, mas eles não o impedem de se tornar programador, apenas ditará se você será um excelente programador, ou um mediano.

Acredito que todos que escolham de coração ou por vocação uma profissão, e não apenas pelo acaso, ou por falta de opção, irão atingir o tão sonhado sucesso profissional. Aprendi uma coisa com minha experiência: não existem atalhos para o sucesso.

No decorrer da leitura, indicarei livros, cursos, vídeos e palestras para estudo. E, mesmo que você seja universitário, cursando alguma graduação, não se engane, precisará seguir este mesmo caminho com muita dedicação.

CAPÍTULO 3 - POR ONDE COMEÇAR?

“O operário que quer fazer o seu trabalho bem deve começar por afiar os seus instrumentos.”

Confúcio

Tomada a decisão de seguir a carreira de programação, identificados os requisitos e as habilidades necessárias, vamos discorrer sobre quais caminhos tomar e quais evitar.

Nos primórdios, antes de existirem as universidades, o ofício era passado de pai para filho. O pai sapateiro ensinava o filho que também se tornava sapateiro, quase que por obrigação. Recentemente assisti a um documentário no *History Channel*, sobre armas antigas.

No documentário, falava-se da arte antiga de forjar espadas ou de criar arcos de batalha, e falou-se que a arte de fabricar os famosos arcos mongóis era passada de pai para filho e não havia registros de como eles eram feitos, tendo a arte morrido com os últimos artesãos.

Atualmente, ainda há ofícios que são passados de pai para filho, como sapateiros, relojoeiros, etc. Meu falecido pai, que era eletricista predial, e que nunca assistiu a uma aula sequer, nem tinha formação técnica, era um excelente profissional, e, tentou passar a sua arte para mim.

Não exerço a profissão de eletricista predial porque temos opção hoje em dia de escolher uma entre um imenso leque de profissões, e podemos seguir nossa vocação e o nosso coração.

Fiz graduação em Ciência da Computação por uma universidade federal, colando grau de Bacharel. Quando fiz o vestibular, no ano de 2004, tinha apenas o curso de sistemas de informação e Ciências da Computação. Hoje em dia, existem diversos cursos de licenciatura, graduação e de tecnólogo.

O curso foi essencial para a minha formação de programador?

Sendo absolutamente sincero, não. Para ser honesto, o curso foi uma droga. Talvez, tenham melhorado a grade curricular, ou a ementa do curso,

mas não se engane, cursar uma graduação não o tornará um programador melhor.

Entretanto, meu curso de graduação trouxe *networking*, amigos, conhecidos na área, além de troca de experiências, vivência, pois leva-se de 4 a 5 anos para se formar, e neste período você tem a oportunidade de crescer como pessoa e como profissional.

A profissão de programador não é regulada, assim não exige habilitação específica para exercê-la. Assim, você pode ler um livro de “como programar” e já aceitar um trabalho para desenvolver um sistema, aplicativo ou site.

Na verdade, acredito que a maioria dos profissionais que oferecem trabalho nas plataformas de *freelancers* não tem graduação, e muitos são autodidatas.

Para ocupar vagas formais em empresas ou em casas de desenvolvimento de softwares eles exigiam, e algumas ainda exigem a formação universitária, mais por pura formalidade do que por qualidade, por assim dizer.

A universidade não dá bagagem suficiente para você se tornar um profissional completo. Na verdade, se você ficar somente com o que vê nas aulas será um profissional abaixo do mediano, para não dizer pior.

Os professores em geral são bons, acadêmicos respeitados, que publicam artigos e excelentes pesquisas. Entretanto, eles não têm tempo suficiente para ensinar tudo o que um aluno precisa aprender para se tornar um bom profissional.

Durante minha graduação, eu estudava muito. Não tinha grana para comprar todos os livros, mas conseguia-os emprestados, tirava fotocópias das páginas ou até de livros inteiros. Na internet havia bastantes livros técnicos em inglês, além dos materiais gratuitos disponibilizados pelas equipes técnicas das linguagens ou tecnologias.

Não faltou material para estudar. Então, eu usava as aulas para validar o que aprendia nos livros e solidificar o conhecimento. Mesmo estudando, consegui trabalhos onde eu podia praticar o que estava aprendendo.

Posso afirmar que muito do que sei hoje aprendi fora da faculdade. Se você quer aprender algo sobre desenvolvimento, deve meter a mão na massa, trabalhando para uma equipe ou profissional mais experiente.

Então, o diploma emoldurado na minha parede não me torna ou tornou programador. Na verdade, não uso ele para quase nada, a não ser quando preciso me apresentar, entretanto, meus projetos, *softwares*, aplicativos e sites criados falam muito mais do que o diploma.

Contudo, se deseja seguir carreira acadêmica, de pesquisador, publicador de livros ou de artigos acadêmicos, professor universitário ou ainda, ser um cientista de dados, recomendo cursar uma boa graduação de ciências da computação.

Ainda assim, recomendo que você curse uma boa universidade na área, dentre tantos cursos atualmente oferecidos, mas ela não te tornará programador. Você pode se tornar programador muito antes de se formar. Utilize-a como um trampolim para novos contatos, *networking* e experiência acadêmica.

3.1 - SEU PERFIL PESSOAL

Cada pessoa tem o seu perfil, independente da profissão que possui. Conhecer o seu perfil, requer autoconhecimento. Se você não sabe, recomendo fazer um teste DISC, que é uma metodologia de avaliação comportamental com objetivo de identificar o perfil dominante do indivíduo. Os quatro grupos são: dominância, influência, estabilidade e cautela. Meu perfil é de dominância.

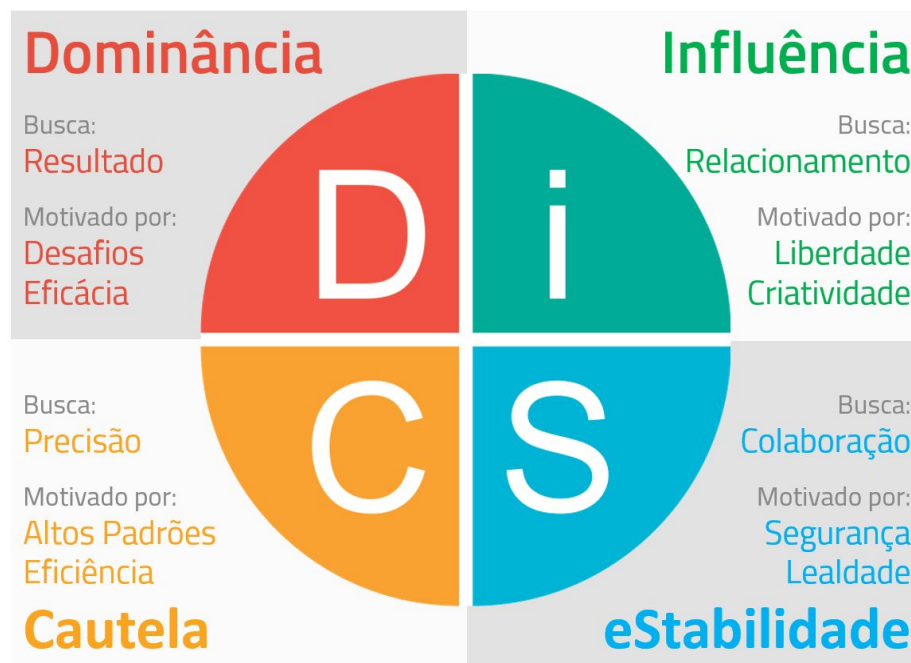


Figura 1 - Imagen extraída de <https://medium.com/@joaopedroguimaraes/teste-disc-ladainha-e-perigo-3a0195bb4c2b>

O teste DISC foi desenvolvido a partir dos estudos do psicólogo americano *William Moulton Marston*, em 1928. Não tem precisão técnico-científica, ou seja, não é seguro totalmente para definir de fato quem é você ou o seu perfil. Entretanto, quando fiz o teste, ele me fez perguntas que me fizeram pensar quem sou e o que penso a respeito de diversos assuntos.

Em suma, o teste DISC me ajudou, não a definir meu perfil dominante, mas a me autoconhecer melhor.

Os *coachs* adoram este teste. Eu nunca fiz uma sessão com um *coach*. Porém, este teste também é aplicado por psicólogos e departamentos de

recursos humanos de empresas na fase de seleção. Por isso, toda a controvérsia acerca do teste.

Então, por que é importante você saber qual o seu perfil dominante? Como já disse, para autoconhecimento. Todos temos os quatro perfis comportamentais dentro de nós, porém, um é dominante, ou seja, fala mais alto. Assim, se seu perfil dominante for segurança ou estabilidade, provavelmente, você não se dará bem sendo empreendedor. Ou, se seu perfil for dominância ou influência, não será feliz sendo empregado ou funcionário de alguma empresa tradicional.

Desta forma, o teste não é para definir quem você é ou como você faz as coisas. Mas, conhecendo seus pontos fortes e pontos fracos, poderá tomar melhores decisões e as melhores ações de acordo com a ocasião.

No livro *A arte da Guerra*, Sun Tzu diz: *“Se você conhece o inimigo e conhece a si mesmo, não precisa temer o resultado de cem batalhas. Se você se conhece mas não conhece o inimigo, para cada vitória ganha sofrerá também uma derrota. Se você não conhece nem o inimigo nem a si mesmo, perderá todas as batalhas”*.

3.2 - DEFINIR SEU OBJETIVO PROFISSIONAL

Outro ponto importante, é definir seu objetivo profissional. Se você já conhece seu perfil, pode definir qual seu objetivo profissional. Obviamente, que é ser programador, já que está lendo este livro. Mas de que tipo?

Você quer empreender? Quer implementar uma ideia? Se esse for o seu caso, a academia pode não ser o primordial. A universidade não vai te formar empreendedor.

Então, não devo cursar a faculdade? Cabe a você a decisão, mas talvez, seus esforços e recursos sejam melhor empregados na implementação da sua ideia.

Se você estudar com afinco, em questão de meses você será capaz de escrever códigos com qualidade. Mais à frente indico livros e cursos para você seguir.

Mas, caso queira empreender, jovem leitor, e ainda não saiba por onde começar, ou sequer tenha uma ideia em mente, recomendo fortemente que se matricule em um curso universitário, para adquirir experiência e, quem

sabe, arrumar parceiros, sócios ou investidor anjo para uma possível ideia. Não fique parado nunca!

Se o seu perfil for de funcionário, ou seja, estabilidade e segurança, poderá traçar o seu caminho para ter um currículo mais sólido e pomposo, com cursos universitários, pós-graduação, mestrado, e principalmente MBAs. E, não tem nada de errado nisso. Cada pessoa tem seu perfil.

Para ser sincero, penso que todos devemos ser empreendedores, desde o cara que queira montar a sua própria empresa, disposto a assumir mais riscos, ao cara que queira trabalhar em uma.

Você pode ter uma carreira corporativa maravilhosa, se for empreendedor. O livro [A nova lógica do sucesso, do médico psiquiatra Roberto Shinyashiki](#), mostra o caminho de um funcionário empreendedor.

Você pode começar por baixo, como programador júnior, ou até estagiário, e chegar ao CTO (*Chief Technology Officer*), se for um profissional acima da média.

Se esta for a sua aspiração, você poderá optar por cursos de graduação em análise de sistemas, ou gestão de tecnologia. E, após concluir o curso, faça MBAs em universidades bem conceituadas e de preferência presenciais para aumentar seu *network*.

Você poderá seguir pelo caminho acadêmico, sendo professor, pesquisador de tecnologia, ou até cientista de dados. Há muitas possibilidades. Se você desenhar bem a sua carreira e aonde quer chegar está muito mais preparado do que a maioria.

3.2 - QUAL CURSO ESCOLHER?

Particularmente, não sei a diferença entre o curso de análise de sistemas, ou de sistemas de informação, para o curso de engenharia de software e ciências da computação.

Busquei na internet, e achei o [currículo](#) do curso de sistemas de informação pela UFSC, e podemos ver diversas disciplinas em comum com ciências da computação, contudo, de acordo com a ementa, uma formação mais generalista, ou do tipo noção. Acredito que seja para uma formação mais de analista de sistemas, ou seja, o cara que desenha o sistema no papel, ou atividades como análise de requisitos, ou análise de dados.

Já o curso de ciências da computação, é uma formação mais para programador, com disciplinas como pré-cálculo, cálculo I, II e III, estatística, além de aspectos mais técnicos como compiladores, sistemas embarcados, ou outros conhecimentos mais específicos para formação mais completa de um programador.

Então, você deve definir o seu perfil, e em que ramo quer atuar.

Vamos fazer uma analogia. Para construir um edifício, são necessários vários profissionais, entre eles, gerentes de projeto, engenheiros estruturais, eletricitas, hidráulicos, de fundações, arquitetos, paisagistas, construtores, além dos trabalhadores comuns etc. Em um sistema comercial complexo, ERP, ou qualquer outro sistema, são necessários os profissionais que projetam o sistema, o banco de dados, e a relação entre os dados, a parte gráfica ou interação com o usuário, a segurança do sistema, além dos aspectos do que chamamos de *Back End* e *Front End*, além de acessos externos, APIs privadas e públicas, dentre outras características.

O programador é o cara que implementa o sistema projetado, fazendo o papel de “construtor” ou até de “trabalhador comum”, o que na minha opinião, é o trabalho mais legal.

Além de implementar o projeto, ou seja, codificar, eu também projeto, ou seja, desenho, inclusive com a elaboração de peças técnicas com base em [UML](#). Todos os programadores aprendem a desenhar projetos, contudo, há os projetistas especializados, o que é mais recomendado, para haver a separação de papéis em projetos mais complexos, por exemplo.

O Analista de sistema aprende a programar, a usar uma linguagem de programação, até porque, precisa escolher qual linguagem, *framework* ou tecnologia usar, ou qual é mais adequada para resolver um problema específico.

Há também a separação em programadores *back end*, o que é o meu caso, que são os caras que dominam as linguagens de servidor, como PHP, Python, C++, Go, Ruby, Java, Node, etc... além de padrões de desenvolvimento.

Há também o programador *front end*, o cara que lida com a interface com o usuário, deve dominar a tecnologias do tipo UIX (*User Interface Experience*), além da linguagem javascript, HTML, CSS, ou pacotes gráficos como QT para C++, Tkinter, PyQt5, Kivy para Python, ou tecnologias novas como React, Vue, bootstrap, Webpack.

Já fui programador *Fullstack*, aquele que engloba o *back end* e *front end*. Mas, não recomendo. Claro que você deve ter conhecimento da maioria das tecnologias citadas acima, mas deve se especializar em alguma delas, e não ser um generalista.

Escolhi me especializar em Python para *machine Learning* aplicado ao mercado financeiro, para robôs quantitativos, ou *quants*. Sou péssimo em *front end*, então, preferi deixar esta tarefa para programadores que tenham talento similar a um *designer*, ou que sejam mais criativos que eu.

Depois de falar um pouco sobre os papéis, vamos retornar aos cursos universitários.

Depois de tudo que já disse, espero que tenha ficado claro para o leitor que depende de qual ramo você quer atuar. Eu costumo recomendar Ciências da Computação, não porque cursei, mas porque pela análise do currículo e da grade de disciplinas de alguns cursos, entendo que é o mais completo, além de ser o mais valorizado internacionalmente, se você tiver pretensões de trabalhar fora do Brasil.

Diria para evitar os tecnólogos, pois parecem-me como cursos técnicos com anabolizantes, pois não há pesquisa, ou seja, você não é obrigado a redigir uma monografia no final para colar grau. Este tipo de curso não é reconhecido fora do Brasil, a priori, ou ao menos não é valorizado.

Então para definir o curso e a faculdade, revise o currículo do curso, a ementa. Veja se a universidade tem publicação técnica, quase todas tem

para publicação das monografias e estudos técnicos, contudo, em algumas não são fáceis de achar ou de consultar.

Compare os currículos de cursos. Leia as ementas e os livros utilizados como base e os indicados para complementos. Se for possível, assista a uma aula, visite o campus, converse com outros alunos.

Qual a qualidade das publicações? A universidade ou campus hospeda algum projeto *open-source*? Quem são os professores? Eles publicaram livros?

Pense fora da caixa. Faça diferente. Escolha os seus mestres e a sua escola com zelo e cuidado. Se for uma universidade particular, você pode pagá-la com a sua renda como programador, ou como *freela*. Já se for pública, o que é melhor na minha opinião, você se mantém com o trabalho.

Outro ponto, é se faz *online* ou presencial. Eu prefiro presencial, porque você tem a vivência e o *network*. Como citei acima, o conteúdo programático costuma ser fraco, e por muitas vezes, insuficiente.

Então, se você fizer online, terá um diploma, mas não terá o principal: amigos, colegas e a experiência de conviver e conversar com diversos profissionais, além do contato pessoal com professores e mestres.

Porém, cada um sabe de sua realidade. Se para você não for possível escolher a universidade, ou se só puder cursar online por impossibilidade de trabalho ou outra qualquer, extraia tudo que puder, e se esforce mais nos seus estudos. Há outros lugares para fazer *network* e conhecer pessoas e programadores legais.

3.3 - AINDA ESTOU EM DÚVIDA SE FAÇO UMA GRADUAÇÃO

Se você me perguntasse se eu acho que todos devem cursar uma graduação, eu diria que depende. Trata-se de uma escolha delicada, que envolve uma reflexão sobre a decisão que deveria tomar.

Tecnicamente, digo que sim, você deveria cursar a universidade, pois, além do conhecimento, trará uma boa network, podendo conhecer pessoas interessantes, parceiros de trabalho, ou até encontrar boas oportunidades, além da experiência acumulada.

Entretanto, a universidade não lhe fará um programador tecnicamente melhor. Por outro lado, dependendo dos seus professores, ou a grade curricular, poderá te agregar conhecimentos que talvez você não adquira sozinho.

Não me arrependo de ter cursado uma graduação, mas coloquei muita expectativa de que com o diploma debaixo do braço haveria mais oportunidades, ou que ele abriria portas.

Porém, a realidade foi que tive que abrir estas portas com meu talento, esforço e dedicação. Não sei dizer se seria o mesmo profissional, ou se teria a mesma cabeça se não tivesse cursado a faculdade.

Então, se tiver dúvidas, se matricule, escolha um bom curso, se dedique, vá além, e se torne um bom profissional. Não por causa da faculdade ou por um diploma, mas por seu próprio esforço, pois, a faculdade pode ser o primeiro passo para o seu sucesso, mas, jamais o caminho completo.

Quando você se formar, verá que apenas um ciclo acabou, e iniciará outro, preparado ou não para trilhar o caminho.

Durante o curso, trabalhe, estude muito, publique artigos, pesquise o ramo que escolheu atuar.

Não cometa o erro que muitos colegas meus cometeram de entrar na faculdade e continuar até o final sem saber em que área específica iriam atuar, perdidos como cegos em tiroteio. Se não souber em que área quer atuar, pense, reflita, pesquise, teste, converse com profissionais. Veja as oportunidades, antes de escolher o curso, disciplinas ou áreas de estudo.

Quanto aos tipos de cursos mencionados anteriormente, há hoje no mercado diversos tipos de cursos para escolher. Isto pode ser uma coisa boa, ou ruim, já que excesso de opções pode mais atrapalhar que ajudar, causando a paralisia decisória.

Então, meu conselho é: se não tiver certeza da área que quer atuar, escolha entre o curso de sistemas de informação ou ciências da computação, por terem um perfil mais geral.

Deixo claro, que não disse aqui para você cursar ou não cursar uma universidade, apenas dei a minha sincera opinião sobre o tema, e expus a minha crítica sobre a formação precária que a maioria dos cursos fornece.

Mas, a decisão cabe a cada um, de acordo com seu plano de vida. Se não tem um plano, então crie.

E, ao se matricular em um curso universitário, você não será automaticamente programador ao concluir e pegar o diploma. Você se tornará na medida em que estudar, se aprimorar e adquirir todas as habilidades necessárias, e como acima exposto, nem sempre virá da sala de aula.

Você terá que correr atrás. A responsabilidade é sua, e não do curso ou dos professores.

3.4 - EXISTEM BONS CURSOS LIVRES DE PROGRAMAÇÃO

Há muitos bons cursos no mercado para especialização de programadores. Plataformas como o *Udemy*, *Udacity*, tem milhares de cursos, e alguns deles são muito bons, criados e ministrados por professores ou profissionais experientes.

Não irei indicar nenhum por desconhecer os ministrantes. Mas há bastante cursos de Estruturas de Dados, Algoritmos, Lógica de Programação, e Cursos introdutórios de diversas linguagens de programação. Particularmente, não gosto destes tipos de cursos. Prefiro bons livros técnicos e o melhor de tudo: a documentação oficial.

Por exemplo, o *Python* possui uma documentação completa e muito clara, com exemplos e casos de uso. Há também uma comunidade muito grande disposta a ajudar e a compartilhar conhecimento. Não gaste seu dinheiro e tempo com cursos.

Há também bons cursos pela plataforma *Coursera*, mas é necessário ter inglês para acompanhar as aulas.

Todos necessitamos de bons mestres. Mas estes mestres não precisam estar presentes. Tenho o professor Luciano Ramalho como meu mestre, pois ele possui um livro incrível, o [Python Fluente](#), além de outros grandes mestres, como Harry Percival, com seu incrível livro [TDD com Python](#). Tenho vários mestres e ao longo do livro vou compartilhar alguns com o caro leitor.

Caso for escolher algum curso nestas plataformas, ou em outras, seja metodoso e escolha os melhores de acordo com as avaliações e as credenciais dos professores. Escolha bem seus mestres.

3.5 - ESTOU MATRICULADO EM UM CURSO UNIVERSITÁRIO, PRECISO FAZER CURSOS LIVRES?

Durante minha graduação, fiz vários cursos livres, da própria universidade, de outras na mesma região, além de cursos online (que na época eram escassos). Você precisará completar atividades extracurriculares, então nada melhor que se aperfeiçoar na área que quer atuar.

Se ainda não sabe qual área, espero que tenha ficado claro a importância de definir o quanto antes. Não precisa definir todo o caminho, mas apenas o alvo, o objetivo.

Vamos a uma analogia que ouvi em uma palestra da Professora e Filósofa, Lúcia Helena Galvão. Você é um alpinista e quer escalar uma montanha muito alta. Então você atira bem alto e finca sua corda próximo ao topo, para ser seu guia de subida. Esta corda pode estar frouxa? Você poderia se dar ao luxo de duvidar onde prender a sua corda, ou seja, até onde quer subir?

Acho que não. Se a corda de um alpinista tivesse frouxa, ele certamente cairia e morreria na queda. Mas, no percurso, pode haver dúvidas, o que é comum e normal.

No caso de programadores, você sabe que quer se tornar programador web especializado em *front end*, por adorar criar sites e interfaces de aplicativos e melhorar a experiência do usuário. Esta é a sua corda presa no topo.

Contudo, durante a subida, você poderá, e certamente terá dúvidas com qual linguagem aprender, qual *framework* utilizar, qual padrão de desenvolvimento utilizar, qual metodologia, quais pacotes gráficos, *plugins*, bibliotecas etc.

Este é o caminho, que você vai construindo ao longo da jornada, e não dá para planejá-lo com muita frequência. Afinal, as tecnologias, pacotes, bibliotecas, *frameworks* ficam ultrapassados e obsoletos com muita

frequência, e você pode, invariavelmente, mudar de ferramentas ao longo do caminho.

Assim, escolha cursos livres conceituais, ou de linguagens base. Por exemplo, você tomou conhecimento de um curso de desenvolvimento web com *React*, ou programação web com *Node* e *Express*, mas você nunca aprendeu *javascript*.

A maioria das pessoas pulam o *javascript* e vai direto para a biblioteca foda da moda. Neste caso, eu recomendaria fortemente que o aluno aprendesse *javascript* que é uma linguagem muito poderosa e riquíssima de recursos que vem sendo aprimorada e melhorada, sofrendo diversas alterações e mutações nos últimos releases.

Depois que se tornar mestre em *Javascript*, escolha uma biblioteca, como *Vue*, *React* ou *Angular*, ou até combine bibliotecas, como eu faço com frequência, com *React* + *webpack*, entre outras.

Então, escolha e curse bons cursos livres de habilidades complementares, como estatística, matemática, algoritmos, linguagens, *frameworks*, bibliotecas, padrões de desenvolvimento, técnicas de programação.

Obviamente que o programador também precisa de habilidades complementares como inteligência social e emocional, experiência do usuário, finanças e básico de economia. Dependendo da área que for atuar precisa de conhecimentos mais específicos.

Eu já criei um *software* para contadores, então tive que estudar e aprender um pouco de contabilidade para criar a solução. Adorei a experiência.

Importante também é não ter pressa. Faça os cursos sequencialmente, com calma. Após cada curso, desenvolva projetos com o que aprendeu, pratique muito, somente após, passe para o próximo nível.

3.6 - O QUE PRECISO ESTUDAR, POR ONDE COMEÇAR?

Como citei anteriormente, a universidade, ou qualquer curso que você faça, mesmo que eles digam que são completos, ou que vão te tornar um programador completo e nota A, não acredite.

O conteúdo que você terá que estudar e aprender é muito extenso e denso para ser ministrado em um único curso. Se nem a faculdade te dá tudo que precisa, não será um cursinho que te tornará foda.

Não existe fórmula mágica e nem atalhos. É estudar e praticar muito. O que todo programador precisa conhecer a fundo é algoritmo, lógica de programação e Estrutura de Dados.

No capítulo passado, falamos sobre as habilidades necessárias e as desejadas. No *Udemy* e *Udacity* há diversos cursos de matemática, estatística, matemática discreta, cálculo, ministrados por excelentes professores. Encontre alguns bons cursos, e os faça em paralelo com as disciplinas universitárias ou com seus estudos.

Não dispense bons livros. Todas as disciplinas que forem estudadas devem ser acompanhadas com bom material impresso, como livros e apostilas.

Há também, bons cursos de algoritmos e lógica de programação. Prefira os específicos e abrangentes. Vou elencar abaixo alguns livros de algoritmos recomendados:

- [Algoritmo: Teoria e prática](#) - O Foda e o mais abrangente. Utilizado por muitos cursos universitários. É muito técnico e pode ser um pouco difícil para quem decidir seguir o caminho de ser autodidata. Se for universitário pode comprar sem medo;
- [Algoritmos Para Leigos](#) - Claro, conciso, abarca os principais algoritmos existentes, além de ter uma linguagem mais direta e de fácil compreensão. Não é um livro acadêmico. Se você for autodidata, recomendo este para começar;

- [Algoritmos: Lógica Para Desenvolvimento de Programação de Computadores](#) - Excelente livro que estuda o algoritmo e a lógica de programação;
- [Aprenda Lógica de Programação e Algoritmos com Implementações em Portugol, Scratch, C, Java, C# e Python](#) - Livro Excelente e muito abrangente. Bom para autodidata e para estudos independentes;
- [Entendendo Algoritmos: Um Guia Ilustrado Para Programadores e Outros Curiosos](#) - Livro básico e para leigos. Bom para quem nunca teve contato com algoritmos antes;
- [Algoritmos E Lógica Da Programação](#) Livro didático utilizado como complemento em diversos cursos universitários;
- [Fundamentos da Programação de Computadores: Algoritmos, Pascal, C, C++ e Java](#) Bom livro, muito abrangente e completo, já apresentada ao leitor as linguagens básicas e alguns aspectos.

Há diversos outros livros, mas escolhi estes por serem mais diretos e com excelente qualidade. Estude por no máximo 2 livros no momento do aprendizado.

Isto é importante: não deixe de estudar algoritmo e lógica de programação. Não negligencie esta etapa, que em geral é pulada pela maioria das pessoas que querem aprender a programar.

Muitos “cursos completos” embrulham o algoritmo e a lógica na introdução da linguagem de programação, o que pode debilitar o programador.

Não decore, aprenda. Estude, veja, reveja e pratique. O bom do algoritmo e lógica é que estão presentes no nosso dia a dia, o que facilita o aprendizado por analogia.

Outra disciplina muito negligenciada, ou postergada para aprender com a linguagem é a Estrutura de Dados.

Mais à frente, você verá que cada linguagem de programação tem a sua própria estrutura de dados. Portanto, muitos professores ensinam a estrutura de dados de uma determinada linguagem, como se estivesse ensinando a estruturar os dados de forma correta. Não cometa este erro.

Programas mal escritos ou com mau funcionamento em geral são erros cometidos por programadores quanto à estruturação dos dados. Usam um tipo de dados *integer* 16 *bytes* para receber valores com 32 *bytes*, ou vice e versa, exigindo da máquina memória desnecessária.

A base da estrutura de dados é a escolha correta do tipo de dado para processar, armazenar e tratar um tipo específico de informação/dado, bem como o correto gerenciamento de memória.

Não vou me ater aqui em ensinar ao leitor tipo de dados, ou até estrutura de dados, lógica, técnica de programação ou algoritmo, visto que este não é o escopo deste livro.

Se eu fosse elencar um passo a passo para iniciar os estudos, seria algo como:

- 1. Algoritmo;**
- 2. Lógica de programação;**
- 3. Técnica de programação;**
- 4. Linguagens de Programação.**

Este seria o início, ou por onde começar, independente se você está cursando a faculdade, ou se é autodidata. Estes pontos seriam o seu guia de estudos e pesquisa. Claro, que está presumido que você possui bons conhecimentos de matemática e lógica.

3.7 - QUERO SER AUTODIDATA

Bom, com tudo que já falei, deve ter ficado claro que é possível um excelente programador ser autodidata. Espero que tenha pensado sobre isto, e que tenha tomado a melhor decisão para o seu caso e de acordo com o seu plano de vida.

O seu caminho não será mais fácil do que aquele que escolhe o caminho da academia, nem mais curto. Você deverá ter muita disciplina para encarar os desafios à sua frente.

Recomendo que você faça alguns cursos técnicos ou não. Se você tiver formação universitária em exatas, poderá cursar uma pós graduação, *lato* ou *stricto sensu*, ou até uma especialização na área de desenvolvimento para adquirir experiência e contatos.

Se você nunca cursou uma universidade, espero que repense, e esteja seguro de sua escolha. Deixei claro que a faculdade não é uma condição vital para a carreira do programador, mas ela ajuda abrir muitas portas. Se você não tiver, terá que fazer mais força, ou seja, ter mais talento e criatividade.

Acima citei alguns livros para começar. Escolha bons cursos introdutórios. Pratique muito. Programação se aprende fazendo e não lendo livros ou assistindo a cursos.

3.8 - QUERO EMPREENDER. EXISTEM FORMAS MAIS RÁPIDAS DE APRENDER?

O caminho da academia deve ser seguido por todos aqueles que querem seguir carreiras acadêmicas ou científicas, ou querem ser reconhecidos como tal. Se você tem uma ideia e quer empreender, certamente, não quer esperar 5 anos para pôr a sua ideia à prova.

Muitos programadores têm ideias de soluções, aplicativos, sistemas, ou seja, negócios. Então, querem mais rapidez, e com razão, neste mundo tão dinâmico, no qual ideias se tornam obsoletas tão rapidamente.

Como citei acima, se você não tem uma ideia ainda, entre na faculdade, pois ela vai te apresentar a outras pessoas, enriquecendo a sua rede de contatos. E, acredite, uma rede de contatos pode alavancar seu negócio.

Mas se tem uma ideia fixa em mente, tome os passos necessários para executá-la. Não sabe quais passos? A sua próxima tarefa então é descobrir os passos necessários para lançar seu negócio.

Tem uma ideia de aplicativo para celular? Siga os passos indicados neste livro para se tornar um programador foda, e aprenda linguagens nativas dessas plataformas que são *Java* para *Android* e *Objective-C* para *IOS*. Quer mais rápido (provavelmente), aprenda *Javascript* e em seguida *React Native* que pode abstrair as linguagens nativas.

Para o leitor empreendedor, sugiro o livro [A startup enxuta, do programador empreendedor Eric Ries](#). Nesta obra, o autor nos ensina a criar MVP (*Minimum Viable Product*), ou em tradução livre, produto mínimo viável. Ele relata a sua experiência em criar um serviço digital com seus erros e acertos. Esta técnica é hoje amplamente utilizada, principalmente o conceito ensinado no livro de teste e erre rápido e valide sua ideia antes da implementação completa.

Dependendo do seu nível de conhecimento atual, se já tiver uma noção boa de programação, não precisará estudar de forma aprofundada de lógica e técnica de programação. Então recomendo que utilize apenas o livro [Aprenda Lógica de Programação e Algoritmos com Implementações](#)

[em Portugol](#), [Scratch](#), [C](#), [Java](#), [C#](#) e [Python](#). Já te dará a base necessária para aprender as linguagens para desenvolver seu aplicativo.

Se você não tiver grana ou não encontrar um sócio especialista, terá que aprender também um pouco de infraestrutura para lançar seu projeto, como *Amazon Web Services* (infelizmente não há muita bibliografia disponível em português), *DigitalOcean* ou *Azure*.

Há bons serviços de IaaS (Infraestrutura como serviço) que facilita muito a vida do desenvolvedor e abstrai um pouco a necessidade de um administrador ou arquiteto de infraestrutura.

O leitor não precisará se preocupar tanto com a arquitetura da infraestrutura de sua aplicação no começo. Na verdade, se for MVP, poderá seguir a infraestrutura básica de cliente/servidor, do tipo *Django* para validar sua ideia e, quem sabe, você encontrará pessoas qualificadas para cuidar desses aspectos tão delicados da sua aplicação.

Gosto de uma passagem que vi no Filme *Tróia*, de 2004, na cena na qual Aquiles, interpretado por *Brad Pitt* e Pátroclo, interpretado por *Garrett Hedlund*, estão praticando a arte de esgrima, e, em uma lição, *Aquiles* troca a espada de mãos para atacar *Pátroclo*, e ele o contrapõe dizendo: “*você me ensinou a nunca trocar a espada de mãos durante um combate*”.

O guerreiro *Aquiles* responde: “*quando você souber o que fazer com a espada, não aceitará mais minhas ordens*”.

As dicas e conselhos que forneci neste capítulo e fornecerei no decorrer do livro são do tipo “*não troque a sua espada de mãos durante o combate*”, ou seja, siga-as até ter conhecimento de causa, e quando você souber o que está fazendo, não aceitará mais os conselhos de outro mestre de como manejar sua “*espada*”.

Seja cirúrgico nas suas escolhas. Não tenha dúvida de onde quer chegar. Durante o percurso, você poderá, eventualmente, decidir tomar um caminho diferente, mas se assim for, será por estratégia, ou de caso pensado. Não deixe a sua vida ao acaso. Tome as rédeas do seu destino e decida: Quero ser CTO de uma grande multinacional! Qual o próximo passo? O que devo aprender? Qual o melhor caminho? Sem este papo barato de autoajuda! É prático. Indecisos não chegam a lugar nenhum. Em qual porto chegará um barco sem rumo?

CAPÍTULO 4 - MUITO ALÉM DAS HABILIDADES TÉCNICAS

“A humildade é a base e o fundamento de todas as virtudes e sem ela não há nenhuma que o seja.”

Miguel de Cervantes

Se você continua lendo, significa que realmente quer ser um programador, ou pelo menos deseja tentar e descobrir. No capítulo passado, falamos sobre as habilidades necessárias e sugerimos um caminho para que o leitor comece sua jornada.

Agora, vamos falar um pouco sobre a base de um bom profissional, ou de uma carreira excelente.



Figura 2 - Imagem extraída do perfil @ricamorim no instagram.

Como podemos notar na imagem acima, a carreira de um profissional é formada com base em habilidades técnicas e comportamentais. A referida postagem feita pelo brilhante e renomado economista Ricardo Amorim, demonstra com clareza que as habilidades técnicas são somente a ponta do *Iceberg*.

Por baixo da água há muito além de conhecimento aprofundado em uma linguagem de programação específica.

Como tenho um perfil mais fechado, tímido, retraído, típico de quem segue a carreira em exatas, tive muita dificuldade com relacionamentos pessoais e profissionais. Eu preferia as máquinas a lidar com pessoas. Programadores costumam ser antissociais. Então, este traço de personalidade pode atrapalhar muito na sua carreira.

4.1 - TRABALHO EM EQUIPE, LIDERANÇA E COMUNICAÇÃO

Ninguém faz nada sozinho. Isto é um fato. Se você tem uma ideia de negócio ou para desenvolver um aplicativo ou site, certamente precisará de uma equipe multidisciplinar para te ajudar na batalha.

O clássico [Pense e Enriqueça, de Napoleon Hill](#), versa sobre a mente mestra, ou seja, uma equipe que desenvolve uma vontade única, sinérgica, direcionada para um fim. Neste livro, ele cita alguns grandes homens que tornaram-se imensamente ricos não porque possuíam habilidades técnicas, mas sim comportamentais.

Homens que não eram engenheiros, e projetistas, mas uniram-se a estes profissionais para criar linhas de ferro e locomotivas. Vale muito a pena a leitura.

[Jeff Sutherland, co-criador da metodologia ágil de desenvolvimento SCRUM](#), fala muito sobre a importância do trabalho em equipe, e do entrosamento dos membros da equipe.

Eu estudei e aprendi a metodologia *SCRUM*, e a aplico em meus projetos, com os famosos *SPRINTS* e entregas parciais de módulos para testes com usuários. Recomendo muito que o leitor desenvolvedor aprenda esta metodologia que é muito aplicada pelas empresas e casas de *software*.

Vou confessar uma coisa para o caro leitor: eu odiava trabalhar em grupo. isto mesmo, eu preferia trabalhar sozinho. Meu primeiro grande projeto foi praticamente sozinho.

Meu antigo chefe era o analista e gerente de projeto e eu o programador/implementador/modelador de banco de dados. Foi muito desgastante. Além disso, eu tive que aprender sobre relacionamento de banco de dados e linguagem SQL, além do *Microsoft SQL Server*.

Então, tive que criar o *front end* e o *back end*, cuidar da segurança da rede e dos dados, preocupar com a experiência do usuário (o que, para ser sincero, eu pouco me lixava na época para isto), além dos testes (para ser sincero, eu não testava quase nada, o teste era em modo produção, muito louco!).

Depois, aprendi sobre a importância do trabalho em equipe. Espero que o leitor entenda este fator como diferencial na carreira e desenvolva esta habilidade.

Na época não havia ferramentas de *pipeline* de desenvolvimento e implementação como as que existem hoje em dia. Hoje o versionamento de código e a automatização de implementação e gerenciamento de equipe tornaram muito fácil o trabalho em equipe.

O profissional que não sabe trabalhar em equipe, certamente, estará fora do mercado de trabalho. Hoje não comporta mais “lobos solitários”.

Mas, se eu for *freelancer*, mesmo assim devo saber trabalhar em equipe? Claro! Você pode ser contratado como membro externo de uma equipe, ou até ter que desenvolver um projeto ou solução para uma empresa, tendo que se relacionar com contadores, consultores, vendedores, usuários finais etc.

A comunicação é importante para qualquer profissão. Quando vamos em seminários ou eventos de programação no Brasil e no exterior, vemos que os programadores são em geral péssimos comunicadores. Há excelentes palestrantes, mas em geral não são bons.

Programador que saiba se comunicar bem, tanto com os membros da sua equipe, quanto com clientes, parceiros e usuários tem um diferencial e tanto. Entender a necessidade do cliente e dos usuários é o princípio do desenvolvimento de soluções úteis e que resolvem problemas.

4.2 - EMPATIA - EXPERIÊNCIA DOS USUÁRIOS

Programadores por vezes se apaixonam por seus códigos e acham que seus programas são muito bons e perfeitos, não merecendo retoques. Confesso também que já briguei com clientes por acharem que não estavam bons ou que não os atendiam, ou pior que achavam difícil usar.

Putz, isso me matava. Esta arrogância e prepotência da minha parte perdurou por muito tempo até entender que o mais importante é que os softwares não são criados para os programadores, mas sim para os usuários, então devem ser fáceis de usar e atender às expectativas deles.

Li no livro [SWITCH: Como Mudar as Coisas Quando a Mudança é Difícil, de Chip e Dan Heath](#), que a *Microsoft* enfrentou este problema com seus desenvolvedores. Para resolver, eles criaram laboratório de usuários, e os desenvolvedores ficavam em espelhos falsos vendo como os usuários comuns por vezes sofriam ou tinham dificuldade em usar o software.

Esta simples ação mudou a mentalidade dos desenvolvedores da *Microsoft* que passaram a se preocupar mais com a usabilidade e a praticidade de seus programas.

Eu já tive que refatorar centenas ou milhares de linhas de códigos para melhorar a usabilidade e tornar mais fácil.

Outro ponto é que quando codificamos e testamos nossos códigos em máquinas potentes, onde o programa roda redondinho, e quando vamos distribuir, na máquina simples do cliente trava, fica lento ou não roda bem. Crie e teste a sua aplicação em ambientes similares aos enfrentados pelos clientes. Pense neles primeiro.

Lembro-me de um robô que rodava em *Metatrader*, criei e testei na minha máquina com 32gb de memória, *intel i7* e placa de vídeo potente, rodando cálculos complexos de redes neurais e calculando mais de 15 indicadores.

No meu computador o tempo de resposta era de 100ms, quando vendi o primeiro robô, o cliente rodou na sua máquina da marca Positivo básica, travando o computador e quando rodava tinha tempo de resposta superior a 5s, o que no mercado financeiro é uma eternidade.

A empatia é essencial para você se colocar na pele do cliente. A solução que você está criando atende ao cliente? Supera as expectativas dele? É fácil de usar? É simples?

4.3 - CRIATIVIDADE

Outra habilidade essencial para um programador: a criatividade. Durante a execução de um projeto, por vezes, nem tudo sai como planejamos. Quantos problemas temos que resolver, e quantos abacaxis temos que descascar!

Não são poucos. A criatividade nos salva quando conseguimos ver soluções simples para problemas complexos.

Aprendi muito sobre criatividade com o livro [Roube como um artista - 10 dicas sobre criatividade - De Austin Kleon](#). Este livro abriu a minha mente e me deu outras perspectivas sobre desenvolvimento e criação. Nele o autor das seguintes dicas:

1. **Roube como um artista;**
2. **Não espere até saber quem você é para poder começar;**
3. **Escreva o livro que você quer ler;**
4. **Use as mãos;**
5. **Projetos paralelos e Hobbies são importantes;**
6. **O segredo: Faça um bom trabalho e compartilhe-o com as pessoas;**
7. **A geografia não manda mais em nós;**
8. **Seja legal. (o mundo é uma cidade pequena);**
9. **Seja chato. (É a única maneira de terminar um trabalho);**
10. **Criatividade é subtração.**

Cada uma dessas dicas e ensinamentos foram valiosíssimos para minha vida e carreira. Leia este livro e mude a forma de pensar e realizar projetos.

Outro importante livro tanto para criatividade quanto para trabalho em equipe foi o [Criatividade S.A., de Ed Catmull](#). Nele o autor mostra como a Pixar se tornou um grande e renomado estúdio, criando filmes de sucesso como *Toy Story* e *Monstros S.A.*

O que gostei muito neste livro é que ele também falou sobre as dificuldades de ser criativo em ambientes corporativos, o que é muito aplicável ao programador.

O livro Como [O Cérebro Cria: O Poder Da Criatividade Humana Para Transformar O Mundo, de David Eagleman e Anthony Brandt](#) também me ensinou muito sobre criatividade.

Eu sempre fui um inconformista nato. Nasci pobre e em uma cidadezinha pequena do interior. Muitos de meus amigos de infância ainda moram e trabalham na mesma cidade. Aquela cidade não comportava os meus sonhos. Eu não me encaixava em estruturas corporativas rígidas e conservadoras.

Depois que li o livro [Originais, do Adam Grant](#), eu compreendi a lógica de pensamento de pessoas que são inconformistas e a importância desse perfil para o mundo.

O programador brilhante é inconformista por natureza. Ele quer melhorar, não aceita o banal, e quer sempre desenvolver um produto melhor, visando aperfeiçoar a experiência com seus usuários. Eles mudam o mundo como cita o Adam Grant.

4.4 - PENSAMENTO CRÍTICO E RESILIÊNCIA

Difícil dizer quais dessas habilidades é a mais importante. Todas têm o seu grau de importância. Ter pensamento crítico é o mínimo para se considerar uma pessoa sábia e inteligente.

Muitos médicos, arquitetos, engenheiros, programadores apenas seguem a maré, ou a boiada. Não ousam pensar diferente de seus pares, ou tentar coisas novas.

Nem tudo que seu professor ou mestre te ensinar pode estar certo, ou até ter outros pontos de vistas diferentes que por vezes são deixados de lado em prol do viés da confirmação.

Eu adoro a minha autonomia de pensamento e a minha independência criativa. O livro citado acima, [Criatividade S.A](#) citou sobre a importância de um ambiente livre para que a criatividade se manifeste.

O leitor deve desenvolver pensamento crítico através de grandes leituras filosóficas. Recomendo o livro [Elástico: Como o pensamento flexível pode mudar nossas vidas, do autor Leonard Mlodinow](#), pois ele nos ensina a ter um pensamento mais flexível e crítico. Sim, um programador com pensamento matemático recomendando leituras filosóficas, ou seja, da área de humanas.

Não se tranque no seu mundo. Não leia somente sobre programação ou tecnologia, pois, você pode desenvolver um pensamento rígido e incapaz de criar algo fora da caixa. Leia tudo que te interessa. Leia sobre filosofia, sociologia, geografia e história. Eu adoro os clássicos. Experimente ler Otelo e Mercador de Veneza de *Shakespeare*, ou ler A República de Platão.

A Série Incerto de [Nassim Taleb](#), e principalmente os livros Anti-frágil e Cisne Negro, me ensinaram a pensar melhor, e analisar as entrelinhas, entender os riscos, os percalços, pormenores, e a entender um pouco as pessoas.

A resiliência é vital para que você aguentar o tranco, os sapos que terá que engolir, e os desafios que enfrentará. A autora [Angela Duckworth, no seu livro Garra. O poder da Paixão e da Perseverança](#), nos ensina o poder da resiliência e da perseverança com diversas histórias, experiências, embasadas por estudos e pesquisas psicológicas.

4.5 - FOCO NA SOLUÇÃO

Um bom programador encontra soluções inteligentes e criativas para os problemas mais cabeludos. As pessoas em geral têm foco no problema, enxergando-o de todas as perspectivas possíveis. No entanto, os resolvedores, focam na solução.

Então, como desenvolver a habilidade de resolver problemas? Como já disse antes, expandindo a sua mente e a sua forma de pensar.

Posso parecer chato em indicar livros para quem deve estar ansioso para escrever o próximo *Facebook*, mas faz parte da rotina do lenhador amolar o machado. Preparar-se para o que der e vier. Não me recordo de onde li ou vi, mas para mim esta frase faz muito sentido: Sorte é quando a preparação encontra a oportunidade.

Outro livro que me ajudou muito a expandir a minha mente, como citado anteriormente, é o [Elástico, de Leonard Mlodinow](#). Livro relativamente pequeno, de leitura fluida e rápida. Muito esclarecedor. Resumindo, a ideia central do livro é como abrir sua mente para insights resolvedores de problemas.

Já aconteceu com você de ter um problemão para resolver, se concentrar, se debruçar sobre ele e nada, mas quando estava dormindo (na cama), correndo ou tomando banho, a solução magicamente surgiu na sua mente?

Comigo já. Eu já saí pelado e molhado do banho correndo para o notebook para escrever a solução. Sempre que vou fazer atividades externas, como exercícios físicos, levo caderninho de anotações, pois, quase sempre tenho muitas ideias para projetos correntes.

Este fenômeno, se assim posso dizer, é explicado pelo professor [Daniel Goleman, em seu livro Foco: a atenção e seu papel fundamental para o sucesso](#). Nesta brilhante obra, ele nos explica como ter uma atenção focada e desfocada, ou foco disperso. Putz... Li este livro a uns 10 anos atrás, e mesmo depois de tanto tempo, ainda me lembro dos ensinamentos e de como eles me ajudaram. Vale muito apenas, para ser sincero, está entre os top 10 para desenvolver uma mente aguçada e resolvedora de problemas.

Encaramos o foco como uma lente apontada concentrando os raios solares em um único ponto (eu já fiz esta experiência), e está correto.

Contudo, Daniel Goleman nos alerta que se você só enxergar o alvo, e mais nada, não saberá lidar com adversidades que eventualmente acontecem no caminho cuja solução encontra-se “fora do manual”.

Ele cita a cegueira especialista, também citada por *Leonard Mlodinow*, que se trata de profissionais que só sabem ou leem o que diz respeito às suas tarefas ou ofícios, não lendo, se informando de nada além, ou seja, um foco extremo.

Não entenda mal, foco é essencial para o sucesso em qualquer área. Ser especialista em *Python* é muito bom, mas precisa ao menos conhecer os aspectos de algumas linguagens ou tecnologias que podem se cruzar.

Segundo esses grandes professores, as ideias se formam com cruzamento, ou choque entre os conhecimentos armazenados em sua mente, durante os períodos de relaxamento ou descanso, ou seja, quando não estamos forçando a mente a pensar.

Então, você alimenta a sua mente com informação de boa qualidade e de preferência variada, e nos momentos que você relaxar, ou estiver meditando, tomando banho, ou em tarefas automáticas que não requeira um processamento mental complexo, voilá! a ideia se forma com o choque das informações.

Recentemente, assistindo um documentário do *NatGeo*, Grandes Construções (sim, sou apaixonado por construir e saber o processo de pensamento dos grandes construtores), sobre o maior túnel do mundo.

Este túnel foi construído para ligar a Suíça à França. Com cerca de 60 km de extensão, os editores para explicar os métodos de construção, contaram a história da construção de túneis, e o primeiro túnel construído, no qual o engenheiro responsável teve a ideia observando uma lagarta que conseguia fazer um furo na madeira e se proteger de esmagamento. Como o cara teve a ideia de construir um túnel observando uma lagarta?

Nos livros citados acima há diversos exemplos em quase todas as áreas onde pessoas encontraram soluções brilhantes totalmente fora do seu ramo de atuação.

Por isso, adoro ler *Agatha Christie* (livros magníficos), *Shakespeare*, biografias (recentemente estou lendo a do *Steve Jobs*), livros do *Robert*

Bryndza (muito instigantes e criativos), assisto documentários históricos e de construções, para abrir a minha mente para soluções, *insights* e ideias.

4.6 - CONSTRUA SUA MARCA PESSOAL

Assim como a comunicação, a inteligência social é muito importante para qualquer profissional. Precisamos construir e manter uma rede de contatos sólida que vá além de “amigos de *facebook*”.

Nerds e estudiosos costumam ter um perfil mais antissocial e até um pouco introvertido. Claro que isto não é regra. Conheço alguns programadores que são extrovertidos, têm bastantes amigos, ou seja, são bastante sociáveis.

Um livro clássico e muito bom sobre como interagir com pessoas é [Como fazer amigos e influenciar pessoas, de Dale Carnegie](#). Este livro me ensinou como lidar com pessoas.

Então, você deverá construir a sua marca pessoal, firmada em valores sólidos e boas práticas. [Arthur Bender, em seu livro *Personal Branding*](#), apresenta a resposta para as seguintes perguntas:

- ***Como avaliar a imagem percebida da minha marca pessoal?***
- ***Como migrar da atual posição para uma melhor?***
- ***Com dar os primeiros passos para construir uma nova posição?***
- ***Como diferenciar minha marca pessoal e tornar-me único no segmento?***
- ***Como trabalhar a visibilidade e criar mais valor para minha marca?***
- ***Que estratégias são possíveis? Que métodos e ferramentas táticas posso empregar?***
- ***Que atitudes podem fazer toda a diferença para melhorar minha reputação?***
- ***Como posso fazer um plano estratégico para minha marca pessoal?***

Arthur Bender, de forma magistral, demonstra a importância de termos uma carreira bem governada, no rumo certo, com um alvo, ou

objetivo em mente. E, afirma, que só assim, conseguiremos sair da média e atingir uma marca pessoal de alto valor.

4.7 - APRENDA A DIZER NÃO!

Putz, que habilidade importante! Sim, é uma habilidade, infelizmente. Atualmente, estou lendo a biografia do grande Steve Jobs, e ele dizia que dizer não é mais importante do que dizer sim. Ele era famoso por cortar o supérfluo e desnecessário. Ele era minimalista.

Na vida, sempre aparece muitas oportunidades. Não podemos dizer sim para todas, senão, estaríamos dizendo não para nossos sonhos. Eu disse sim para “oportunidades” que deveria ter rejeitado. Aprendi da pior forma. Eu deveria ter dito muito mais não. Até hoje, confesso que é difícil negar algo. Como escolher qual é a melhor? Como saber?

Imagine se você tivesse aprendido a desenvolver em *Laravel*, mas surgir uma outra framework mais atual, dinâmica, prometendo ser mais rápida, segura e ágil do que ela? Como dizer não para evitar ficar pulando de galho em galho e se especializar em algo?

Como dizer não a um projeto ou emprego que é bom em curto prazo, mas a longo prazo será prejudicial, ou nos afastará do nosso objetivo?

Vamos supor que você, formando, recém saído da faculdade, receba uma oferta para trabalhar em uma indústria, fazendo coisas que odeia, mas o salário é bom, e você precisa do dinheiro, como dizer não? Como não ficar preso em um lugar que não gosta com gente de que não gosta por não saber dizer não? Por que dizer não é tão difícil?

Foque no que você quer e seja como um raio laser, diga não para tudo que não for deixar você mais perto do seu objetivo.

Espero que este capítulo tenha despertado no leitor a vontade de aperfeiçoar suas habilidades comportamentais, para se tornar um ser humano, programador e profissional melhor.

Vivemos em uma sociedade superficial e líquida, como cita o grande filósofo [Zygmunt Bauman](#). Quase todas as pessoas vivem no “modo automático”, seguindo o rebanho, e fazendo o que a sociedade espera que elas façam, como estudar, fazer faculdade, arrumar um emprego, se casar,

trabalhar 30 anos em um emprego que em geral você detesta e depois descansar com a aposentadoria estatal.

Este sistema está fadado ao fracasso diante da complexidade humana. Profissionais fora da média são cada vez mais raros, e sendo as pessoas cada vez mais voláteis, líquidas, artificiais, se você desenvolver as habilidades comportamentais acima citadas, além de outras demais, você será muito foda.

Talvez você esteja pensando: que programador é este que lê e indica livros de negócios e de filosofia? Cadê os livros de informática, desenvolvimento? Quero logo aprender a escrever programas, criar *app* para *Android* ou sites! Calma, pequeno gafanhoto (comoalaria o Gustavo Guanabara), comece pelo começo.

Você será um programador melhor quando for uma pessoa melhor. Quando for capaz de entender que soluções, aplicativos, sites não são um fim em si mesmos, mas um meio para melhorar a vida das pessoas, ou seja, o que fazemos? Tornar a vida das pessoas melhor, e não escrever códigos.

Não sei se você já leu ou assistiu ao discurso aos formandos da *Universidade de Stanford*, em 12/06/2005, realizado pelo *Steve Jobs*, que é um grande ensinamento de vida, então deixarei transcrito abaixo:

“É preciso encontrar o que você ama

Estou honrado por estar aqui com vocês em sua formatura por uma das melhores universidades do mundo. Eu mesmo não concluí a faculdade. Para ser franco, jamais havia estado tão perto de uma formatura, até hoje. Pretendo lhes contar três histórias sobre a minha vida, agora. Só isso. Nada demais. Apenas três histórias.

A primeira é sobre ligar os pontos.

Eu larguei o Reed College depois de um semestre, mas continuei assistindo a algumas aulas por mais 18 meses, antes de desistir de vez. Por que eu desisti?

Tudo começou antes de eu nascer. Minha mãe biológica era jovem e não era casada; estava fazendo o doutorado, e decidiu que me ofereceria para adoção. Ela estava determinada a encontrar pais adotivos que tivessem educação superior, e por isso, quando nasci, as coisas estavam armadas de forma a que eu fosse adotado por um advogado e sua mulher. Mas eles terminaram por decidir que preferiam uma menina. Assim, meus pais, que estavam em uma lista de espera, receberam um telefonema em plena madrugada: “temos um menino inesperado aqui; vocês o querem?” Os dois responderam “claro que sim”. Minha mãe biológica descobriu mais tarde que minha mãe adotiva não tinha diploma universitário e que meu pai nem mesmo tinha diploma de segundo grau. Por isso,

se recusou a assinar o documento final de adoção durante alguns meses, e só mudou de idéia quando eles prometeram que eu faria um curso superior.

Assim, 17 anos mais tarde, foi o que fiz. Mas ingenuamente escolhi uma faculdade quase tão cara quanto Stanford, e por isso todas as economias dos meus pais, que não eram ricos, foram gastas para pagar meus estudos. Passados seis meses, eu não via valor em nada do que aprendia. Não sabia o que queria fazer da minha vida e não entendia como uma faculdade poderia me ajudar quanto a isso. E lá estava eu, gastando as economias de uma vida inteira. Por isso decidi desistir, confiando em que as coisas se ajustariam. Admito que fiquei assustado, mas em retrospecto foi uma de minhas melhores decisões. Bastou largar o curso para que eu parasse de assistir às aulas chatas e só assistisse às que me interessavam.

Nem tudo era romântico. Eu não era aluno, e portanto não tinha quarto; dormia no chão dos quartos dos colegas; vendia garrafas vazias de refrigerante para conseguir dinheiro; e caminhava 11 quilômetros a cada noite de domingo porque um templo Hare Krishna oferecia uma refeição gratuita. Eu adorava minha vida, então. E boa parte daquilo em que tropecei seguindo minha curiosidade e intuição se provou valioso mais tarde. Vou oferecer um exemplo.

Na época, o Reed College talvez tivesse o melhor curso de caligrafia do país. Todos os cartazes e etiquetas do campus eram escritos em letra belíssima. Porque eu não tinha de assistir às aulas normais, decidi aprender caligrafia. Aprendi sobre tipos com e sem serifa, sobre as variações no espaço entre diferentes combinação de letras, sobre as características que definem a qualidade de uma tipografia. Era belo, histórico e sutilmente artístico de uma maneira inacessível à ciência. Fiquei fascinado.

Mas não havia nem esperança de aplicar aquilo em minha vida. No entanto, dez anos mais tarde, quando estávamos projetando o primeiro Macintosh, me lembrei de tudo aquilo. E o projeto do Mac incluía esse aprendizado. Foi o primeiro computador com uma bela tipografia. Sem aquele curso, o Mac não teria múltiplas fontes. E, porque o Windows era só uma cópia do Mac, talvez nenhum computador viesse a oferecê-las, sem aquele curso. É claro que conectar os pontos era impossível, na minha era de faculdade. Mas em retrospecto, dez anos mais tarde, tudo ficava bem claro.

Repito: os pontos só se conectam em retrospecto. Por isso, é preciso confiar em que estarão conectados, no futuro. É preciso confiar em algo - seu instinto, o destino, o karma. Não importa. Essa abordagem jamais me decepcionou, e mudou minha vida.

A segunda história é sobre amor e perda.

Tive sorte. Descobri o que amava bem cedo na vida. Woz e eu criamos a Apple na garagem dos meus pais quando eu tinha 20 anos. Trabalhávamos muito, e em dez anos a empresa tinha crescido de duas pessoas e uma garagem a quatro mil pessoas e US\$ 2 bilhões. Havíamos lançado nossa melhor criação - o Macintosh - um ano antes, e eu mal completara 30 anos.

Foi então que terminei despedido. Como alguém pode ser despedido da empresa que criou? Bem, à medida que a empresa crescia contratamos alguém supostamente muito talentoso para dirigir a Apple comigo, e por um ano as coisas foram bem. Mas nossas visões sobre o futuro começaram a divergir, e terminamos rompendo - mas o conselho ficou com ele. Por isso, aos 30 anos, eu estava desempregado. E de modo muito público. O foco de minha vida adulta havia desaparecido, e a dor foi devastadora.

Por alguns meses, eu não sabia o que fazer. Sentia que havia desapontado a geração anterior de empresários, derrubado o bastão que havia recebido. Desculpei-me diante de pessoas como David Packard e Rob Noyce. Meu fracasso foi muito divulgado, e pensei em sair do Vale do Silício. Mas logo percebi que eu amava o que fazia. O que acontecera na Apple não mudou esse amor. Apesar da rejeição, o amor permanecia, e por isso decidi recomeçar.

Não percebi, na época, mas ser demitido da Apple foi a melhor coisa que poderia ter acontecido. O peso do sucesso foi substituído pela leveza do recomeço. Isso me libertou para um dos mais criativos períodos de minha vida.

Nos cinco anos seguintes, criei duas empresas, a NeXT e a Pixar, e me apaixonei por uma pessoa maravilhosa, que veio a ser minha mulher. A Pixar criou o primeiro filme animado por computador, Toy Story, e é hoje o estúdio de animação mais bem sucedido do mundo. E, estranhamente, a Apple comprou a NeXT, eu voltei à empresa e a tecnologia desenvolvida na NeXT é o cerne do atual renascimento da Apple. E eu e Laurene temos uma família maravilhosa.

Estou certo de que nada disso teria acontecido sem a demissão. O sabor do remédio era amargo, mas creio que o paciente precisava dele. Quando a vida jogar pedras, não se deixem abalar. Estou certo de que meu amor pelo que fazia é que me manteve ativo. É preciso encontrar aquilo que vocês amam - e isso se aplica ao trabalho tanto quanto à vida afetiva. Seu trabalho terá parte importante em sua vida, e a única maneira de sentir satisfação completa é amar o que vocês fazem. Caso ainda não tenham encontrado, continuem procurando. Não se acomodem. Como é comum nos assuntos do coração, quando encontrarem, vocês saberão. Tudo vai melhorar, com o tempo. Continuem procurando. Não se acomodem.

Minha terceira história é sobre morte.

Quando eu tinha 17 anos, li uma citação que dizia algo como "se você viver cada dia como se fosse o último, um dia terá razão". Isso me impressionou, e nos 33 anos transcorridos sempre me olho no espelho pela manhã e pergunto, se hoje fosse o último dia de minha vida, eu desejaria mesmo estar fazendo o que faço? E se a resposta for "não" por muitos dias consecutivos, é preciso mudar alguma coisa.

Lembrar de que em breve estarei morto é a melhor ferramenta que encontrei para me ajudar a fazer as grandes escolhas da vida. Porque quase tudo - expectativas externas, orgulho, medo do fracasso - desaparece diante da morte, que só deixa aquilo que é importante. Lembrar de que você vai morrer é a melhor maneira que conheço de evitar

armadilha de temer por aquilo que temos a perder. Não há motivo para não fazer o que dita o coração.

Cerca de um ano atrás, um exame revelou que eu tinha câncer. Uma ressonância às 7h30min mostrou claramente um tumor no meu pâncreas - e eu nem sabia o que era um pâncreas. Os médicos me disseram que era uma forma de câncer quase certamente incurável, e que minha expectativa de vida era de três a seis meses. O médico me aconselhou a ir para casa e organizar meus negócios, o que é jargão médico para "prepare-se, você vai morrer".

Significa tentar dizer aos seus filhos em alguns meses tudo que você imaginava que teria anos para lhes ensinar. Significa garantir que tudo esteja organizado para que sua família sofra o mínimo possível. Significa se despedir.

Eu passei o dia todo vivendo com aquele diagnóstico. Na mesma noite, uma biópsia permitiu a retirada de algumas células do tumor. Eu estava anestesiado, mas minha mulher, que estava lá, contou que quando os médicos viram as células ao microscópio começaram a chorar, porque se tratava de uma forma muito rara de câncer pancreático, tratável por cirurgia. Fiz a cirurgia, e agora estou bem.

Nunca havia chegado tão perto da morte, e espero que mais algumas décadas passem sem que a situação se repita. Tendo vivido a situação, posso lhes dizer o que direi com um pouco mais de certeza do que quando a morte era um conceito útil mas puramente intelectual.

Ninguém quer morrer. Mesmo as pessoas que desejam ir para o céu prefeririam não morrer para fazê-lo. Mas a morte é o destino comum a todos. Ninguém conseguiu escapar a ela. E é certo que seja assim, porque a morte talvez seja a maior invenção da vida. É o agente de mudanças da vida. Remove o velho e abre caminho para o novo. Hoje, vocês são o novo, mas com o tempo envelhecem e serão removidos. Não quero ser dramático, mas é uma verdade.

O tempo que vocês dispõem é limitado, e por isso não deveriam desperdiçá-lo vivendo a vida de outra pessoa. Não se deixem aprisionar por dogmas - isso significa viver sob os ditames do pensamento alheio. Não permitam que o ruído das outras vozes supere o sussurro de sua voz interior. E, acima de tudo, tenham a coragem de seguir seu coração e suas intuições, porque eles de alguma maneira já sabem o que vocês realmente desejam se tornar. Tudo mais é secundário.

Quando eu era jovem, havia uma publicação maravilhosa chamada The Whole Earth Catalog, uma das bíblias de minha geração. Foi criada por um sujeito chamado Stewart Brand, não longe daqui, em Menlo Park, e ele deu vida ao livro com um toque de poesia. Era o final dos anos 60, antes dos computadores pessoais e da editoração eletrônica, e por isso a produção era toda feita com máquinas de escrever, Polaroids e tesouras. Era como um Google em papel, 35 anos antes do Google - um projeto idealista e repleto de ferramentas e ideias magníficas.

Stewart e sua equipe publicaram diversas edições do The Whole Earth Catalog, e quando a ideia havia esgotado suas possibilidades, lançaram uma edição final. Estávamos na metade dos anos 70, e eu tinha a idade de vocês. Na quarta capa da edição final, havia uma foto de uma estrada rural em uma manhã, o tipo de estrada em que alguém gostaria de pegar carona. Abaixo da foto, estava escrito "Permaneçam famintos. Permaneçam tolos". Era a mensagem de despedida deles. Permaneçam famintos. Permaneçam tolos. Foi o que eu sempre desejei para mim mesmo. E é o que desejo a vocês em sua formatura e em seu novo começo.

*Mantenham-se famintos. Mantenham-se tolos.
Muito obrigado a todos."*

CAPÍTULO 5 - JÁ COMECEI, E AGORA?

“Um bom começo é a metade.”

Aristóteles

Já está estudando. Cursou ou está cursando as disciplinas básicas. Já está lendo a biografia, e agora? Esta fase nunca se acabará. Você sempre estudará. Se acostume com esta rotina. Eu quase todos os dias desde quando decidi ser um bom profissional.

5.1 - QUAL RAMO ESCOLHER, DEVO ME PREOCUPAR COM ISTO?

O Advogado precisa saber em qual ramo do direito precisa atuar. Quer ser criminalista, civilista, administrativista? Se ele definir a especialização, poderá se dedicar a aprimorar as suas habilidades de forma mais focada e certa.

De igual forma, o médico precisa escolher em qual área vai atuar, vai ser pediatra? Ginecologista?

Putz, tenho que escolher isto logo? Bom, o melhor cenário é o do que o profissional já sabe de antemão qual ramo quer atuar, para se especializar.

Pode ser que você seja totalmente iniciante e não tenha ideia, neste caso, não perca seu tempo tentando escolher. Teste, faça, tenha contato com o ramo, e você saberá aquele que mais te excita, que faz o seu coração bater (Tá bom, estou sendo muito romantizado), ou que você mais se identifique.

Como já citei antes existe o programador *Full stack*, ou *Full Stack Developer*, fazendo uma analogia, é o “clínico geral” do desenvolvimento. Ele precisa conhecer um pouco de tudo. Geralmente, *freelancers* são *full stack* por necessidade.

Há os desenvolvedores *front end*, que precisam conhecer o dominar HTML, CSS, *javascript*, bibliotecas de *scripts bootstrap* e/ou *material UI*, além de *React*, *Vue*, *Angular*, *Webpack*, entre outras. Em geral estes profissionais são bons designs, criativos, lidam bem com arrumação, e são perfeccionistas.

Os desenvolvedores *Back End*, como eu, precisam dominar linguagens como PHP, *Node*, *Ruby on Rails*, *Java* (Springs) ASP.NET, *redis*, além dos *frameworks Laravel*, *Django*, *Flask*, *FastAPI*. Claro que você não precisa nem deve ser especialista em *Django* e *Flask* ao mesmo tempo. São *frameworks* para *Python* com aplicações recomendadas diferentes, mas que podem ser utilizados para o mesmo fim.

O *Database developer*, precisa conhecer os RDBMS do tipo MySQL, MSSQL, Postgres, MariaDB, bem como NoSQL, como MongoDB, *Cassandra*, *Redis*. Há também os *Graphs*, que eu particularmente odeio, que são *ArangoDB* (não sei muito, ou quase nada sobre eles).

Os desenvolvedores *DevOps*, que é um movimento, além das linguagens de programação, devem dominar tecnologias de infraestrutura como *Nginx*, *AWS*, *Azure*, *ELK*, bem como tecnologia de automação CD/CI como *Ansible* e *Jenkins*. Devem também ser mestres em virtualização como *Docker* (descubra o poder do *container* e seja um programador diferenciado), *Kubernetes*, *Vagrant* e *VMWare*.

Eu estou estudando para dominar o *DevOps*. Uso servidores *AWS*. Adoro usar o *docker*. Recomendo muito que o leitor aprenda e use. Excelente para simular ambiente de produção e testes em seu computador. Estou estudando *Kubernetes* que é orquestração de *containers*. Ainda não concluí nenhum projeto com esta tecnologia. Estou em fase de aprendizagem.

5.2 - QUAIS OS OPORTUNIDADES DE TRABALHO PARA UM PROGRAMADOR ATUALMENTE

Já falei sobre isto, mas cabe um adendo. Os ramos de especialização são em geral a forma como você vai trabalhar. Com o desenvolvimento da inteligência artificial e automatização da indústria, bem como a informatização de quase todos os serviços, há diversos ramos de atuação para um programador.

Você pode participar do projeto que está construindo máquinas para curar o câncer. Projetos de internet das coisas. Hoje até as geladeiras e fogões estão inteligentes. É trabalho de programador escrever estes códigos.

Recentemente, comprei uma TV inteligente para minha mãe. A TV recebe e executa comandos de voz. O controlador foi desenvolvido e escrito por um programador.

Estamos na era dos carros autônomos. Daqui a alguns anos, teremos carros dirigindo sozinhos na rua. Há um projeto legal na empresa brasileira Vale onde alguns caminhões e máquinas são operados por inteligência artificial.

Praticamente em todas as indústrias e ramos da economia a programação está presente. As possibilidades são infinitas. Que maravilha!

Os negócios estão demandando cada vez mais processamentos inteligentes. O livro [Máquinas Preditivas, dos economistas Agrawal, Gans e Goldfarb](#), demonstra que cada vez mais a economia demanda processamento inteligente e o uso de análise preditiva para auxiliar a tomada de decisão, ou até automatizar processos mecânicos e repetitivos.

Se quiser conhecer um pouco mais de inteligência artificial, sugiro a leitura do livro [O Algoritmo Mestre, do Pedro Domingues](#). Não é um livro técnico. é um livro introdutório onde o autor cita todas as correntes atuais com seus respectivos algoritmos mestres. Neste livro o autor nos chama para ajudar a criar o algoritmo mestre, que seria como a mente humana, capaz de aprender tudo a um custo energético razoável e com

processamento rápido. Nele, o leitor poderá ter uma visão geral das áreas de atuação em AI e ver qual se adapta melhor ao seu perfil.

5.3 - QUANDO TEREI QUE ESCOLHER A LINGUAGEM DE PROGRAMAÇÃO?

Esta pergunta não tem uma resposta clara e objetiva. Quando você começar a estudar lógica e estrutura de dados, certamente, já lhe serão apresentadas algumas linguagens de programação clássicas. Como já citei neste livro, aprendi a programar com *Cobol* e *Pascal*, porém, nunca escrevi um programa nestas linguagens.

Eu experimentei diversas linguagens e já escrevi programas inteiros em pelo menos 10 linguagens. Quando você aprende a lógica, algoritmo e a estrutura básica de dados de uma linguagem como C, que inspirou a maioria das linguagens atualmente em produção, você adquire facilidade para aprender rapidamente qualquer outra linguagem do mercado.

Obviamente que a experiência em uma determinada linguagem é necessária para entender toda a API, funções avançadas e manipulação de memória, arquivos e sistema operacional.

Há dezenas de linguagens de programação no mercado. Cada uma tem a sua aplicação e vantagem. Difícil dizer ao autor qual é a melhor, até porque, entendo que não existe a melhor para todos os casos. Cada uma é mais apropriada a determinadas aplicações.

Por exemplo, se quer escrever aplicativo para Android de forma nativa deve-se adotar o *JAVA*. Quer escrever um *app* para o IOS (*Iphone*) de forma nativa? é o *OBJECTIVE-C*.

Contudo, há linguagens multiplataformas como *Python*. Você pode desenvolver um aplicativo em *Python + Kivy* que roda tanto em *Android* ou IOS. Ou até aprender *React Native* para esta tarefa.

Quer dizer que terei que aprender *Java* para escrever *app* para *Android*? Seria bom, mas pode ser *Kotlin*, uma linguagem novinha, moderna e um pouco mais simples que *Java*. Eu já fiz programas em *Java*, e para ser sincero não gostei. *Kotlin* se assemelha à *Java*, embora seja mais prática.

Vamos a um exemplo porque prefiro *Python* a *Java*. Um comando simples de imprimir um texto na tela escrito em *Java*:

```
public class ImprimeTextoSimples {  
    public static void main(String[] args) {  
        System.out.println("Hello World");  
    }  
}
```

O mesmo comando escrito em Python:

```
class ImprimeTextoSimples:  
    def __init__:  
        print("Hello World")
```

Não quero assustar o leitor, ou enchê-lo de escolhas. Tenha paciência. A escolha é pessoal. Leia sobre as linguagens e suas respectivas aplicações. Escreva um programa inteiro e meça quanto tempo levou?

Gostou da experiência? Foi mais fácil? Mais produtiva? A linguagem tem uma biblioteca rica e prática? Tem uma boa documentação? A comunidade é grande e prestativa? Conhece outro programador que usa a mesma linguagem? Há bons livros sobre a linguagem?

Então, não tenha pressa. Estude o básico e experimente. Você sabendo a lógica por trás das linguagens e algoritmos, bem como a estrutura de dados, você aprende uma linguagem nova em poucos dias (não se tornará mestre, mas será capaz de utilizá-la).

Não sei tudo sobre *Python*. Mesmo depois de anos, vira e mexe tenho que consultar a documentação para lembrar os parâmetros de um comando ou outro. E há diversos comandos que não uso muito. Estes esqueço mais rápido. Para isto serve a documentação!

Aprender a lógica é como aprender andar de bicicleta. Você jamais esquece, e pode andar em qualquer bicicleta.

5.4 - ESCOLHA LINGUAGENS E FRAMEWORKS ATIVOS

Quando for escolher uma linguagem, verifique se ela está sendo atualizada. Quando foi a última atualização. Quais programadores ou organização estão por trás da atualização? Há muitos desenvolvedores no core? Verifique o log de modificações.

Há versão nova em desenvolvimento? Há versão LTS (*Long-term Support*) até quando? Eu sempre utilizo a versão LTS em meus códigos. Por exemplo, não utilizava o Python 2.7 nos meus códigos faz tempo, porque o LST dela foi até 2020.

A partir daí, a equipe não vai manter o core atualizado, ou seja, pode haver vulnerabilidades. Imagine você escrever seu complexo sistema em uma linguagem que não terá mais suporte ou atualização de segurança? Terá que refatorar o que é um inferno. Já peguei projeto para reescrever um sistema inteiro de *Python 2.x* para *Python 3.x*.

Na verdade, foi para adaptar, mas havia tantos erros, bugs, incompatibilidades e partes de códigos complexas demais, que tive que reescrever quase que por completo.

Vamos a um exemplo prático. Eu utilizo o pacote [*FastAPI*](#) para escrever algumas de minhas APIs. Este pacote foi desenvolvido por um único programador. Entretanto, olhando na página do repositório do *GitHub*, há 208 programadores que fizeram contribuição para o código. O projeto possui grandes empresas patrocinando, o que é um bom sinal. Além disso, grandes empresas já o utilizam em seu sistema, como o *Instagram* e a *Microsoft*.

Verificando a página de [*Release*](#), constata-se que as atualizações são constantes e significativas. Outro ponto que verifico é a documentação. Neste caso é satisfatória. Como estou usando bastante o pacote, vou fazer uma contribuição com algumas funções e características que senti falta.

Faça testes. Na dúvida, utilize linguagens, pacotes e *frameworks* consolidados, como Python e *Django*.

Nunca utilize pacotes sem atualização ou defasados. Não comprometa a segurança e estabilidade de seus programas. No último caso, dê o *Fork* no

projeto que foi abandonado, verifique porque foi abandonado, se vale a pena fazer as atualizações e correções e continue você mesmo a manter o pacote ou biblioteca atualizado (isto requer experiência, paciência e prática). No *JazzBand*, fazemos isto.

Pegamos bons pacotes abandonados por seus criadores e damos continuidade com a manutenção. Sim, até grandes pacotes podem ser descontinuados por diversos motivos.

5.5 - A LINGUAGEM DE PROGRAMAÇÃO IMPORTA PARA O SUCESSO DO PROGRAMADOR?

Já deve ter ficado claro para o caro leitor que pouco importa a linguagem que você usa, mas sim a sua habilidade como programador. Conhecer a linguagem ou o *framework* da moda, como por exemplo o *Laravel*, para *back end developers*, ou *React*, *Vue* ou *Angular*, para *front end developers*, não te garantirá sucesso.

Essas tecnologias mudam com muita rapidez. Até há pouco tempo, não se falava em *Laravel*. Todos queriam *Ruby on Rails*, ou até *cakePHP*, ou *Symfony*. Essas tecnologias ainda existem, mas perderam o “protagonismo” para o poderoso *Laravel*, feito para “artesãos”.

Preocupe-se em dominar a lógica ou os padrões por trás destas tecnologias. As bibliotecas, pacotes, funções que fazem parte delas são mais facilmente aprendidas.

Fazendo uma analogia, a lógica, técnica e padrão é como a habilidade de mecânica de automóveis, e os frameworks, tecnologias e linguagens, são como as caixas de ferramentas.

As ferramentas podem se modernizar, ficar diferentes, mais simples, dinâmicas, mais precisas, seguras, mas o princípio por trás delas não muda.

5.6 - APRENDENDO A USAR AS FERRAMENTAS

Não diferente de qualquer outra profissão, os desenvolvedores possuem diversas ferramentas para desempenhar seu trabalho. Destaco que uma das principais e que todo programador deveria aprender desde o início é o sistema de versionamento de código, ou [VCS \(Version Control System\)](#) ou em bom português [SCV \(Sistema de Controle de Versão\)](#).

O mais utilizado e famoso de todos é o [GIT](#), desenvolvido por *Linus Torvalds*, o criador do *Kernel* do *Linux*. Eu confesso que nunca utilizei outro. Todos os projetos que já desenvolvi utilizaram o GIT, bem como as empresas com quem já trabalhei.

Antes do GIT, era muito difícil versionar código, ou pior, quando dois programadores modificaram o mesmo arquivo, como saber qual era a versão final?

No meu primeiro projeto não utilizei nenhum controle de versionamento, eu não tinha este conhecimento ainda, então, passei pela penúria de versionar código na mão, manipulando os arquivos manualmente, salvando-os em pastas separadas.

Ninguém entendia aquele código! Eu era o único que conseguia saber qual era o atual, e por onde começar. Graças a Deus eu aprendi a trabalhar de forma padronizada e ordeira. Hoje, mesmo se eu não puder terminar um código, qualquer programador consegue continuar com um pouco de leitura da documentação.

5.7 - O QUE É GIT

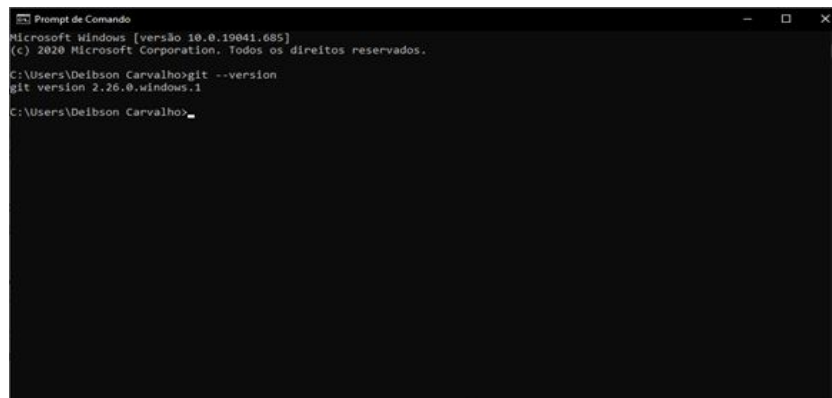
Lembra que no início deste livro falei que você iria precisar se familiarizar com linha de comando? Chegou a primeira prova: o *git*. Originalmente, ele é um software de linha de comando, ou seja, não possui interface gráfica para manipulação.

Recomendo desde já que você baixe e instale o *git* diretamente do site [_git-scm.com](https://git-scm.com). A documentação é muito boa. E eles disponibilizam o livro *Pro Git* de graça no site para leitura online. Eu tenho a edição impressa para consulta. É um livro antigo, mas não se preocupe, pouco mudou desde sua última edição.

Como o próprio site do *GIT* cita, é fácil de aprender através da [documentação oficial](#). Aprenda, pratique. Você vai usar muito na sua vida profissional e nunca sai da moda.

O intuito deste livro não é ensinar a usar o *GIT*, o que daria um livro inteiro, mas vou demonstrar como iniciar.

Após devidamente instalado, seguindo a própria orientação do assistente de instalação, abra a linha de comando, e digite “*git --version*”.



```
Prompt de Comando
Microsoft Windows [versão 10.0.19041.685]
(c) 2020 Microsoft Corporation. Todos os direitos reservados.

C:\Users\Deibson Carvalho>git --version
git version 2.26.0.windows.1

C:\Users\Deibson Carvalho>
```

Figura 3 - Tela de linha de comando do windows com o comando “*git --version*”

Para iniciar um novo repositório, crie uma pasta no seu diretório de trabalho e digite “*git init*”.



Figura 4 - Tela de comando do Windows com o comando "git init"

Com este comando, o *GIT* cria um arquivo oculto “.git” no seu diretório com as configurações do projeto. Pronto, o *GIT* já está versionando todas as pastas e arquivos que forem inseridos na pasta do projeto.

O *GIT* possui muitas funções e características que você deverá dominar para ser um bom programador e saber resolver um problema. Quando você modificar um projeto existente e o seu release der um bug, você irá agradecer muito por ter usado o *GIT* e reverter o bug até a versão estável anterior.

5.8 - O GITHUB

Apresentamos a você o *GIT*, porém, o *git* é um gerenciador de repositório local, na sua máquina, ou em uma máquina central em uma rede. Agora imagine se a máquina der pau!

Comigo já aconteceu de o HD queimar com todos os arquivos dentro. A minha sorte na época era que eu guardava backups diários em um *pendrive*. Método arcaico.

Hoje em dia existem os repositórios em nuvem. A beleza e praticidade da nuvem. Um dos mais famosos é o *GitHub*. Eu o utilizo desde 2010. Antes era pago para usar como repositório, hoje em dia é grátis, possuindo versão premium para grandes equipes.

Além de ser um repositório, o *GitHub* também é uma espécie de rede social para programadores. Já fiz muitos parceiros e negócios ali. Se você ainda não tem um perfil, crie um o quanto antes e comece a guardar e organizar os seus códigos com ele. É muito seguro.

É importante citar que há outros, mas eu só uso o *GitHub* que me atende muito bem.

Bom que o *GitHub* possui um aplicativo para desktop que dá uma interface gráfica para o *GIT*, porém eu não utilizo. Ainda prefiro a linha de comando, porque me dá mais liberdade e praticidade.

Te vejo no *GitHub*, me segue lá [@deibsoncarvalho](#), vamos trocar uma ideia ou fazer um projeto juntos, adoro dividir projetos com outros programadores.

5.9 - ENTRE EM COMUNIDADES DE DESENVOLVEDORES

Além do *GitHub* citado acima, há diversas outras comunidades boas para conseguir orientação, conhecer outros desenvolvedores e fazer parcerias. Uma que utilizo muito e já salvou a minha pele várias vezes é a famosa [*stackoverflow*](#).

Crie seu perfil profissional nela e comece a ajudar outros desenvolvedores, assim como obter ajuda e orientação. Tem de tudo, desde aspectos básicos de linguagens de programação a tecnologias complexas. Os programadores são muito solidários.

Há outras redes e sites onde programadores se encontram. Pesquise e encontre outras para se cadastrar e interagir com outros profissionais ou aprendizes.

5.10 - PRATIQUE SUA HABILIDADE DE RESOLVER PROBLEMAS

Programador é um resolvidor de problemas nato. Pegamos situações complexas ou tarefas mecânicas e automatizamos. Um site que descobri recentemente é o [Codewars](https://www.codewars.com/). Nele o programador treina suas habilidades em resolver problemas, desde situações bobas e banais as mais cabeludas.

Não aceito projetos fáceis ou banais. Gosto de desafios. Projetos que vão me fazer pensar e criar algo. Lembro-me de um projeto que executei para um serviço de venda de ingressos para *shows* no exterior, em 2008.

Este cliente queria uma interface gráfica complexa, como os cinemas fazem hoje para alocar lugares, sendo que à época, tal feito era complicadíssimo de fazer com *javascript*, HTML e CSS.

Nesta época, o *AJAX* estava na moda, então tive que improvisar com jogo de imagens e processamento assíncrono e truques de css. Hoje qualquer programador iniciante faz isto com os pés nas costas com *REACT* ou *VUE* em 3 ou 4 linhas de código.

Há outras plataformas e serviços bacanas para praticar suas habilidades online, além de interagir com outros programadores.

5.11 - GERENCIAMENTO DE PROJETOS

Gerente de projetos é uma função específica, com formação e certificação em diversas metodologias e padrões de projetos, como por exemplo o *SCRUM*. É um cargo que não é necessariamente exercido por programadores.

Contudo, se você for trabalhar como freelancer, terá que gerenciar seu próprio projeto. E se você contratar outro programador para te ajudar em um projeto grande, como gerenciar os trabalhos? Como dar transparência para seu cliente do curso do projeto?

Então você deve ter uma boa noção de algumas metodologias de gerenciamento de projetos. Eu recomendo a *SCRUM*, contudo, há outras, cabe você analisar qual se adapta melhor ao seu perfil.

Recomendo os livros [Gerenciamento de Projetos Para Leigos](#), simples mas dá uma boa noção geral de gestão de projetos sem aprofundar, e o livro [Gerenciamento de Projetos Para Não Gestores](#), também simples, mas com um aprofundamento um pouco maior que o anterior.

Quanto às ferramentas, eu utilizo o [Trello](#), simples, direto, funcional, permite integrações com outras ferramentas como o *Slack* e *Google Drive*. Você pode incluir outras pessoas nos projetos.

Eu testei e gostei do *JetBrains* [YouTrack](#), especializado em gerenciamento de projeto de desenvolvimento para equipes ágeis, com a metodologia *KanBan* compatível com a metodologia *SCRUM*. Eu estou pensando em utilizar ele para projetos de clientes.

Você pode criar uma página para cada projeto, e cada página tem a sua configuração e equipe. Ele também tem integração profunda com as IDEs da *JetBrains*. Eu utilizo o *PyCharm* para desenvolver em *Python*, *Django* e *R*.

Posso criar uma *pipeline* de implementação facilitada e totalmente integrada com o versionamento de código que pode ser do *GitHub* ou da própria *JetBrains*

Eu utilizo a metodologia *DevOps*, então estas plataformas com integração, permitem uma entrega contínua do código.

Há outras ferramentas e detalhes de gerenciamento de projetos. Sugiro que o leitor estude o assunto e tenha uma noção para que não fique vendido em uma reunião com um possível cliente ou parceiro. Conheça e teste as ferramentas e veja qual te atende melhor e se adequa ao seu propósito.

CAPÍTULO 6 - ESCOLHENDO A SUA LINGUAGEM DE PROGRAMAÇÃO

“Quando minha escolha é consciente, nenhuma repercussão me assusta. Quando não é, qualquer comentário me balança.”

José Eustáquio

Após um bom início, aprendendo lógica de programação, algoritmo, estrutura de dados, você vai sentir vontade de começar a escrever códigos, então precisará de um compilador, ou um interpretador para executar seu código.

Certamente você deve ter tido contato com linguagens como C, *JAVA*, C++, *PASCAL*, ou *FORTRAN*, em alguns livros de algoritmos e lógica de programação. Muitos autores utilizam estas linguagens clássicas para exemplificar seus algoritmos.

Eu aprendi C, e recomendo que você também aprenda C. Você não vai utilizar C para escrever seus programas, provavelmente, mas quase todas as linguagens atualmente tem forte influência em C.

Python, por exemplo, tem parte do seu código fonte em C, e tem até o *CPython*, que aceita códigos de C. Os tipos de dados de *Python* são baseados em C. O código compilado em C é muito rápido, então quando se precisar de rapidez na resposta e no processamento se utiliza C ou C++.

Eu utilizo C++ para escrever HFTs ou algoritmos de alta frequência que têm precisam devolver uma resposta em milissegundos.

É um leque tão grande de linguagens para escolher, que por vezes, pode bater a dúvida de qual utilizar. A tendência atual é escrever menos código e fazer mais, então, utilizam-se *frameworks* para facilitar o meio de campo, colocando a bola no seu pé, na frente do gol para chutar, sem precisar percorrer o campo inteiro.

6.1 - PARADIGMA DE PROGRAMAÇÃO

Paradigma de programação é um meio de se classificar as linguagens de programação baseado em suas funcionalidades. As linguagens podem ser classificadas em vários paradigmas. Um paradigma de programação fornece e determina a visão que o programador possui sobre a estruturação e execução do programa.

Há diversos paradigmas existentes, porém muitos não estão mais ativos. Podemos dizer que, de forma clara e objetiva, possuímos programação estruturada, procedural e orientada a objetos.

Por ora, você não precisa se preocupar em escolher o paradigma no qual vai escrever seus programas.

Eu utilizo a programação orientada à objetos por me permitir escrever códigos mais limpos, melhor gerenciamento de memória, mais leves e seguros.

Contudo, já escrevi programas de forma estruturada. Mas eles eram pesados, pois, não me permitia reutilizar códigos, ou gerenciar o acesso às variáveis de forma mais eficiente.

Programação orientada à objeto tem recursos riquíssimos como polimorfismo, interfaces, classes, herança, o que possibilita escrever um código limpo, claro, leve e simples.

A maioria das linguagens atuais são multiparadigmas. Você pode escrever um programa de forma estruturada ou orientada a objetos com *Python* ou *Go (Golang)*, por exemplo. A linguagem C é estruturada. C++ foi, trocando em miúdos, a implementação da orientação à objetos de C. *Java* é outra linguagem poderosa multiparadigma.

Então teste, aprenda. Decida depois que testar e escrever programas. Mas aposto que você vai escrever seus programas com orientação a objetos. Só escrevo código estruturado quando é para scripts simples escritos em *Python*, *PHP* ou outra linguagem.

6.2 - PARA CADA FUNÇÃO TEM UMA LINGUAGEM ESPECIALIZADA

Para escrever programas para *Android* de forma nativa é o *JAVA*. Temos o *Kotlin*, que é uma linguagem simples e interessante que permite escrever programas de forma nativa, porque ele compila para linguagem *JAVA*.

Se for escrever *scripts* para executar em um *microservice*, recomendo linguagens mais rápidas e com multiprocessamento nativo, como *GO* e *JAVA*. No *Python*, temos o *GIL*, que é uma máquina que somente executa um código por vez, mesmo que o computador tenha vários núcleos.

Você tem que implementar a funcionalidade de *multi threaded* para diversos processos paralelos no mesmo núcleo de processador ou multiprocessamento para vários núcleos, o que torna a execução de código mais lenta. Como eu não domino o *GO (Golang)*, utilizo *JAVA* com *SPRINGS*, ou *Node* para serviços mais simples.

Então certamente, você lidará com mais do que uma linguagem, pois para cada desafio você vai utilizar a linguagem mais recomendada. Deste modo, você deve estudar e entender cada linguagem e as suas aplicações. Faça testes. Estude a diferença entre as linguagens e como você pode utilizar cada uma da melhor forma.

6.3 - PORQUE O PYTHON É UMA ESCOLHA CERTA PARA PROGRAMADORES INICIANTE

Eu amo *Python*. Sou *Pythonic*. É uma linguagem riquíssima e multiplataforma. Posso escrever códigos para praticamente qualquer finalidade. Utiliza-se *Python* para sistemas embarcados ou IOT. Para *machine e Deep Learning*. Para *web*. Você pode escrever códigos para serem executados no servidor, ou se preferir, utilizar *frameworks* como *Flask* ou *Django* para escrever sites completos e funcionais de forma fácil e rápida. Pode escrever uma API, com diversos pacotes como *FastAPI*.

Abra o console *Python* e digite “import this”:

The Zen of Python, by Tim Peters

Beautiful is better than ugly.

Explicit is better than implicit.

Simple is better than complex.

Complex is better than complicated.

Flat is better than nested.

Sparse is better than dense.

Readability counts.

Special cases aren't special enough to break the rules.

Although practicality beats purity.

Errors should never pass silently.

Unless explicitly silenced.

In the face of ambiguity, refuse the temptation to guess.

There should be one-- and preferably only one -- obvious way to do it.

Although that way may not be obvious at first unless you're Dutch.

Now is better than never.

*Although never is often better than *right* now.*

If the implementation is hard to explain, it's a bad idea.

If the implementation is easy to explain, it may be a good idea.

Namespaces are one honking great idea -- let's do more of those!

Não vou traduzir, porque a esta altura do campeonato, o leitor já sabe da importância de ser fluente em inglês. Mas é o lema do *pythonic*, e o que levo para qualquer outra linguagem.

Na verdade, considero para qualquer projeto que faça. Conselhos como erros não deveria passar em silêncio já me salvou várias vezes, ou se a implementação é difícil de explicar é uma má ideia, também fazem muito sentido.

Tá bom! Sou suspeito para falar de *Python*, por gostar tanto desta linguagem. Mas comprove por si mesmo. Leia o livro *Python Fluent*, do Luciano Ramalho, e se tornem excelentes programadores.

Além disso, é uma linguagem fácil de aprender. A curva de aprendizado é muito curta. Em dias você já é capaz de escrever programas simples.

6.4 - PRECISO ESCOLHER UM FRAMEWORK?

Como já citei antes, você deve dominar a linguagem antes de utilizar um *framework*. Afinal o que é um *framework*? Todos os projetos têm partes comuns, e por vezes, as mesmas necessidades, como segurança, controle de acesso, permissão, persistência de dados, interface gráfica, etc.

Imagine como seria se você tivesse que escrever tudo do zero todas as vezes que iniciasse um novo projeto? Ou, você tivesse que reescrever os códigos que já sabe que vai utilizar? seria muito trabalho, não.

Então, deste problema, surgiu o *framework*, que é como uma caixa de ferramentas, ou um monte de códigos, procedimentos, interfaces, funções que são frequentemente utilizadas para mesma finalidade, não precisando serem novamente reescritos, ou que o desenvolvedor não precise se preocupar novamente com aquilo.

Eu já escrevi sites em PHP pelado, sem *framework*. É muito chato e trabalhoso. Em geral é um *control+C*, *control+V* de arquivos e partes de códigos. A cada arquivo .php que escrevia para uma determinada função, por exemplo processar o recebimento de dados de formulários postados, a parte do código que sempre se repetia, eu mantinha arquivos básicos para copiar e colar.

Esta metodologia é danosa, pois, se eu tivesse que alterar uma parte no procedimento de tratamento dos dados de um POST, ou implementar um filtro de segurança novo, teria que reescrever em todos os arquivos, o que seria muito trabalhoso.

As coisas melhoraram quando comecei a utilizar o *CakePHP*, pois na época não existia o *Laravel*, que é relativamente recente. Ele facilitou todas essas funções banais e básicas, me deixando somente com as regras de negócio. Eu não utilizo mais PHP em meus projetos. Cheguei a estudar *Laravel*, mas não concluí nenhum projeto grande com ele.

Eu uso muito o *Django*, que é um *framework* para escrever aplicações web síncronas e assíncronas (atualmente eles implementaram esta função, até que enfim) na linguagem *Python*. Escrevo sites complexos desenhados em horas.

Há *frameworks* de *frameworks*, como o *restframework* para *Django*, para implementação de APIs do tipo REST.

Então, preciso escolher o *framework*? Depende da área que vai atuar, e do nível de conhecimento na linguagem base do *framework*. Quer desenvolver aplicações de uma página com *React*? Aprenda *javascript*.

Mas se você já escolher com qual *framework* quer trabalhar, pode te auxiliar no estudo da linguagem de programação.

6.5 - PRECISO ESTUDAR BANCO DE DADOS OU SQL?

Você certamente terá que lidar com banco de dados. Diversos aplicativos têm alguma persistência de dados. Administrador de banco de dados é uma profissão respeitadíssima e importantíssima, pois é o profissional que estrutura, configura e gerencia o servidor de banco de dados. Eu já tentei fazer isso e afirmo, não é nada fácil. Passei a respeitar muito mais este profissional.

Mas, como desenvolvedor, deve conhecer os tipos de dados do motor de banco de dados a ser utilizado, como por exemplo o *MySQL/MariaDB*, ou *PostgreSQL*. Se você não souber, pode fazer cagadas no tratamento dos dados recebidos e enviados.

A maioria dos *frameworks* possuem ORMs, mas não todos. ORMs é uma forma de abstrair a estrutura do banco de dados e suas tabelas, permitindo que o desenvolvedor acesse e manipule os dados com códigos, sem precisar conhecer a linguagem SQL.

Porém, eu recomendo fortemente que o leitor estude e aprenda SQL, para ser um desenvolvedor completo. os ORM não fornecem suporte para consultas muito complexas, o que só é possível escrevendo a consulta em SQL. Então, vá se inteirando do SQL e dos bancos de dados. Uma noção é o suficiente, mesmo se for para desenvolver seus próprios projetos com banco de dados.

6.6 - PRECISO UTILIZAR UM EDITOR DE TEXTO OU CÓDIGO, OU ATÉ UMA IDE?

Se você está aprendendo uma linguagem, siga o que o mestre que está te ensinando o orientar. Se o autor ou professor não falar nada, pergunte a ele, e mesmo se ele for escorregadio ou não recomendar nada, utilize uma IDE simples e gratuita como o *VS Code* ou *Atom*.

Há IDEs profissionais muito boas para cada linguagem. Eu utilizo o *PyCharm Pro*, que custa US\$ 89 por ano, mas vale cada centavo. É uma IDE complexa, com possibilidade de instalação de *plugins* para estender as funcionalidades, e com integração à serviços como AWS (*Amazon Web Services*), Git, e banco de dados. Você ainda não precisa pagar.

O *VS Code* é muito bom. Eu o utilizo para escrever códigos *Javascript*, *Node*, *Express* e *React*.

Se você quer se tornar um bom programador terá que dominar as suas ferramentas para explorar ao máximo o seu potencial. Estude e leia a documentação oficial da ferramenta que utilizar.

CAPÍTULO 7 - JÁ ESCOLHI MINHAS FERRAMENTAS, AGORA COMO FAÇO PARA COMEÇAR A TRABALHAR

“Para o trabalho que gostamos, levantamo-nos cedo e fazêmo-lo com alegria.”

William Shakespeare

Uma das dificuldades mais comuns entre os profissionais iniciantes: onde conseguir trabalho. Se você chegou até aqui, já deve ter tudo em mente, já iniciou seus estudos, já escolheu a linguagem que irá se especializar, o ramo, se vai utilizar algum *framework*, e deve estar louco para meter a mão na massa.

Vamos começar a testar as suas habilidades. Mas, onde conseguir um projeto legal?

Quando comecei, a internet ainda estava engatinhando. Eu utilizava conexão discada a uma velocidade de 56k. Putz, você jovem não sabe o que é isto! Hoje você reclama porque um vídeo do *YouTube* demorou uns segundos a mais para carregar no seu celular.

Imagine você entrar em um site com fotos pesadas, e o site ir carregando aos poucos, cara fração da foto por vez, e levar por vezes minutos para carregar por completo? Estes jovens não sabem o que perderam.

Neste período era difícil achar parcerias ou projetos para participar. Fiz meus próprios projetos, escrevi meus códigos. Em 2006, conheci um rapaz que queria criar um site da cidade ou da região (na época isto estava na moda), então aceitei a sociedade (furada) para criar o site, unicamente pela experiência, não estava em busca do dinheiro.

Criei o site em *PHP* e *MySQL*, rústico, feio, com *HTML* e *CSS* de péssima qualidade, bem o site não vingou, como suspeitava, mas escrevi e testei meu código.

Acredite, só se aprende programar programando. A teoria que você estuda em um livro será posta à prova quando você escrever seus códigos e tiver que implementá-los. Tudo que pode dar errado dá.

Então como consegui meu primeiro trabalho remunerado *freelancer*? Na época conheci a plataforma *freelancer.com*, me inscrevi e fiz propostas para trabalhos simples dando o menor lance, apenas para pegar projetos e escrever códigos. Tive alguns problemas. tiveram códigos que não deram certo, porque não havia entendido o que o cliente queria ou por estar totalmente bugado, com erro em tudo. Mas tudo é experiência. Não cometo os mesmos erros. Até hoje eu erro e as vezes faço cagadas monumentais, por isso tenho backup, versionamento de código e documentação de tudo. Cada cagada me ensinou muito. Por isto acredito que a derrota ensina mais do que a vitória.

7.1 - COMPUTADOR PARA PROGRAMADOR

A ferramenta mais importante que você precisa é o seu cérebro. Contudo, precisará de um computador para materializar as suas ideias.

Se você já tem um computador em casa, ou até um notebook, já comece. Não espere o ideal.

Como eu tenho um bom conhecimento de computadores, gosto de montar minhas máquinas de desktop. Atualmente, uso um *intel i7*, com 32gb DDR4 2666mhz de memória, 1 terabyte de SSD, e placa de vídeo *Geforce GTX 1050 4G DDR5*, rodando *Windows 10 PRO*, com 2 telas 24" e 1 tela 25" wide. Para melhorar a produtividade, utilizo [suporte de monitores de mesa ELG](#).

Tenho uma mesa de escritório ampla com boa iluminação e uma [Cadeira do tipo Gamer](#). Como passo muitas horas sentado, deve ter o máximo de conforto para evitar problemas lombares.

Claro que esta configuração custou muito caro, por volta de R\$ 14 mil reais. Um programador iniciante precisa disso tudo? Claro que não.

Minha namorada tem um *Notebook* com *intel i3* com 4gb de memória e HDD 1 *Terabyte* 5400rpm. É muito lento para um programador, reduzindo a produtividade. Se for rodar IDEs pesadas, ou se for *game developer* ou cientista de dados, não dá nem para o começo. Mas tem um lado positivo em desenvolver em computadores lentos: seus programas rodam bem em qualquer computador.

Recomendo no mínimo 8GB de memória e um *Intel i5 8ª* geração. Se for possível, um HD SSD, para acelerar as coisas. Não precisa mais ser *Windows PRO*. Eu utilizei a versão *pro* porque antes, a versão *home* não suportava virtualização.

Se você puder, prefira 16GB de DDR4 com placa mãe que suporte 2 pentes de 8 GB com *dual channel*.

Se for cientista de dados, for trabalhar com *machine learning*, *deep learning*, ou *game developer*, será preciso uma boa placa de vídeo com ênfase em processamento, para você treinar as suas habilidades com CUDA.

Mas, não faça dívidas. Trabalhe com o que tem e, à medida em que ganhar dinheiro, vá fazendo *upgrades* da máquina.

E *notebook*? Eu utilizo *notebook*, mas como precisamos de mais poder de processamento, eles costumam ser caros, ainda mais com o dólar na casa dos R\$ 5.

Os programadores gringos amam *Apple iMac Pro*, pois, o poder de processamento é excelente. Eu não o utilizo porque no Brasil ele custa mais de R\$ 20 mil. Ele é mais recomendado para *front end developers*. Invista a sua grana em conhecimento primeiro. Quando você tiver grana, viaje para o USA e compre o seu *iMac* muito mais barato.

7.2 - -CRIE SEU PORTFÓLIO

Sim, mesmo que você seja aprendiz, esteja começando ou nunca tenha feito um projeto antes. Crie os projetos exemplos dos livros, ou arrume projetos pequenos para criar que demonstrem as suas habilidades. Você estará praticando o que está aprendendo e criando uma demonstração da sua capacidade e do seu trabalho para seus futuros clientes e parceiros.

Crie e lance aplicativos simples do tipo lançamento de despesas para web, desktop ou celular. Você pode fazer isso de graça! Isso mesmo, sem gastar nada! Serviços de Cloud como *AWS* e *Azure*, oferecem 1 ano de alguns de seus serviços de graça para o pequeno desenvolvedor testar sua ideia e implementar seu aplicativo.

A *Google Cloud Platform* também tem um projeto similar, além do *Firebase*, que te auxilia no lançamento de seu App. Ou seja, você não tem desculpas!

Registre um domínio por R\$ 20 (mais barato que um *BigMac*) ou ingresso de cinema. E, crie seu portfólio na web. É mais profissional.

7.3 - PLATAFORMAS DE FREELANCERS

Se você tiver inglês fluente, existem atualmente diversos sites de freelancers, como *fiverr*, *freelancer.com*, entre outros.

Mesmo sendo pago em dólar, a concorrência com os palestinos, indianos e russos é cruel, eles cobram US\$ 4 a hora, o que no câmbio atual é pouco mais que R\$ 20. Mas, vale muito pela experiência e contato com outro idioma e com os clientes.

No começo, se o dinheiro não for vital para você, ou seja, se você não precisar se manter, priorize a experiência. Caso você precise da grana, aí não tem jeito, mas busque também a experiência.

Aqui no Brasil, há o Workana e o 99Freela. Já usei os dois. Eles têm perfil de iniciantes. Os trabalhos são muito baratos e os contratantes são inexperientes, e muitas vezes são usuários leigos que querem saber quanto custará para implementar uma ideia.

Lembro de uma proposta que fiz para uma empresa que queria implementar um serviço de gerenciamento de *lead*, similar ao *ActiveCampaign*, e quando especifiquei o valor com mais de 5 dígitos, quase com 6 dígitos, com uma equipe de 6 desenvolvedores, os caras me chamaram de louco.

Mantive contato com o representante da empresa, e eles fecharam contrato com um freelancer que cobrou R\$ 3 mil para 30 dias de trabalho. Após 2 meses, este mesmo representante me enviou mensagem no *whatsapp* dizendo que o cara não conseguiu entregar o projeto e me pedindo para abaixar o preço ou reduzir o trabalho para um MVP.

Não aceitei. Aprendi nestes anos de trabalho que não deve-se aceitar trabalho de chorões, pois são os piores. Eu não uso mais estas plataformas.

Sempre tive problemas com clientes chorões ou chatos. Os que pagam mais barato são os piores, pois em geral, por serem miseráveis, exigem tudo que podem e não podem. Escolha bem seus clientes.

O próprio *GitHub* e *StackOverFlow* são plataformas de *freelancers* para encontrar trabalho.

Então, cadastre-se nessas plataformas, crie perfis atrativos e verdadeiros. O melhor é a prova social e o seu portfólio de trabalhos. Se

não tiver um, se vire para criar o quanto antes, mesmo que sejam projetos de estudo ou gratuitos.

7.4 - REDES SOCIAIS

Preencha seu perfil do *Linkedin*, mas é importante que seja autêntico e verdadeiro. Não invente história ou conte balela. Fale a verdade. Se está estudando, informe isto. Publique artigos de estudos ou de aprendizado em alguma tecnologia. Uma das melhores formas de aprender é ensinando.

Participe de grupos de discussão e eventos. Faça contatos e se conecte com outros profissionais. Se tiver procurando emprego formal, conecte-se à recrutadores, mas não encha o saco deles. Mostre seu trabalho de forma sutil e bacana, sem ser chato.

Não despreze também o poder de redes sociais como o *instagram* ou *facebook*. Eu não as uso muito, tendo em vista que há muito ruído e bobagens, mas não menosprezo seu poder.

Há muitos grupos sérios de desenvolvedores no *facebook*, e nestes grupos pode-se aprender muito e estreitar laços, além de estabelecer contato com outros profissionais.

7.5 - CRIE SEU BLOG E PUBLIQUE ARTIGOS

É muito importante compartilhar conhecimento. Quem tem compartilha. E, certamente, se você conseguir emplacar algum artigo na primeira página do Google, você será mais facilmente encontrado.

Aproveitando a oportunidade, conhecimentos de SEO (*Search Engine Optimization*) é de grande valia. Se você for *Front End developer* será indispensável. Estas habilidades podem te ajudar muito nos seus perfis sociais e nos seus artigos e *blog*.

Claro que esta estratégia é de médio e longo prazo. Mas deve ser plantada o quanto antes.

A experiência de escrever e a começar compartilhar conhecimento é gratificante. Invariavelmente, você poderá ser abordado por pessoas interessadas no seu talento. Eu já arrumei muito trabalho no *LinkedIn*, e olha que meu perfil é básico e eu não o alimento muito.

7.6 - PARTICIPE DE DEBATES, DISCUSSÕES E RESPONDA PERGUNTAS NO GITHUB E STACKOVERFLOW

Responda dúvidas nas plataformas, participe de grupos de discussão ou de desenvolvimento. É importante que participe da comunidade de programadores que é muito solidária. Estamos sempre dispostos a ajudar outro programador.

Seja ativo nessas comunidades. Compartilhe seu código. Ajude a outro programador a melhorar o código dele. Acredito muito na reciprocidade.

Você será visto se participar, e ao longo do tempo, poderá desenvolver relacionamentos de amizade, ou até parcerias.

7.7 - ENCONTRE ESTÁGIO

Se você for universitário, procure estágios de preferência em casas de software para ter experiência com desenvolvimento em equipe.

Prefira projetos a empregos ou trabalhos fixos. Mas cabe a você decidir de acordo com as suas possibilidades e necessidades. Se a coisa estiver braba, tem que pegar o que vier (Sei muito bem como é isso).

Bom, se você se empenhar, não será difícil encontrar projetos ou trabalho. Na sua própria faculdade deve ter algum, ou um projeto de um colega. Participe de mentorias, ajude outros iniciantes, compartilhe seu conhecimento.

Se você tiver dificuldade de encontrar trabalho remunerado. Não pare, nem desanime. Faça seus próprios projetos. Ajude empreendedores a implementar ideias em troca de sociedade, isto pode trazer boas experiências.

Cada um sabe de sua situação e do que precisa. Alguns colegas de faculdade não tinham grana para se manter, e como não achavam oportunidades na área de programação, tiveram que aceitar qualquer emprego para a sua subsistência. É normal e compreensível. Não se sinta diminuído por estar nessa situação. Talvez você só tenha que se esforçar mais.

E, mesmo que você trabalhe fora do ramo, pode arrumar tempo para se dedicar a projetos e ao aprendizado. Desculpa como “não tenho tempo” quer dizer que isto na verdade não é prioridade para você. Quando você quer algo, se vira e arruma tempo!

Sei como é difícil equilibrar a vida pessoal com a profissional, ainda mais na fase de estudos. Para ser sincero, eu não consegui manter nenhum relacionamento amoroso ou amigos durante o meu processo. Lamentavelmente, me afastei de alguns amigos queridos que eram de outras áreas, ou pessoas simples, com empregos simples e que não entendiam que precisa se dedicar a algo para conquistar coisas maiores.

Bom, acredito ser possível. Se eu tivesse a cabeça que tenho hoje, quando era mais jovem e estava começando “a subir a montanha”, teria considerado um balanceamento entre carreira e relacionamentos pessoais.

Mesmo que você só tenha uma hora por dia para se dedicar a projetos e estudos, se concentre e aproveite esta hora ao máximo. Não é a quantidade, mas sim a qualidade que define o progresso.

Eu tinha uma vida insana. Começava a estudar e programar 6h da manhã e parava só no dia seguinte quando estava exausto. Isto não é sustentável. Dedique-se ao aprimoramento pessoal, à família e aos amigos também, para ter uma vida equilibrada e saudável.

Não faltam oportunidades para um bom profissional, mesmo que na fase de aprendizagem. Esteja disposto, preparado, faça sua parte, e, provavelmente, a oportunidade certa aparecerá, uma hora ou outra.

CAPÍTULO 8 - PRIMEIRO PROJETO. POR ONDE COMEÇAR?

“Você não tem que ser grande para começar, mas você tem que começar, para ser grande.”

Zig Ziglar

Você aprendeu o básico, ou está aprendendo. Já escolheu as suas ferramentas. Está praticando e desenvolvendo as suas habilidades técnicas e comportamentais. Agora, conseguiu seu primeiro projeto. E agora?

Se você está em um projeto em andamento, ou com outros desenvolvedores experientes, gerente ou líder de projeto, tudo bem. Seja humilde, e aprenda com eles, tanto com os erros quanto com os acertos. Replique sucesso e aprenda com os fracassos.

Sei como é querer um papel importante, mais responsabilidade, ou escolher o que quer fazer, mas, repito: seja humilde! Aceite seu papel de bom agrado. Faça-o da melhor forma possível. Lembre-se do que falamos anteriormente da importância do relacionamento interpessoal e de saber lidar com as pessoas.

8.1 - SEJA PROATIVO

Especialmente o caro leitor que está em um projeto com outras pessoas. Independente se o projeto é remunerado, ou se você tem ou não algum benefício direto, seja proativo.

Antecipe-se aos problemas. Seja solidário com seus colegas de projeto. Ajude-os sem pedir nada em troca. Faça sua parte de forma zelosa e responsável.

Falamos anteriormente sobre a importância de saber trabalhar em equipe. Agora é a hora de provar que você aprendeu. Qual o padrão de gestão de projeto adotado pela equipe? Quem é o líder ou *Product Owner*? O que você tem a acrescentar ou a otimizar? Você está entregando o que é esperado de você? Está sendo sincero e íntegro? Está ajudando a solucionar os problemas do projeto?

Não tenha medo de ser imperfeito. Abrace a vulnerabilidade, como nos ensina [Brené Brown, em A coragem de ser imperfeito](#). Mas, como? Se lhe for atribuída uma tarefa da qual você não dá conta, não conhece a tecnologia ou não tenha experiência, peça ajuda aos colegas.

Eles vão confiar mais em você se não tiver medo de dizer “*não sei como fazer isto*”. Lembre-se, vocês são uma equipe e devem trabalhar juntos! Se souber de algo que seus colegas não sabem, ou se vir algum colega em dificuldades, ofereça a sua ajuda de forma amigável.

8.2 - MEU PRIMEIRO PROJETO SOLO

Conseguiu um *freela*? Que massa! É um projeto pessoal? Isso ae! Na verdade, não importa, o zelo e dedicação devem ser os mesmos.

Eu já me deparei com esta situação e não sabia por onde começar ou o que fazer primeiro.

Recomendei que você estudasse um pouco de gerenciamento de projeto, te indicando a metodologia *SCRUM*, por ser a metodologia ágil mais utilizada atualmente. Existem outras que já estão em desuso.

Comece pelo começo! Simples. Putz... tá de sacanagem com a minha cara!? Qual é o começo? haha. Se você fosse construir uma casa, qual seria o começo? Provavelmente, você definiria onde a casa vai ficar, em qual terreno, compraria o terreno, faria terraplanagem.

Depois, você decide qual o *layout* ou modelo da casa você quer e contrataria um arquiteto para desenhar a planta baixa. Você escolheria e contrataria o construtor, compraria os materiais de construção e daria início à construção.

8.3 - PROJETANDO O SISTEMA

Fazendo uma analogia, vamos supor que é um projeto de terceiro e o seu cliente quer um sistema de gestão de imobiliária online. Então, o cliente já definiu que vai ficar na internet (localização do terreno) e o que (como quer o modelo da casa). Vamos supor, ainda, que o sistema dele não está projetado (desenhado), ou seja, não tem a “planta baixa”? Qual o próximo passo? Desenhar o sistema.

Eu já recebi projetos totalmente desenhados, ou seja, eles haviam contratado um analista de sistemas que documentou os requisitos, o banco de dados e criou toda a documentação UML, o que facilitou meu trabalho.

Então eu preciso desenhar o sistema? Criar documentação UML? Dependendo do caso, não. Vou ser sincero com o leitor, na maioria dos casos, eu não crio todos os documentos. Depende do acordo com o cliente, e se ele vai querer a documentação burocrática. Mas, então você faz o código sem qualquer documentação? Claro que não, senão seria uma bagunça!

Os requisitos negociais, formais e funcionais devem ser levantados. Traduzindo: o que o cliente quer e precisa. O que ele espera que a tela de cadastro de imóveis faça e como se comporte? Quais cores, filtros, características, tamanho de fotos e quantidade de anúncios. Ele quer que seja rápido, não pode carregar a página? tem que ter algum tipo de aviso, quem pode fazer o quê? Qual o usuário alvo? Tudo isto deve ser levantado e documentado. [Há um bom artigo no Medium que explica como fazer isto.](#)

É importante a participação dos interessados ou *stakeholders*. Se possível, converse com usuários, e, após valide tudo com seu cliente. Talvez seja necessário você agendar reuniões online ou presenciais para interagir com os interessados. Se você não interagir com eles, o projeto estará fadado ao fracasso.

Após, você fará um documento simples com a declaração do problema. Neste documento será como o escopo do projeto. Defina-o bem, dê ciência ao cliente. De preferência, toda a comunicação com o cliente

deve ser registrada para uma auditoria, se for o caso de conflito, mas evite isso ao máximo.

Um escopo de projeto mal definido poderá te trazer dor de cabeça além de grandes problemas, inclusive para sua reputação como desenvolvedor. Preze por sua marca pessoal. Seja sempre transparente com o cliente. Lembre-se do lema: “*Cliente tem sempre razão*”.

8.4 - STORYBOARD

Há diversas metodologias de desenvolvimento. Eu utilizo muito a XP (*Extreme Programming*) juntamente com o *KanBan*. Na fase de exploração, unida com a engenharia de requisitos, podemos criar o *storyboard* com base na história do usuário. No meu escritório/estúdio tenho um quadro branco onde desenho o que vou fazer.

Como eu faço: Primeiro, peço ao cliente para descrever passo a passo o que um usuário faz. Por exemplo:

“um corretor quer cadastrar um imóvel para venda. Ele faz o login com seu usuário e senha. Através do seu dashboard, ele clica em imóveis e cadastrar um novo imóvel. Abre uma nova tela, o corretor passa informações sobre os donos do imóvel. Tamanho, dimensão, localização e valor. Seleciona as características do imóvel. Adiciona fotos e salva.”

A partir da história de usuário acima, pesquisamos os requisitos, como, quais os dados devem conter? Quais são obrigatórios? O corretor pode alterar os dados? Quem define o preço? O sistema calcula as comissões ou é o corretor que informa? Quais dados estarão visíveis e para quem?

Após você se certificar de que levantou os requisitos de negócios e funcionais necessários, você cria o *storyboard*.

8.5 - MODELO DE DADOS

Com o *storyboard* pronto, é hora do modelo de domínio, ou modelo de dados. Você define as entidades, por exemplo, Imóveis e Usuários, e o relacionamento entre eles, bem como os dados que serão necessários armazenar no banco de dados.

Esta parte envolve conhecimento de estrutura de dados bem como noções básicas de banco de dados, conforme mencionado anteriormente.

Se você não se sentir seguro para modelar a estrutura do banco de dados, subcontrate este serviço. Procure um profissional para modelar os dados para você, então ele te entregará todo o *schema* do banco de dados prontos, incluindo consultas básicas, relacionamentos, *views*, *triggers* etc.

Não é difícil de fazer, mas requer cuidado com os relacionamentos e com o tipo de dado armazenado. Observe como ele se relaciona com a estrutura de dados da linguagem de programação a ser utilizada. Muita atenção com dados sensíveis, como valores financeiros, números decimais e datas. Saiba o que você está fazendo.

Atente-se à codificação do banco de dados e para a linguagem de programação. Não se esqueça do *timezone* que deve estar alinhado ao servidor, banco de dados e interpretador/compilador da linguagem.

Tome cuidado com a integridade dos dados. Fala o devido relacionamento entre as entidades e o tratamento dos dados.

8.6 - CRIE PROTÓTIPOS

Há serviços na internet que disponibilizam a função de criar protótipo para o seu cliente ver como vai ficar a interface. Eu utilizo o [Invision](#). Ele tem um plano gratuito que só permite um projeto por vez. Com ele, eu crio a interface e submeto à aprovação do cliente antes de iniciar a implementação.

A prototipação também pode ser com a própria linguagem de programação que você utilizará. *Django* é um *framework* muito bom para estes casos. Ele tem sistemas de autorização e permissão de usuários pronto, com telas de cadastro, edição, login, tudo pronto e de fácil customização. Com o *Django*, você consegue criar telas básicas muito rapidamente.

Só finalize as telas ou interfaces com os usuários após a provação do *layout* pelo cliente. Evite retrabalho ou insatisfação com o cliente.

8.7 - INICIANDO E DOCUMENTANDO O PROJETO

Finalizada a fase de aprovação do layout por parte do cliente ou validação do protótipo, passamos à implementação do código.

Isto mesmo. Eu não faço o desenho do projeto ou documentos de UML, ou coisa do tipo. A única documentação que tenho são as histórias do cliente e o storyboard. Não sou analista ou projetista. Claro, que em projetos grandes ou muito complexos, é altamente recomendado que você os modelos de arquitetura mínima. O caro leitor pode se perguntar: Mas, como ele organiza o sistema? Como outros profissionais podem trabalhar nele? Bom, já adianto que não é a melhor e mais recomendada forma de se trabalhar a arquitetura de um sistema. Para falar a verdade, arquitetar um sistema é um dos assuntos mais controversos do desenvolvimento de software. Existem diversos padrões e não há consenso entre os estudiosos do tema.

Confesso que li poucos livros de arquitetura de software, pois acho eles muito chatos, extensos e burocráticos. Se pegar um antigo que segue a cartilha tradicional, é um sonífero e leva uma eternidade para se criar um sistema. Eu aprendi a criar sistemas antes de aprender na faculdade as noções de arquitetura de software. Como já falei antes, eu me tornei programador prático e profissional muito antes de me formar na universidade.

Eu aprendi a XP (*Extreme Programming*) que é uma metodologia de *agile development*. Nela, a documentação é mínima e não burocrática. Então, nos meus sistemas produzo o mínimo possível. Na verdade, só o extremamente essencial ou indispensável para a compreensão de como o sistema irá funcionar. Eu uso diagramas UML de casos de uso, componentes e de implantação, por exemplo.

Para quem se interessou em arquitetura de sistemas ou engenharia de software, [Ian Sommerville](#) é o cara. Os livros dele são adotados por quase todos os cursos de engenharia de software e ciências da computação. Já adianto que são livros técnicos, extensos e pouco didáticos. Só recomendo se você for se aprofundar no assunto ou quiser parecer foda, dizendo que

leu o cara. Se você só quer ter uma noção mínima para escrever seus próprios documentos, recomendo [Engenharia de Software na Prática, de Hélio Engholm Jr](#), por ser mais prático, didático e conter mais exemplos. Se quiser estudar UML, recomendo o livro [UML 2 - Uma Abordagem Prática: uma Abordagem Prática](#) é muito bom.

8.8 - POR ONDE COMEÇO MEU PROJETO?

Esta foi uma dúvida minha quando estava aprendendo: por onde começar? Não sabia o que fazia primeiro, qual a estrutura de pastas ou organização de um projeto.

Eu gosto de aprender com exemplos. Então busquei na internet por projetos prontos feitos por profissionais. Eu gosto de fazer engenharia reversa e ver como funciona. Os livros que lia eram muitos abstratos e davam poucos exemplos completos. Eram sempre fragmentados e não explicavam o básico, ou seja, por onde começar.

Abaixo falarei de *templates* de projeto. Eu recomendo você iniciar o seu por um *template* feito por profissional experiente. Caso queira começar do zero, arquivo por arquivo, pasta por pasta, a escrever os arquivos de configuração de gerenciadores de pacotes, compiladores etc., ou seja, *hardcore*, recomendo que estude os projetos similares no *GitHub* e como eles fizeram, para que motivo etc. Pode te ensinar mais que um livro abstrato. Bom, recomendo que se você queira realmente aprender, faça pelo menos um do total zero. Passe o perrengue de configurar as integrações de pacotes, crie os *scripts* de configuração e compilação.

Agora, quanto à pergunta qual função ou módulo começar, eu sempre começo com o mais crítico, ou seja, o mais importante, antes de tudo. Como já citei anteriormente, utilizo uma abordagem *SCRUM* de *sprints* para gerenciar o projeto. E, os criadores recomendam que você comece pela parte mais importante e visual para o cliente, por assim dizer. Por exemplo, vai fazer um sistema de *ecommerce*, comece pela tela de pedidos! Faça a tela, insira manualmente os dados de teste antes mesmo do CRUD, e mostre-a para o cliente, deixe ele cadastrar um pedido.

Então, recomendo que você comece pela função mais importante para o seu cliente e a partir dela, passe para as acessórias e os CRUDs. Mas, não precisa ser rígido. Cada projeto terá um começo mais recomendado, dependendo da urgência e do cliente. Se for adotar os *sprints*, separe-os por funções importantes.

8.9 - TEMPLATES DO PROJETO

Os programadores mais experientes possuem *templates* de projetos. Alguns elementos são comuns a todos os projetos do mesmo tipo. Não há necessidade de se iniciar projetos similares sempre do zero. Por exemplo, se vai desenvolver um sistema web em *Django*, a estrutura básica é a mesma, então precisa sempre iniciar todos os sistemas do zero?

Eu utilizo gerenciadores de pacotes, como o *npm* para instalar o projeto novo. Como disse, a estrutura básica utilizada por você é a mesma, por exemplo, estrutura de pastas, compilação de CSS, as bibliotecas de interface, segurança, log, mecanismo de testes, *coverage*, *linter*, etc.

Até algumas telas e funções são as mesmas. Por exemplo, quase todos os sistemas do estilo intranet tem controle de acesso, então, tais mecanismos podem ser reutilizados, evitando retrabalho. As telas de CRUDs (*Create, Read, Update and Delete*) também são do mesmo tipo e às vezes não muda quase nada.

Quando você for mais experiente, terá seus próprios *templates*. Em *Django*, eu utilizo o [CookieCutter](#). Nele posso iniciar um projeto já com os dados reais utilizando *templates* criados. Busque por *templates* no *GitHub* ou no *Google*, e adapte-os para o seu uso. Aqui tem um bom exemplo de [template](#) para criar uma API.

Busque *templates* para sua linguagem ou *framework* para facilitar seu trabalho e vá criando os seus.

Obviamente que este capítulo não tem o escopo de debater assuntos tão complexos como gerenciamento de projetos, arquitetura e engenharia de software, pois cada um daria um livro grande, e mesmo assim, não abarcaria toda a complexidade dos respectivos temas.

Recomendo que não aceite projetos grandes no começo. Comece pequeno e vá crescendo. Se quiser se aprofundar mais, precisará ler, estudar e praticar muito. Se você estiver cursando ou for cursar análise de sistemas ou qualquer curso do tipo, você aprenderá sobre estes temas e terá oportunidade de estudá-los mais a fundo com o acompanhamento dos

professores. Mas, caso você não esteja cursando ou não vá cursar a universidade, recomendo a leitura dos livros indicados acima e a assistir cursos introdutórios sobre o tema. Procure um bom no *Udemy* ou *Udacity* (escolha bem e de professores experientes).

CAPÍTULO 9 - SOU PROGRAMADOR, E AGORA, COMO ALAVANCAR MINHA CARREIRA

“A maneira de se conseguir boa reputação reside no esforço em se ser aquilo que se deseja parecer.”

Sócrates

Estamos na era dos *influencers*. A internet possibilitou o maior compartilhamento de ideias e removeu as limitações geográficas. Hoje você pode acompanhar o trabalho de um grande programador que mora do outro lado do mundo em tempo real, acompanhando as suas redes sociais, tanto pessoais quanto profissionais, bem como interagir com ele.

Se você quer se tornar conhecido na sua área, deve compartilhar muito conhecimento e o seu trabalho. Os seus resultados falam mais alto que as suas palavras.

9.1 - ÉTICA PROFISSIONAL

É desnecessário lembrar o caro leitor de que a ética é primordial para uma carreira bem sucedida, contudo, nos dias de hoje, infelizmente, o óbvio precisa ser dito e reafirmado diariamente.

Já me deparei com profissionais e até clientes antiéticos, com práticas questionáveis, sem qualquer respeito para com o próximo. Um cliente a um tempo atrás encomendou-me um algoritmo complexo para realizar operações no mercado de *Forex*, do qual ele iria comercializar assinaturas para o uso, e altamente parametrizável.

Na época, gostei do projeto, cobrei o justo e criei toda a estrutura, *front end* do gerenciador na *web*, *back end*, os robôs rodavam da nuvem, direto do *DigitalOcean*, rodando redondinho, contudo, o cliente me pediu para inserir no robô algumas diretivas que copiavam os dados, operações de clientes e enviavam para ele, além de configurações do tipo *Black Box* que podiam prejudicar os clientes dele. Não eram meus clientes.

Eu podia não estar nem aí. Era um projeto de R \$15 mil, que estava pronto, mas me recusei a implementar as mudanças antiéticas. Eu expliquei ao cliente que isto não era correto e violava diretrizes internacionais de compartilhamento de dados e de confiança e privacidade dos clientes dele.

Não adiantou, este meu ex-cliente, cancelou o projeto e não me pagou o restante do combinado, alegando que eu me recusei a concluir. Na verdade, não, já que não estava no escopo do projeto. Não processei o cliente, pois, de pessoas assim quero distância.

Na sua carreira, você irá se deparar com clientes assim, infelizmente. Mas, não venda sua alma ao diabo! Não traia seus princípios por dinheiro, jamais. Durma tranquilo. Jamais perturbe seu sono ou sua vida com atitudes ilícitas ou antiéticas.

Respeite outros profissionais e colegas. Uma pessoa honrada e ética, justa, case nunca sai perdendo. As pessoas costumam recompensar e reconhecer a honestidade.

Seja transparente com seus clientes e parceiros. Já está cheio de profissionais com caráter discutível por aí, o mundo não precisa de mais.

Certa vez ouvi um ditado muito legal (desconheço a fonte): “*Se o malandro soubesse que ser honestidade dá lucro, seriam honestos por malandragem*”

9.2 - NÃO À PIRATARIA

Não vou ser hipócrita. Já baixei músicas e programas piratas. Hoje, não faço mais. Só utilizo softwares originais. Sim, alguns são caríssimos. Meu Windows custou R\$ 500 versão OEM, pois eu monto as minhas próprias máquinas, e a do meu *Notebook* veio com ele.

Utilizo IDEs profissionais pagas e todas possuem licenças apropriadas. Respeite o trabalho dos outros. Há diversas opções gratuitas para serem usadas, caso você não esteja disposto a pagar, ou simplesmente não tenha recursos. Para ser realista, você não precisará de *software* pago no começo, somente quando sentir a real necessidade de um, compre a licença.

Quando parei com o péssimo hábito de utilizar jogos, *softwares* e sistemas piratas, as coisas mudaram. Não sei se tem a ver, mas acredito muito em karma.

Imagine quando você começar a vender seu *software* ou suas licenças, irá gostar que outros desrespeitem seu trabalho com pirataria? Vamos valorizar o trabalho alheio.

9.3 - HACKERS

Os temidos *hackers* são conhecidos por serem criminosos digitais. Na faculdade, aprendi que um *hacker* é um profissional brilhante. E, como todas as pessoas brilhantes, podem usar seus conhecimentos para o bem ou para o mal.

Quando eu aprendi C e *Shell Script*, comecei a estudar o *Kernel* do *Linux*, e algumas distribuições como o *Debian* e *Ubuntu*. Escrevi alguns *scripts* para explorar vulnerabilidades, mas muito bobos, sem qualquer pretensão de prejudicar.

O que a mídia ou a polícia chama de *hacker* é o indivíduo com profundos conhecimentos em *Assembly* (Linguagem de máquina), protocolos de rede como HTTP, FTP, SSH, além de serem especialistas em C, *Shell Script* e arquitetura de sistemas operacionais. Estou dizendo os criadores dos *scripts*, vírus, *exploits*, etc, e não os meros criminosos utilizadores.

Há uma classe especial de criminosos que compram aplicativos destes programadores *hackers* para utilizarem nos seus golpes. Estes bandidos não são *hackers*, pois sequer devem saber programar. Uma analogia básica, o *hacker* cria a arma, e eventualmente pode usá-la, o bandido compra a arma e a usa para assaltar. Claro, que só criar uma arma com fins ilícitos já é crime.

Quando vejo no Jornal “*Hackers roubam dados de celulares ...*”, ou coisa parecida, vejo os “*hackers*” que não sabem nada de programação nem rede ou segurança digital. Na maioria das vezes são bandidos que compraram a ferramenta de um verdadeiro *hacker*. Encontra-se qualquer tipo de programa invasor no mercado negro (*Deep Web*).

Difícilmente, a polícia encontra um *hacker* na rede ou digitalmente. Só amadores são pegos. Mas, como então a polícia às vezes pega um *hacker*? *Follow the money*! Eles pegam os caras pelo dinheiro ou quando vão fazer entregas físicas, ou coisa parecida.

Um famoso *hacker* russo foi pego porque gastava muito dinheiro, e qual era a fonte do dinheiro? A própria polícia sabe que é quase impossível encontrar um *hacker* pela internet, pois eles cobrem muito bem seus rastros,

e às vezes, estão do outro lado do mundo, fora da jurisdição. Como exigir o log de acesso de um servidor que está na China? A polícia brasileira não tem jurisdição lá, então dificulta a localização de um verdadeiro *hacker*, e os meios burocráticos demoram demais, então dá tempo para apagar os rastros, ou encobri-los muito bem.

Hacker são especialistas em segurança digital. Existem alguns que usam seu conhecimento para o bem. Fiquei sabendo de um *hacker* chinês que ganha dinheiro informando às empresas falhas de segurança em seus serviços. Dá para ganhar uma boa grana, mas tem que ser um gênio, praticamente, estudar a fundo os protocolos e arquitetura de redes. Não é para mim.

Você é brilhante, um excelente programador ou tem um grande potencial? Recomendo que use seus talentos para o bem. Lembrando o “Tio Ben”, do Homem-aranha: “*Grandes poderes trazem grandes responsabilidades*”.

9.4 - MELHORE A VIDA DAS PESSOAS

Crie soluções para melhorar a vida das pessoas para projetos pagos, ou até para projetos gratuitos.

Sempre que sou convidado para participar de um projeto, verifico de que se trata, qual a solução e para quem. Certifico-me de que aquela ideia ou solução vai ajudar na vida das pessoas. Sim, isto é ideológico. Você pode não ser ideológico, ou estar cagando para as pessoas, querendo apenas a sua grana. Se este for seu caso, desculpe, pois este livro será inútil para você.

A melhor forma de alavancar a sua carreira é sendo útil para as pessoas. Certa vez, ouvi a história de uma loja de manutenção da Apple, em São Paulo, que era conhecida por sua honestidade e transparência.

Uma cliente teria chegado com um *Iphone* com um probleminha muito simples: sujeira na entrada do cabo de carga. Outras lojas haviam cobrado muito caro para resolver o problema dela. Ela foi nesta referida loja, e lá, o técnico olhou, viu o problema, pegou um pincel, limpou a entrada de carga e pronto, funcionou!

Ela perguntou quanto, e quando o técnico respondeu nada! Ela ficou abismada, e feliz, compartilhou nas suas redes sociais. A história viralizou e hoje em dia a lojinha se transformou em uma rede com fila para atendimento. Sim, atender bem, com honestidade dá muito lucro.

Alguns podem exclamar: ah, e meu trabalho, preciso pagar os boletos! Claro, todos nós precisamos pagar as contas. A sociedade é uma máquina de trabalhar para pagar contas. Parece que tudo gira em torno de consumo e pagamento dos boletos. As pessoas e profissionais estão automatizados, viraram robôs, que pouco se importam com o próximo.

Há tantas histórias de pessoas e profissionais que criaram soluções que ajudavam as pessoas e hoje estão riquíssimas, ou são reconhecidas e vivem muito bem. Eu prefiro seguir o que deu certo a seguir a boiada ou como todos fazem. Penso que se você fizer como todos, terá o mesmo resultado: aposentadoria do governo no final da vida.

Se você quer algo a mais, precisa pensar em ajudar as pessoas com o seu trabalho e talento, mas, claro, sem esperar a recompensa, senão não seria ajuda, mas troca de interesses.

Então, pense, como você pode melhorar a vida das pessoas por meio da tecnologia, aplicativos, programas ou serviços?

9.5 - VALORIZE SEU TRABALHO

Anteriormente, citei que você poderia trabalhar de graça, ou até doar seu tempo. Não entenda mal. Se você doa seu tempo e talento não está desvalorizando seu trabalho. Pelo contrário. Está sendo valorizado pelas suas ações.

Você se desvaloriza quando cobra abaixo do justo. Aceita receber menos do que merece pelo seu empenho, ou se submete ao trabalho indigno.

Estabeleça seu preço. Mas como calcular o preço do meu trabalho? Eu utilizo uma fórmula legal. Vamos lá. Quanto você quer ganhar no mês? 5 mil, por exemplo. Quantas horas de trabalho você quer dedicar na semana para seu trabalho? vamos supor que seja 30 horas. Então temos 30 horas semanais x 4 semanas é igual a 120 horas. Dívida R\$ 5 mil por 120 dá o valor de R\$ 41,66 (você arredonda para R\$ 42). Você pagará o imposto de renda e o ISS, se se registrar para emitir nota fiscal eletrônica (o ISS você acrescenta para o seu cliente pagar). Para trabalhar para empresas, provavelmente, você deverá criar o registro de profissional autônomo e emissor de nota fiscal eletrônica de serviços. Verifique na prefeitura de sua cidade as regras e se ela disponibiliza esta licença. As empresas costumam exigir nota fiscal de serviços.

Para calcular quanto tempo você levará para concluir um projeto, aí demanda experiência. Faça os primeiros projetos e meça quanto tempo gastou para concluir um módulo, por exemplo. Em quanto tempo você criar um *Plugin* simples para *WordPress*? Depende. Cada um tem seu ritmo. Descubra o seu.

Estabelecido seu preço, não trabalhe por menos. Os clientes vão chorar, pedir para reduzir, mas valorize seu trabalho. Claro que você não precisa ser tão inflexível. Cada caso é um caso. Mas se tiver muitas exceções, passa a ser regra. Valorize-se.

9.6 - FAÇA O SIMPLES

É muito importante ressaltar que o leitor deve se ater ao simples. Diga não ao complexo e complicado.

Vamos a um exemplo: Converter uma string para o estilo *Camel Case*.

Código Simples:

```
function toCamelCase($str){
    return preg_replace_callback("~[_-](\w)~", function($m) { return
        strtoupper($m[1]); }, $str);
}
```

A mesma função escrita com mais linhas:

```
function toCamelCase($str){
    $explodetArray = multiexplode(['-', '_'], $str);
    $count = count($explodetArray);
    for($i = 1; $i <= $count; $i++) {
        $explodetArray[$i] = ucfirst($explodetArray[$i]);
    }
    return implode('', $explodetArray);
}

function multiexplode ($delimiters, $string) {
    $ready = str_replace($delimiters, $delimiters[0], $string);
    $launch = explode($delimiters[0], $ready);
    return $launch;
}
```

Neste exemplo temos dois códigos com o mesmo resultado. A primeira versão é sucinta, não usa variável, e usa expressão regular com uma função de retorno. Solução simples. No segundo código, utiliza duas funções separadas, laço de repetição for, que demanda mais processamento, bem com variáveis de 32 bytes, consumindo muito mais recursos.

A primeira solução é elegante e consome poucos recursos, sendo fácil de debugar se for o caso. A segunda é mais custosa.

Você aprenderá na disciplina algoritmo a importância de medir o custo do algoritmo bem como a sua eficiência. Quando você escreve um

código para ser compatível com máquinas mais simples, ou até celulares com pouca potência, deve escrever códigos leves, simples e que consumam muito pouca memória e processamento.

Por isso a importância de aprender algoritmo. Para evitar escrever códigos longos e deselegantes e escolher a melhor e menos custosa solução.

Por exemplo, se você precisar de um algoritmo de ordenação. Temos o *Bubble sort*, *Selection Sort*, *Insertion sort* e *Quick sort*. Há outros, mas estes são os mais famosos e utilizados. Qual escolher? Depende do seu uso e tamanho dos dados a ordenar. Estudando algoritmos, você aprende a escolher os melhores algoritmos para cada situação, e a mensurar o consumo de recursos de seus próprios algoritmos.

9.7 - SIGA AS BOAS PRÁTICAS DE PROGRAMAÇÃO

Escreva códigos limpos, seguindo as boas práticas de programação. *Robert C. Martin*, em seu livro *Clean Code: A Handbook of Agile Software Craftsmanship*, destaca boas e más práticas de programação. Fala sobre correto tratamento de erro, comentários e documentação objetiva do código, definição de nomes, funções e classes, bem como uso de concorrência, ou seja, processamento paralelo.

É um excelente livro de leitura obrigatória para um bom programador. Tem edição em português se o leitor preferir.

Quando vou avaliar um código de outros programadores, em geral, vejo diversos erros e más práticas de programação, como uso incorreto ou desnecessário de variáveis, estruturação de dados equivocada, nomes de procedimentos ou funções dúbias, sem comentários, com comentários não informativos ou incorretos, dentre outros.

Para ser sincero, eu já cometi muitos desses erros. Quando pego um código meu escrito há 15 anos, tenho vergonha de mostrar para um programador experiente.

Em *Python* há ferramentas que auxiliam na tarefa de manter o código limpo. Há por exemplo o *PyLint*, um pacote chegador do código fonte, buscando erros, incoerências, incorreções ou qualquer termo em desacordo com os melhores padrões de desenvolvimento. O termo “*Lint*” surgiu por volta de 1978 como um chegador de programas escritos em C. *Lint* quer dizer “fiapo”. Podemos dizer que ele cuida da estilização do código. Para que ele fique elegante e evite erros ou *bugs*.

9.8 - ESCREVA TESTES PARA SEUS CÓDIGOS

Eu tenho vergonha de assumir que já fiz muitos programas sem escrever testes. Putz. Eu achava desnecessário ou perda de tempo.

Pura ignorância minha. Testes são essenciais. Hoje em dia, graças ao Mestre *Harry Percival*, eu sigo o desenvolvimento orientado a testes TDD, onde primeiro se escreve os testes, para depois se escrever o programa. Sei que parece loucura, mas funciona, poupa muito tempo e trabalho, além de ajudar o CD/CI em *DevOps*.

Já evitei muita dor de cabeça testando códigos de forma automatizada. Hoje as coisas são tão fáceis. Eu commito o código, e no *GitHub*, tem uma *action* onde o *Travis* (serviço de automatização de testes) é acionado e testa todo o código utilizando o pacote *PyTest*, para meus códigos em *Python*.

E, após os testes passarem, por vezes, leva horas para testar todo o código, o *Travis* envia o código para o servidor de implantação *Jenkins* que já distribui a nova versão testada, sem preocupação, e se caso algo falhar, ele retorna para a versão estável anterior, automaticamente. É outro mundo!!!

Não precisa seguir a metodologia TDD, mas escreva testes mínimos para seus códigos em qualquer linguagem. Seja profissional.

9.9 - FAÇA PROGRAMAS SEGUROS

Seguindo as boas práticas de programação, você terá um código robusto e seguro. Contudo, é bom sempre lembrar e se atentar à segurança da informação.

Falei anteriormente para utilizar somente linguagens robustas e seguras que estão em constante atualização e otimização. Assim como os pacotes de terceiros que você usa em seus softwares. Mantenha tudo em ordem e atualizado.

Tome todas as precauções possíveis em seus programas para não expor os dados de seus clientes. Existem serviços de monitoramento de código constante para verificar tanto a performance quanto a estabilidade e segurança do sistema. Alguns são pagos, mas existem boas opções gratuitas.

Cuide bem dos dados. Trate-os conforme as melhores práticas. Estude bastante a estrutura de dados e projete seu software de forma robusta e segura, tratando os dados de forma adequada.

Utilize sempre que possível *frameworks* com ênfase em segurança e mantenha-os atualizado, observando os cuidados com incompatibilidades.

Mantenha o log de erros, se possível de acessos e alterações de dados. Pode-se utilizar bons serviços de terceiros. Isto facilita a auditoria de um sistema em caso de falha ou violação de dados.

9.10 - TRATE TODAS AS POSSÍVEIS FALHAS

Sim. Nossos códigos falham. E, falham com frequência. Ainda mais em ambientes complexos como a web. A internet do cliente pode cair ou ficar instável. Ele pode digitar algum dado incorreto.

Tantas coisas previsíveis e imprevisíveis podem acontecer? Tantas possibilidades, que é quase impossível prever todas. Porém, você deve tratar as possíveis falhas previsíveis e estar preparado para falhas não previsíveis. Putz, sei como isso é difícil. Só testes massivos vão revelar as falhas ocultas do seu código.

Já peguei códigos de funções importantes como processamento de pagamento no qual o desenvolvedor anterior não havia colocado um tratamento de falha. Ou seja, se desse algo errado, perdia-se a transação completa ou ficava-se sem saber o que tinha acontecido.

Por isso acho importante ensinar aos novos programadores a tratar todos os erros.

Ainda hoje pego alguns códigos Python escrito desta forma:

```
try:
    processPayment()
except:
    pass
```

Isto é foda! Ou seja, um erro passa em silêncio, ou seja, é totalmente ignorado. No bloco de código acima, temos o comando “*try*” que abre um bloco de código onde qualquer comando executado dentro dele que tiver alguma exceção, será tratado pelos termos “*except*” na ordem que forem apresentados.

Neste caso o bloco chama a função *processPayment*, e se ela retornar um erro, vai ser executado o *except*, que retorna em branco. O termo *pass* em *Python*, grosseiramente, quer dizer passe e não faça nada. Como *Python* é obrigatoriamente indentada, todo bloco deve haver código ou o termo *pass*.

A simples alteração abaixo, mesmo que de forma sucinta e ainda não satisfatória, já não deixaria o erro passar em branco, pois registraria o erro em um log para que o administrador do sistema ou o desenvolvedor pudesse tomar conhecimento dele:

```
try:
    processPayment()
except Exception as e:
    logger.error('Function error occurred in
processPayment -> {}'.format(e))
    return False
```

Eu não faria assim. Em geral, eu implemento classes para tratamento de erros conhecidos e possíveis, com a devida resposta, ou registro em log os erros desconhecidos e imprevisíveis, com o devido tratamento, ou ao menos uma mensagem para o usuário.

9.11 - ENCANTE SEUS CLIENTES

Tratar seu cliente com cordialidade, honestidade, transparência e eficiência é primordial. Porém, não é isto que vemos por aí.

Eu mesmo já fui enrolado por outro *freelancer*. Contratei outro programador para me ajudar a concluir um projeto urgente. Conteí com o cara, mas ele deu no pé. Sumiu, não dizendo mais nada.

Você deve colecionar clientes satisfeitos e que fariam negócio com você novamente. Rastro de clientes insatisfeitos pode te trazer péssimos resultados no futuro.

No exterior, os desenvolvedores brasileiros têm péssima reputação. Certa vez, um cliente americano me perguntou se eu não ia sumir. Fiz questão de atendê-lo de forma rápida, eficiente e eficaz. Temos que mudar este jogo. Nem todos são safados.

Então, certifique-se de que vai dar conta do projeto e que vai entregar no prazo. Seja transparente com seu cliente. Se for precisar de mais tempo, comunique-o.

Faça entregas parciais, se for possível. Certifique-se de que sua solução atende ao problema do cliente. Lembre-se, você não foi contratado para programar, mas sim para resolver um problema.

Entregue mais do que foi contratado. Faça mais e melhor. Não seja mesquinho. Lembre-se de porque você está fazendo.

9.12 - DOE SEU TEMPO E TALENTO

Participe de projetos *open sources*. Existem vários entre várias linguagens. Eu faço parte do *JazzBand*, que cuida de diversos projetos em *Python*, muitos com milhares de usuários. Aprendi muito com aquelas pessoas e escrevendo projetos *open sources*. Eu participo também do desenvolvimento core do *Django*, entre outros projetos pequenos.

Você não receberá dinheiro pela sua participação. Mas, garanto que é gratificante ver a sua criação usada por outras pessoas e ajudando ou melhorando algo.

O caro leitor pode pensar: poxa, estou iniciando a carreira, sem dinheiro e com tempo reduzido, por que doaria meu tempo para participar de projetos de código abertos? Porque isto trará muitas recompensas materiais e reconhecimento para você. Ser um dos desenvolvedores que criaram um aplicativo ou pacotes utilizados por empresas ou outros programadores é recompensador.

Um dos colegas destes projetos que trabalha em uma grande empresa americana de *software*, me disse que essa empresa escolhia os seus desenvolvedores entre os que doaram seu bem mais precioso que é o seu tempo para projetos de códigos abertos, como uma dinâmica de seleção, e de quebra eles mediam as qualificações do candidato. Ele mesmo foi selecionado desta forma.

Se você for participante do núcleo de qualquer tecnologia com créditos, poderá utilizar isto em seu currículo, o que será um diferencial, além de desenvolver muito as suas habilidades de programação, já que para escrever códigos complexos requer muito treinamento.

Eu já fui contratado como consultor de um site que utiliza *Django* para implementação de processamento assíncrono. Ou seja, a sua participação no desenvolvimento de uma tecnologia pode te render trabalho, dinheiro e reconhecimento.

Mas, não participe unicamente por isto. Pense de fato em ajudar. Esteja disposto a dar para receber. Eu acredito nesta lei do universo.

Não conheço outra forma mais eficaz de ganhar reconhecimento por seu trabalho senão compartilhando com o mundo as suas criações.

Claro, que precisamos comer. Mas pense em ajudar também. Quem sabe você não se dedique a um projeto paralelo de um aplicativo na área que atua para distribuir gratuitamente? Mas de graça! sim, doe, compartilhe. Você verá o quanto isto retornará em dobro com oportunidades de trabalho e projetos.

9.13 - COMPARTILHE SEU CONHECIMENTO

Escreva artigos e publique-os no seu blog/site ou em blogs parceiros. Há diversas plataformas para publicar textos técnicos e reflexivos. Não esqueça de publicar textos no *LinkedIn*, além de participar de eventos e grupos de discussão.

Participe de eventos, e se possível como expositor ou palestrante. Sim, é possível, mesmo não sendo graduado, basta você ter muito conteúdo para compartilhar e ensinar.

Há também a possibilidade de publicação de vídeos tutoriais no *YouTube*, mesmo em fase de aprendizagem. Fica a seu critério, mas acredito que você tenha entendido a ideia central que é ser visto, e claro, bem visto, com boas informações e boas práticas, associando-se à qualidade e presteza no trabalho, além de qualificação técnica.

9.14 - TENHA PACIÊNCIA!

O principal é ter paciência e não visar unicamente dinheiro. Se você faz o que gosta, o sucesso virá com o tempo. É só plantar as sementes. Não seja afobado.

Por isso escrevi o primeiro capítulo, para você descobrir se realmente gosta disso, ou se é o que quer fazer o resto da vida. Se você for apaixonado pelo seu trabalho, o fará com tamanho zelo e dedicação, que estará plantando as sementes certas, e certamente colherá no futuro bons resultados.

Não faça uma boa ação esperando ou visando resultados materiais. É gratificante saber que você melhorou a vida das pessoas, ou que criou algo de útil para o mundo.

9.15 - APRENDA A LIDAR COM DINHEIRO

Putz, essa não tem a ver diretamente com programação pois é aplicável a qualquer profissional. Existem programadores pobres, assim como Médicos que vivem para pagar boletos e que são endividados até o pescoço. Ter casa bonita, carro foda não é ser rico.

Aprendi isto com o Livro Pai Rico, Pai Pobre, de Robert Kiyosaki. Recomendo a leitura para aprender a crescer e ser rico de verdade.

Para mim a verdadeira riqueza é acordar onde você quiser, ficar quanto tempo quiser e com quem quiser. Nada pior do que ser obrigado a ficar ou trabalhar com pessoas desagradáveis ou que você não goste.

Em resumo, para mim, riqueza é liberdade! A programação me trouxe isto.

O que é riqueza para você? Não seja mais um na multidão. Tenha escolha. Escolha ser livre e feliz.

9.16 - APERFEIÇOAMENTO CONSTANTE

Você nunca para de estudar. Sei que já falei isso várias vezes, mas é bom lembrar sempre. Esteja atento aos releases das tecnologias que você usa. Leia muito e sempre sobre a linguagem que você utiliza, e de preferência de diversos autores.

Assine o *newsletter* da tecnologia, e se possível entre no grupo de *email* dos desenvolvedores ou de usuários avançados para saber as futuras mudanças que virão.

Atente-se à compatibilidade das tecnologias, e os reflexos. Comunidades com *Stackoverflow* falam muito sobre isto. É bom estar atento.

Ah! Um conselho que aprendi após apanhar muito: Não aplique em seus projetos ou refatore com *releases* novos, ou atualizações de versões, pois eles costumam vir com alguns bugs, ou incompatibilidades com pacotes ou tecnologias atuais.

Fiz uma cagada das grandes uma vez com um site de cliente que me rendeu uma multa e um cliente insatisfeito. Um *release* do *Python 2.x* com o *Django 1.x* que bagunçou todo o coreto, tudo parou de funcionar e o site ficou fora do ar por horas. E os testes foram bem, mas quando foi para os dados de produção, caiu tudo.

Estude o *release*, entenda, faça testes. Só altere projetos existentes após muitos testes e se realmente for necessário. Se por exemplo um *release* de uma linguagem corrigir uma falha grave de segurança, aí é prioritário, então faça com calma e com muito cuidado.

Faça testes com dados reais em ambiente controlado. Em geral, faço uma cópia do banco de dados para um servidor na nuvem especial para testes, e teste com os dados reais.

Há excelentes serviços para gerenciamento de logs e acompanhamento de execução de códigos e implementações com *dashboards* onde o desenvolvedor pode acompanhar o desempenho do seu código.

9.17 - CUIDE DA SAÚDE

Tive um conhecido que perdeu a vida com apenas 36 anos. Ele teve um ataque cardíaco. Ele pesava mais de 180 kg. Era um excelente programador. O mundo perdeu o seu talento.

Eu não estou saudável. Adoro fazer caminhadas, mas não tenho me exercitado muito. Estou pesando 120 kg, o que já é perigoso para quem mede 1,79m. Vou seguir meu próprio conselho. Emagrecer e ser mais saudável.

Beba bastante água. Eu quase tive problemas com meus rins, porque bebia pouca água durante minhas sessões *hardcores*. Fico horas sentado em frente a 3 telas de computador e meu notebook, o que também pode ser prejudicial.

Para evitar problemas com a coluna, eu fico bastante de pé. Comprei uma mesa móvel, que se eleva ou abaixa, para trabalhar em pé.

Não uso óculos porque cuido bem dos meus olhos, embora fique mais de 12h por dia de olhos em telas.

Caminhe bastante, se movimente. Faça uma caminhada pela manhã, e sentirá a diferença na hora que se sentar para escrever seus códigos, pois, ficará mais disposto, alerta, pensando mais rápido.

Alimente-se bem. Consulte o médico com regularidade. Tenha uma nutricionista para te ajudar na alimentação.

Recomendo a você contratar um *personal trainer* para te fazer exercitar. Gostei da experiência.

9.18 - PRODUTIVIDADE

O mundo está cada vez mais rápido e dinâmico. Acompanhar tudo está cada vez mais difícil. Programadores lidam contra o tempo. Os projetos devem ser escritos cada vez mais rápidos.

Recebo projetos que devem ser escritos em dias e até horas. Devemos estar constantemente otimizando a nossa forma de desenvolver e criar.

Os *frameworks* ajudam nesta tarefa, agilizando o desenvolvimento. Eles reduzem cada vez mais a quantidade de linha de código necessária para um projeto. Incrível evolução.

Hoje, você pode escrever e lançar um site em horas. A metodologia *SCRUM* tem os seus *sprints*, dos quais se conclui projetos complexos de forma mais eficiente e eficaz.

Você deve estar atento a sua produtividade. Você trabalha melhor pela manhã ou à tarde? Eu, por exemplo, sou matinal. À tarde eu não produzo muito, por ficar mais lento. Meu cérebro entra no modo preguiça. Por isso deixo as tarefas mecânicas para a parte da tarde. Quando preciso ser criativo e pensar, faço na parte da manhã.

Há diversos cursos de produtividade e gerenciamento de tempo, bem como diversas metodologias. Eu já fiz alguns, e para ser sincero, não consigo ser tão metódico, como por exemplo ter uma agenda detalhada, ou algo do tipo.

Prefiro a metodologia que me foi apresentada pelo *coach* Geronimo Theml, em seu livro produtividade para quem quer tempo, de divisão de blocos de tempo.

O importante é você se conhecer. Saber o que funciona ou não para você. Não existe fórmula mágica ou métodos que funcionem com todos. Teste e veja qual se adequa melhor ao seu perfil.

CAPÍTULO 10 - É SÓ O COMEÇO DE UMA LONGA JORNADA

“Deus nunca disse que a jornada seria fácil, mas Ele disse que a chegada valeria a pena.”

Max Lucado

Espero de coração que este pequeno livro tenha ajudado ao leitor neste começo de jornada para uma carreira excepcional. Expus aqui quase 20 anos de experiência de aprendizado e conhecimento.

A minha intenção ao escrever este livro jamais foi assustar ou desestimular ninguém. Apenas, mostrar ao caro leitor que é possível ter uma carreira brilhante. Vemos diariamente pessoas apáticas, vivendo feito robôs, indo de um lado para o outro, sem qualquer esperança de uma vida melhor. E, não se engane achando que somente pessoas que fazem trabalhos mais triviais são assim. Já cruzei com programadores, engenheiros, matemáticos e estatísticos com o mesmo olhar apático e sem vida. Talvez fossem muito bons, e alguns realmente eram, mas as escolhas e ações que os levaram até aquele ponto não os conduziram ao que realmente desejavam. Não culpe a vida, o sistema, o país, o governo corrupto, muito menos seus pais ou sua origem. Quer achar um culpado? Olhe no espelho. Todos temos escolhas. Na verdade, se abster de escolher já é uma escolha. Aceitar a sua situação atual, ou acreditar na falácia de que sua origem ou por ser pobre o impedirá de crescer é escolher ser pequeno. Quando você escolher ficar ao celular acompanhando a vida dos outros ao invés de estudar e praticar, está escolhendo ser medíocre. Estamos cultivando uma sociedade de medíocres.

Espero que o leitor perceba que pode ter uma grande carreira e que isto depende apenas de você e de mais ninguém. A vida é cruel e certamente te dará umas porradas de vez em quando, mas cabe a você ser forte para aguentar e ficar mais forte ainda. Gosto de seguinte trecho (desconheço o autor): *“Já viu mar calmo formar bom marinheiro?”*

Imagino que o jovem leitor deve estar afoito, louco para começar um grande projeto, querendo aprender *React*, *Vue*, *Angular*, criar um *site* foda e ganhar muito dinheiro. Eu passei por esta fase, então, sei como é. A ansiedade de começar logo, a afobação. Mas, vá com calma. Siga os passos corretos.

Vá adquirindo as habilidades necessárias, lendo bastante, se aprimorando como pessoa e como profissional.

Espero que o leitor esteja seguro da sua opção. Segundo a Professora Lúcia Helena Galvão, a sociedade atual tem dúvidas de tudo. Você não pode ter dúvidas de onde quer chegar, ou não chegará a lugar nenhum.

Ao longo das linhas, fui sincero com o leitor e compartilhei meus pensamentos e reflexões acerca da conduta atual da profissão.

Cada vez mais somos demandados. A sociedade precisa de nós, programadores. Nós fazemos parte da revolução digital que está ocorrendo. É o nosso dever fazer a nossa parte para termos um mundo melhor.

Torço para que este livro tenha ajudado o leitor a se encontrar, e quem sabe, ser um dos programadores que vão melhorar as vidas das pessoas.

Em um mundo tão líquido, confuso e caótico, é difícil ter serenidade, pensar com calma. Somos atacados de todos os lados com informações falsas e verdadeiras, estímulos de compras, propagandas, promoções. É uma luta diária para concentrar-se em algo de bom e fazer a diferença. Vou compartilhar com o leitor uma oração que eu recito quase todos os dias:

*DEUS,
Deus, conceda-me a serenidade
para aceitar as coisas que não posso mudar,
a coragem para mudar as coisas que posso
e a sabedoria para discernir uma da outra.
Vivendo um dia de cada vez,
apreciando um momento de cada vez,
recebendo as dificuldades como um caminho para a paz
e, como Jesus, aceitando as circunstâncias do mundo
como realmente são, e não como gostaria que fossem.
Confiando que o Senhor tudo fará
se eu me entregar à Sua vontade;
pois assim poderei ser razoavelmente feliz nesta vida
e supremamente feliz ao seu lado na eternidade.
Amém.*

Aguardo o leitor para vir comigo fazer um mundo melhor, contribuindo com seu tempo e talento. Se todos fizermos nossa parte, da melhor forma possível, caminharemos juntos para um futuro mais aconchegante e acolhedor.

Seja um programador foda! Tenha uma carreira brilhante! Nos encontramos por aí. Terei um imenso prazer em apertar a sua mão.

Agradeço muito a sua atenção e por ter lido este humilde livro.

Até a próxima, e boa sorte na sua jornada!